Mastering Statistical Process Control Charts in Healthcare

A practical, hands-on, step-by-step guide for data scientists using R

Jacob Anhøj & Mohammed Amin Mohammed

2026-08-21

Contents

What is Statistical Process Control?

> This is the PDF version of **Mastering Statistical Process Control Charts in Healthcare**, a book under continuous development.
>
> The latest online version is available at https://spc4hc.org/book/.

The ultimate purpose of collecting and analysing data is to support better decision-making and actions that will lead to improvement. Statistical Process Control (SPC) is a proven methodology for doing just that.

SPC methodology provides a philosophy and framework for continually learning about the behaviour of processes for analytical purposes – where the aim is to act on the underlying causes of variation to maintain or improve the performance of a process.

SPC was initially developed by Walter A. Shewhart in the 1920s to improve the quality of manufactured products and has since been successfully used in many settings including healthcare.

At is core, SPC methodology involves the plotting of data over time to detect unusual patterns or variations that might indicate a change or problem with a process. This simple graphical device is underpinned by an intuitive theory of variation, the hypothesis-generating testing-cycle of the scientific method, and statistical theory.

To master SPC, it is essential to first understand the underlying theory of variation, which we explore in Part 1 of this book. Next, proficiency in using software to construct SPC charts is key, and this is covered in Part 2. In the later sections, we delve into specialised topics for more advanced SPC practitioners, offering deeper insights and expertise.

Preface

This book is about making better decisions in healthcare using data. Every day, healthcare teams track data: waiting times, infection rates, mortality, patient experience. These data go up and down. The challenge is knowing when and how to act.

Act too quickly, and you risk chasing noise. Wait too long, and you may miss an important signal to improve care. Take the wrong action and you risk making things worse.

Statistical Process Control (SPC) provides a practical way to navigate this uncertainty.

At its core, SPC is built on a simple but powerful idea: all processes vary, and that variation arises from two fundamentally different sources: **common cause** and **special cause**. Understanding this distinction is crucial to help us interpret data more reliably and act more effectively.

SPC brings these ideas to life through simple visual tools: **run charts** and **control charts**. Yet despite their conceptual simplicity, applying these tools correctly can be challenging. As noted in a systematic review of SPC in healthcare:

> … although SPC charts may be easy to use even for patients, clinicians, or managers without extensive SPC training, they may not be equally simple to *construct correctly*. To apply SPC is, paradoxically, both simple and difficult at the same time.
>
> – Thor et al. (2007)

This book addresses that paradox. It provides a clear, practical guide to constructing and using SPC charts appropriately in healthcare, using modern software and reproducible methods.

Who this book is for?

This book is for anyone working with healthcare data who wants to use SPC in practice. While it is particularly suited to data scientists and analysts, we recognise that our readers may include, healthcare professionals, educators, students, patients and improvers. If you are involved in monitoring or improving healthcare processes, this book, as highlighted below, is for you.

What sets this book apart?

This is a hands-on, step-by-step guide to producing SPC charts correctly and confidently. What makes this book distinctive is its practical, healthcare-focused and modern approach.

- **Healthcare focus** Unlike general SPC texts, this book is grounded in real healthcare applications, including patient safety, clinical outcomes, and operational performance. It complements the companion book, *Statistical Process Control: Elements of Improving Quality and Safety in Healthcare* (Mohammed 2024), by focusing on practical implementation.
- **Practical and applied** This is not a theoretical text. You will find worked examples, real datasets, and step-by-step guidance. The aim is to help you understand SPC and use it confidently in practice to improve healthcare.
- **Comprehensive coverage** From foundational principles to advanced SPC techniques, this book covers it all. It is designed to cater to both beginners and experienced data scientists, ensuring everyone finds value.
- **Additional resources** To support your learning, we provide: R scripts, datasets, and a dedicated GitHub repository for continued learning and collaboration.

These resources are designed to help you apply what you learn and continue developing your skills.

Prerequisites

No prior knowledge of SPC is required. However, some familiarity with the R programming language will be helpful.

If you are new to R, we recommend resources such as:

- Learn R Programming with Johns Hopkins University, an excellent series of video tutorials covering all aspects of R programming – everything you need (and more) for this book.
- R Programming for Data Science, Roger D. Peng's companion book to the R Programming video series.
- NHS-R Community, which provides excellent free learning materials.

Why R?

Because SPC sits at the intersection of statistics, computing, and visualisation, our software of choice is R – widely regarded as the *lingua franca* of statistical computing.

R is free, open-source, and widely used. More importantly, it supports:

- **Transparency and reproducibility** Every step – from raw data to final chart – can be documented and reproduced.
- **Automation** Routine tasks such as generating reports, dashboards, and visualisations can be automated, saving time and reducing error.

However, although we use R throughout, the principles in this book apply regardless of the software you use.

How to use this book?

- **New to SPC?**: Start with Chapter 1 to understand variation.
- **Want to get started quickly?**: Jump to Chapters 5 and 6 to begin creating charts.
- **Looking to deepen your understanding?**: Later chapters cover more advanced topics and chart types.
- **Need code and data?**: Visit the GitHub repository for additional materials and updates.

We hope this book proves valuable, and we welcome your feedback for ongoing improvement.

About the authors

- **Jacob Anhøj**: Medical doctor with over 30 years of experience and a diploma in Information Technology. Author of 45+ peer-reviewed papers and several books, Jacob is committed to patient safety and quality improvement. He has extensive teaching experience and is also the creator of several R packages, including qicharts2. Contact: jacob@anhoej.net

- **Mohammed Amin Mohammed**: Emeritus Professor of Healthcare Quality and Effectiveness at the University of Bradford, with over 100 peer-reviewed publications. His work has been central to introducing SPC into healthcare practice. He is also founder of the NHS-R Community. Contact: profmaminm@gmail.com; m.a.mohammed5@bradford.ac.uk

Part 1: Understanding Variation

Chapter 1

Understanding Variation

> The central problem in management and in leadership […] is failure to understand the information in variation.
>
> Deming (1986), p. 309, quote attributed to Lloyd S. Nelson

Imagine writing the letter 'a' by hand using pen and paper. Figure 1.1 shows eight instances of this letter written by one of the authors. The first seven were produced using the dominant (right) hand, while the final one was written with the non-dominant hand.

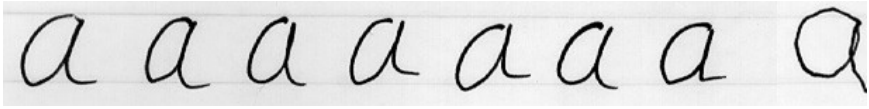

Figure 1.1: Handwritten a's

Now look closely at the seven letters on the left. Even though they were written under the same conditions – same hand, same time, same place, same pen, paper, lighting, and so on – they are not identical. Instead, even under essentially identical conditions, they show small, natural differences. This type of variation is called common cause variation because it is caused by the process itself.

When we see this kind of variation, it's tempting to judge or rank the results deciding which letters look "better" and trying to copy those while avoiding the "worse" ones. However, this approach misses the point. Since all seven letters were produced under the same conditions, none is inherently better or worse than the others. From a process perspective, they are all

equivalent. The differences between them are simply the natural result of the process.

So, how do we improve the quality of these letters? The key is to focus on improving the process, not the individual outcomes.

For example, we might try using a different pen or paper, practising more, or even switching to a computer. Intuitively, switching to a computer would produce the biggest improvement. Why? Because of the theory of constraints, which tells us that every process is like a chain, and its overall strength is limited by its weakest link. Improving that weakest link leads to the greatest overall improvement.

In this case, the weakest link is the act of handwriting itself. Changing the pen or paper won't make a big difference – but removing the need to write by hand altogether (by using a computer) will.

Now consider the final letter in Figure 1.1 – the one written with the non-dominant hand. It stands out clearly from the others. This difference points to a special cause.

Special cause variation comes from factors outside the normal process and signals that something unusual has happened. When we see it, we need to investigate – like a detective – what caused it.

If the special cause makes things worse, we should try to eliminate it. If it improves the outcome, we should try to understand it and, where possible, build it into the process.

These handwritten letters illustrate the two fundamental types of variation:

- **Common cause variation** This is the natural variation within a stable process. It is predictable within limits and can only be reduced by changing the process itself.
- **Special cause variation** This arises from specific, external factors. It is unpredictable and requires investigation to understand its source.

These ideas were first developed by Walter A. Shewhart in the 1920s as he worked to improve industrial quality (Shewhart 1931). Shewhart recognised that quality is not just about meeting specifications – it is about understanding and managing variation. He originally described these as chance causes and assignable causes, which later became known as common and special causes.

The key differences are summarised below:

Table 1.1: Common versus special cause variation

Common Cause Variation	**Special Cause Variation**
Comes from a stable, predictable system.	Comes from an external, identifiable cause.
Represents normal process behaviour.	Represents a signal that something unusual has happened.
Neither good nor bad – it simply exists.	May be helpful or harmful.
Requires changes to the whole process to reduce.	Requires investigation and specific action.
Also called random, chance, or natural variation.	Also called assignable or non-random variation.

In summary, Statistical Process Control (SPC) is about understanding variation – what kind it is, where it comes from, and what to do about it. It's not just about spotting outliers; it's about improving processes and outcomes in a meaningful way.

As Donald J. Wheeler puts it:

> Statistical Process Control is not about statistics, it is not about 'process-hyphen-control', and it is not about conformance to specifications. […] It is about the continual improvement of processes and outcomes. And it is, first and foremost, a way of thinking with some tools attached.
>
> – Wheeler (2000), p. 152

With this foundation in place, the rest of this book focuses on the practical tools of SPC – especially the SPC chart. In the next chapter, we'll explore the "anatomy and physiology" of these charts and how they can be used in practice.

Chapter 2

Understanding SPC Charts

Having explored the concepts of common and special cause variation through the example of handwritten letters, the next step is to understand how these ideas apply to real-world processes in healthcare. This involves constructing charts that represent the "voice" or behaviour of a process over time. By drawing on statistical theory, such charts help us determine whether a process is exhibiting common cause variation or whether special causes are present. These visualisations are known as SPC charts, control charts, or process behaviour charts.

SPC charts are point-and-line plots of data collected sequentially over time. They provide an operational definition for distinguishing between common and special cause variation. An operational definition is one that is practical and useful, ensuring that different individuals interpret the data in a consistent way.

Figure 2.1 shows a control chart of daily systolic blood pressure measurements. One data point (#6), marked with a red dot, stands out from the others. This deviation triggers a signal, indicating that the point is unlikely to have arisen by chance alone and may therefore be attributed to a special cause.

Special causes are identified through unusual patterns in the data. By "unusual", we mean patterns that are unlikely to occur purely by chance, such as an extreme data point. In contrast, common cause variation represents the natural, inherent noise in a stable process. When no signals are present, the observed variation is consistent with common cause variation. When signals are present, the data are consistent with special cause variation.

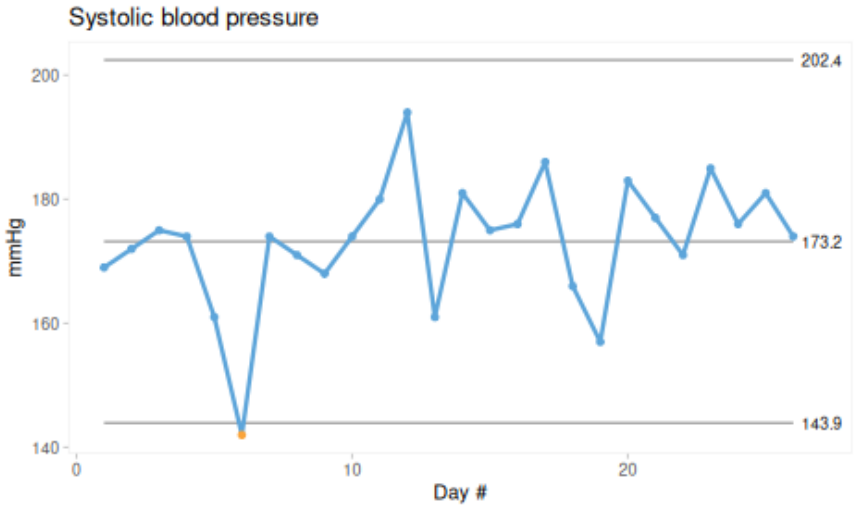

Figure 2.1: Control chart of systolic blood pressure

There are many types of control charts, and the choice of chart depends primarily on the type of data being analysed. Despite these differences, most SPC charts share a common appearance and are interpreted in the same way, using a set of statistical tests (or rules) to identify unusual patterns.

In this chapter, we introduce seven of the most commonly used control charts in healthcare, often referred to as the Magnificent Seven, along with their practical applications. In Chapter 3, we examine the rules used to detect unusual patterns in greater detail. Later, in Part 2 of the book, we provide a detailed guide to the calculations required to construct and analyse control charts.

2.1 Anatomy and physiology of SPC charts

A key innovation introduced by Shewhart was the concept of control limits, which define the expected range of common cause variation. Data points within these limits are consistent with common cause variation, whereas points outside the limits suggest the presence of special causes.

Figure 2.2 shows a standardised Shewhart control chart constructed from random numbers drawn from a normal distribution with mean 0 and standard deviation 1. The x-axis represents time or sequence order, and the y-axis shows the observed values. Each point represents a subgroup, that is, a set of observations collected under similar conditions at a given point

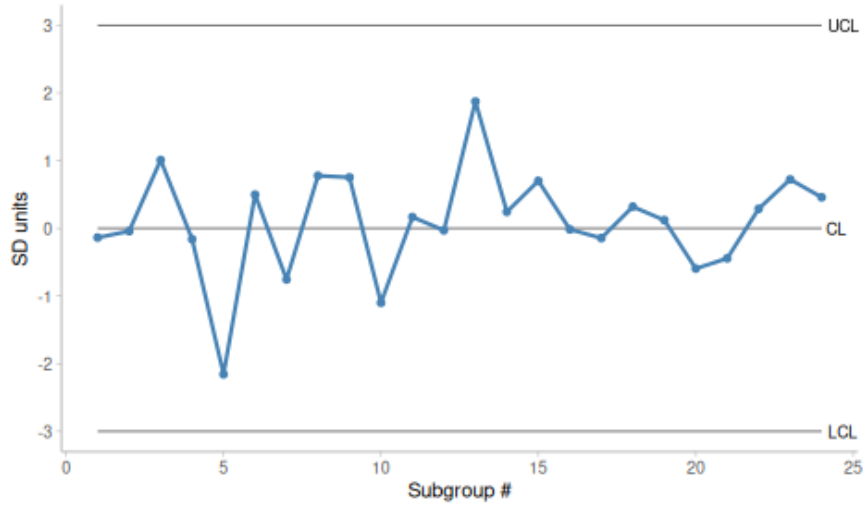

Figure 2.2: Standardised control chart. SD = standard deviation; CL = centre line; LCL = lower control limit; UCL = upper control limit

in time. The points are connected in sequence to emphasise the temporal structure of the data.

The chart includes a centre line (CL), representing the average of the data (typically the mean or median), and two control limits: the lower control limit (LCL) and the upper control limit (UCL). These limits define the range within which common cause variation is expected to fall. Because they are placed three standard deviations (SD) on either side of the centre line, they are often referred to as 3-sigma limits.

The data points tend to cluster around the centre line, with most values close to the average and fewer observations occurring as the distance from the centre increases. The control limits are set to include almost all points arising from common cause variation. Points falling outside these limits are therefore likely to represent special causes.

2.2 Why 3-sigma limits?

Shewhart's choice of 3-sigma limits was informed by empirical observations from real-world data. For normally distributed data, such as those in Figure 2.2, approximately 99.7% of observations lie within three standard deviations of the mean. This implies that the probability of a point falling outside the limits purely by chance is about 0.3%, or roughly 3 in 1,000.

In a chart with 20 data points, the probability that all points fall within the control limits is therefore approximately 95%.

However, Shewhart did not base his method solely on theoretical considerations. He recognised that most real-world data are not normally distributed and argued that the use of 3-sigma limits is justified because "they work" (Shewhart (1931), p. 18). In practice, this choice provides a reasonable balance between sensitivity (detecting special causes) and specificity (avoiding false alarms), regardless of the underlying distribution.

2.3 Common types of control charts: The Magnificent Seven

As noted above, control limits are typically expressed as:

$$CL \pm 3SD$$

Constructing a control chart therefore requires estimates of both the process mean and the process standard deviation. Importantly, the standard deviation used should reflect only common cause variation. Using the overall standard deviation of all data points may incorporate variation from special causes and result in control limits that are too wide.

To avoid this, SPC charts use estimates based on within-subgroup variation, typically obtained by pooling variation across subgroups. The exact method depends on the type of data being analysed. Detailed formulas are provided later in Table 6.1.

Broadly, data fall into two categories: count data and measurement data.

2.3.1 Count data

Count data consist of non-negative integers representing the number of events or cases. The distinction between events and cases is important.

An event is an occurrence in time and space, such as a patient fall. If a patient falls multiple times, each fall is counted as a separate event.

A case refers to an individual unit possessing (or not possessing) a given attribute, such as a patient who has fallen. Each individual is counted only once, regardless of how many events occur.

Both events and cases can be expressed relative to a denominator, often referred to as the area of opportunity.

Events are typically expressed as rates, such as the number of falls per 1,000 patient-days. In this case, the denominator represents time or exposure and differs in nature from the numerator.

Cases are expressed as proportions, such as the percentage of patients who experienced at least one fall. Here, the numerator is a subset of the denominator and cannot exceed it.

The most common SPC charts for count data are:

- C chart: counts of events
- U chart: rates (events per unit of time or opportunity)
- P chart: proportions (cases as a subset of the total)

2.3.2 Measurement data

Measurement data are continuous and may take any value within a range, often including decimals. Examples include blood pressure, height, weight, and waiting times.

Such data can be plotted either as individual observations or as subgroup summaries. For example, waiting times may be recorded for each patient individually or averaged over a period such as a day or month.

When measurements are plotted individually, an I chart (also called an X chart) is used, where each subgroup consists of a single observation. The I chart is often paired with a moving range (MR) chart, which displays the variation between consecutive observations.

When measurements are grouped, an X-bar chart is used to display subgroup averages. This is typically paired with an S chart, which shows the variation within each subgroup.

Thus:

- I and MR charts are used for individual measurements (subgroup size $= 1$)
- X-bar and S charts are used for grouped measurements (subgroup size > 1)

2.4 Summary of common SPC charts

A Shewhart control chart is a point-and-line plot of data over time, supplemented by three horizontal reference lines:

- The centre line (CL), representing the process average
- The lower control limit (LCL) and upper control limit (UCL), defining the range of expected variation

Although different chart types are used for different kinds of data, they share a common structure and interpretation. Selecting the appropriate chart begins with identifying the type of data.

For count data:

- C chart: counts of events
- U chart: rates
- P chart: proportions

For measurement data:

- I chart with MR chart: individual observations
- X-bar chart with S chart: subgroup averages and variation

In the next chapter, we examine the rules used to detect signals of special cause variation. These rules are designed to achieve high sensitivity while maintaining a reasonable false alarm rate.

Chapter 3

Signals Beyond the 3-Sigma Rule

So far, we have focused primarily on Shewhart's 3-sigma rule, which identifies special cause variation when one or more data points fall outside the control limits.

The 3-sigma rule is particularly effective at detecting large shifts in data. To illustrate this, consider a process where the data follow a normal distribution.

If the process suddenly shifts upward by three standard deviations (SDs), the previous upper control limit (UCL) effectively becomes the new centre line (CL). In this situation, approximately half of the subsequent data points will fall above the old UCL, providing a clear signal that the process has changed.

If the shift is smaller – for example, one standard deviation – the previous UCL corresponds roughly to the new 2-sigma limit. In this case, only about 2.5% of the data points will exceed the old UCL. The shift is still detectable, but signals occur much less frequently.

The strength of the 3-sigma rule lies in its ability to detect large and clearly distinguishable changes in process behaviour. However, smaller and more gradual shifts may remain undetected for extended periods. To improve sensitivity to such changes, additional rules and techniques are often used (Figure 3.1).

The performance of the 3-sigma rule has been studied extensively (see, for instance, Montgomery (2020) or Anhøj and Wentzel-Larsen (2018)). In general, it is most effective for detecting shifts of approximately 1.5 to 2 standard deviations or larger. Smaller shifts may go undetected for some

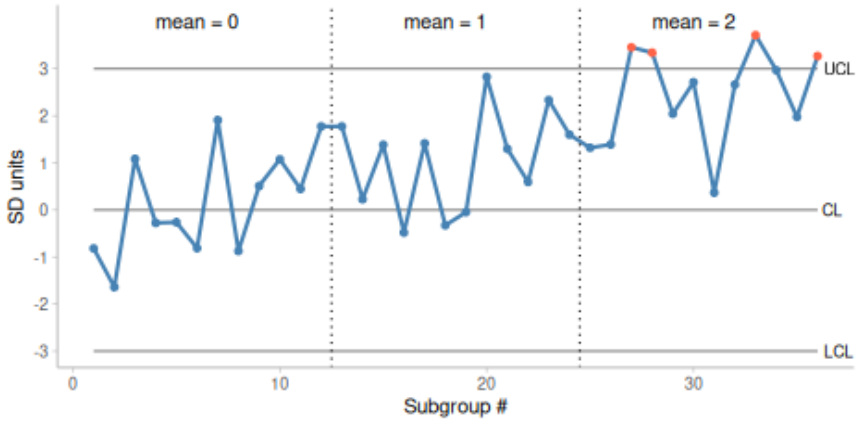

Figure 3.1: Control chart with progressive shifts in data

time. Before introducing additional rules, it is helpful to consider the types of patterns that often signal the presence of special causes.

3.1 Patterns of non-random variation

The Western Electric Handbook (Western Electric Company 1956) describes a range of patterns that can assist in the interpretation of control charts. These patterns are based on the observation that certain data behaviours are often associated with specific causes.

In our experience, the most common patterns encountered in healthcare data are freaks, shifts, and trends.

3.1.1 Freaks

A freak is a data point, or a small number of data points, that are distinctly different from the rest of the data (Figure 3.2). By definition, freaks are transient – they appear suddenly and then disappear.

Freaks are often caused by data or sampling errors, but they may also arise from temporary external influences on the process, such as changes in patient case mix during holiday periods. Occasionally, a freak may simply represent a false alarm, which is to be expected from time to time.

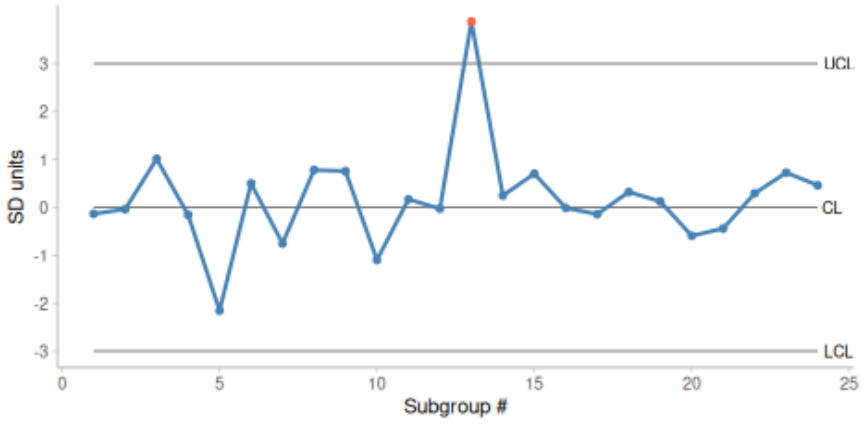

Figure 3.2: Control chart with a large (2 SD) transient shift in data

3.1.2 Shifts

Shifts represent sudden and sustained changes in process behaviour (Figure 3.3).

In healthcare improvement, the aim is often to create shifts in the desired direction by changing the underlying structures and processes that generate the data.

3.1.3 Trends

A trend is a gradual change in the process level over time (Figure 3.4).

While some definitions restrict trends to sequences of continuously increasing or decreasing points, we use the term more broadly to describe any gradual movement in one direction.

Trends typically arise when external influences act consistently over time. This is often observed when improvements are introduced incrementally. For example, small changes at the department level may accumulate into a gradual system-wide improvement.

3.1.4 Other unusual patterns

Many other types of patterns may occur in time series data. However, in healthcare settings, changes in performance are most often associated with freaks, shifts, or trends. Recognising these patterns is an important step in

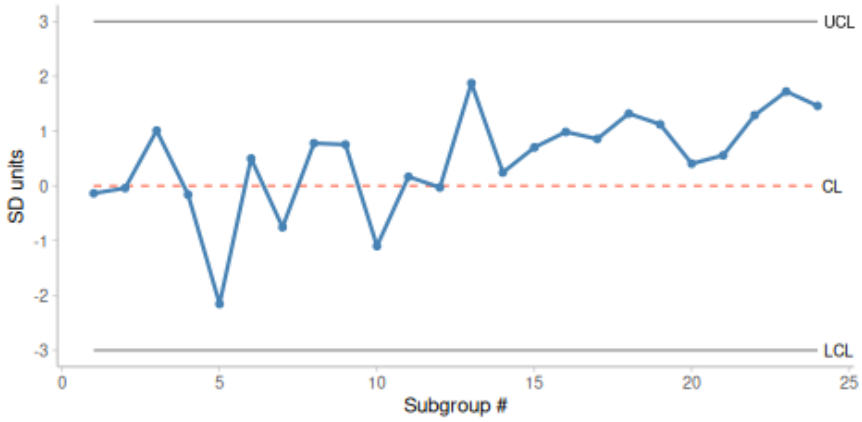

Figure 3.3: Control chart with a minor (1 SD) sustained shift in data introduced at data point #16

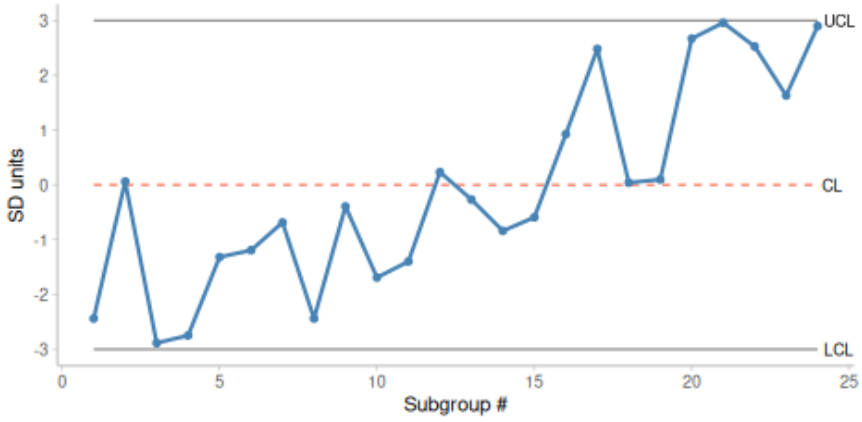

Figure 3.4: Control chart with a trend in data

understanding process behaviour. In some situations, other patterns may also be relevant. For example, cyclic or seasonal variation may produce recurring patterns linked to time of day, week, or year (Jeppegaard et al. 2023). Such patterns may not appear as freaks, shifts, or trends, but they can still be important for understanding and improving processes.

3.2 SPC rules

If patterns of special cause variation were always as clear as in the figures 3.2-3.4, there would be little need for formal SPC rules. In practice, however, special causes are often less obvious. For this reason, we use statistical tests – commonly referred to as SPC rules – to help identify patterns that are unlikely to occur by chance.

A large number of such rules have been proposed. While it may be tempting to apply many rules simultaneously, doing so increases the likelihood of false alarms, where common cause variation is mistakenly interpreted as special cause variation.

It is therefore important to strike a balance: using as few rules as necessary to detect meaningful signals while limiting the number of false alarms.

In this book, we focus on two sets of rules that have been well studied and shown to perform effectively.

3.2.1 Tests based on sigma limits

One of the best-known sets of rules is the Western Electric (WE) rules (Western Electric Company 1956). These consist of four tests based on the position of data points relative to the centre line and control limits:

- One or more points outside the 3-sigma limits
- Two out of three successive points beyond a 2-sigma limit
- Four out of five successive points beyond a 1-sigma limit
- Eight successive points on the same side of the centre line

The first rule corresponds to Shewhart's original 3-sigma rule. The remaining rules increase sensitivity to smaller shifts.

The WE rules have been widely used for decades. However, their performance depends on the number of data points: with fewer observations they may miss signals, and with many observations they may produce more false alarms.

The fourth rule is independent of sigma limits and is based on the concept of runs, which we consider next.

3.2.2 Runs analysis – tests based on the distribution of data points around the centre line

Runs analysis is based on the distribution of data points around the centre line. If the centre line divides the data into two equal halves, the probability that a point lies above or below the centre is equal.

A run is a sequence of consecutive points on the same side of the centre line. A crossing occurs when two consecutive points lie on opposite sides.

In a random process, the number and length of runs follow predictable patterns. When a process shifts or trends, runs tend to become longer and fewer. This observation forms the basis for runs tests.

What constitutes an "unusually" long run depends on the total number of observations or subgroups. The more subgroups we have in our SPC chart, the longer the longest runs are likely to be.

Based on theoretical work and practical experience (Anhøj and Olesen 2014; Anhøj 2015; Anhøj and Wentzel-Larsen 2018), we recommend two run-based tests:

- **Unusually long runs**: The longest run exceeds approximately $\log_2(n) + 3$, where n is the number of useful data points (Schilling 2012).
- **Unusually few crossings**: The number of crossings is below the lower 5% limit of a binomial distribution (Chen 2010).

These two measures are closely related: as runs become longer, crossings become fewer.

Critical values for longest runs and number of crossings for 10-100 data points are given in Appendix D. For example, in a run chart with 24–26 useful observations (i.e. data points not on the centre line), a longest run of *more* than eight points or *fewer* than eight crossings signals a shift in the data.

In Figure 3.3 the longest run exceeds the expected limit, indicating a shift. In Figure 3.4, long runs and few crossings similarly suggest non-random behaviour.

It is important to note that these tests indicate the presence of non-random variation but do not explain its cause. Interpretation requires knowledge of the process and context.

3.3 SPC charts without borders – using run charts

Some SPC rules rely only on a centre line. If we remove the control limits and use the median as the centre, we obtain a run chart.

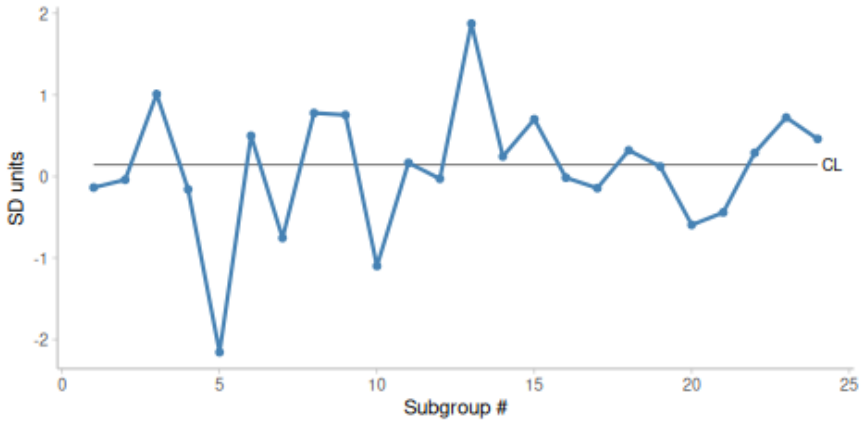

Figure 3.5: Run chart

Figure 3.5 shows a run chart of the data from Figure 2.2. Because the median is used, the data are evenly divided around the centre line. The runs analysis shows no unusually long runs or unusually few crossings, indicating no evidence of persistent shifts.

Figure 3.6 shows a run chart with a trend. The runs analysis confirms the presence of unusually long runs and few crossings.

Run charts are simple to construct and do not rely on distributional assumptions. They are particularly useful for detecting moderate and persistent changes in process behaviour.

Control charts, however, provide additional information. They are effective at identifying large, isolated deviations and define the expected range of common cause variation.

Run charts and control charts therefore serve complementary purposes.

3.4 A practical approach to SPC analysis

SPC charts are used for two related purposes: process improvement and process monitoring. In process improvement, the goal is to achieve

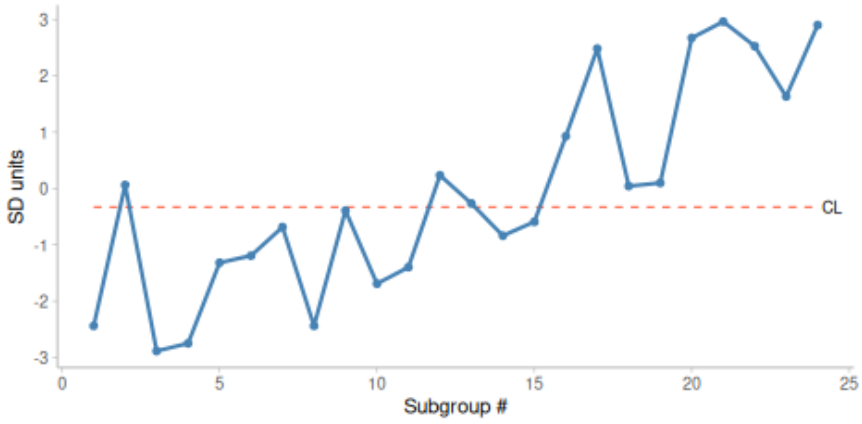

Figure 3.6: Run chart with a trend

sustained changes in performance. In this context, shifts are expected, and runs analysis is particularly useful for detecting them.

In process monitoring, the aim is to maintain stability. Here, the focus is on detecting deterioration as early as possible. The 3-sigma rule is effective for this purpose, and additional rules may be used to increase sensitivity.

Regardless of the purpose, it is often useful to begin with a run chart using the median as the centre line. If the data show evidence of non-random variation, the focus should be on understanding the causes of change. If the data appear stable and the outcome is satisfactory, control limits may be added to support ongoing monitoring using the 3-sigma rule in combination with either the remaining WE rules or the two runs-tests.

3.5 SPC rules in summary

SPC rules are statistical tools used to signal special cause variation in time series data. While many rules exist, using too many increases the risk of false alarms. A small number of well-chosen rules can provide an effective balance between sensitivity and specificity.

In this chapter, we have introduced the 3-sigma rule, the Western Electric rules, and runs-based tests. Together, these provide a practical framework for identifying signals of special cause variation.

Chapter 4

Using SPC in Healthcare

As briefly discussed in the previous chapter, SPC methodology is used in healthcare in two main ways: to **monitor** the behaviour or performance of an existing process, and to support efforts to **improve** a process. Examples of monitoring include tracking complication rates following surgery, while examples of improvement include redesigning a care pathway for patients with fractured hips.

4.1 Using SPC to monitor a process

In monitoring mode, the primary aim is to determine whether a process is deteriorating. Deterioration is usually indicated by signals of special cause variation, which prompt further investigation to identify and eliminate the cause. One way to undertake such detective work is by using the Pyramid Model of Investigation described below.

The central purpose of using SPC charts to monitor healthcare processes is to ensure that the quality and safety of care are adequate and not worsening. When a signal of special cause variation appears on a control chart, investigation is therefore necessary. However, the chosen method of investigation must recognise that the relationship between recorded outcomes and quality of care is often *complex, ambiguous, and open to several possible explanations* (Lilford et al. 2004). Failure to recognise this may lead to premature conclusions and foster a culture of blame that undermines both the engagement of clinical staff and the credibility of SPC.

As Rogers et al. (2004) noted:

> If monitoring schemes are to be accepted by those whose outcomes are being assessed, an atmosphere of constructive

> evaluation, not 'blaming' or 'naming and shaming', is essential as apparent poor performance could arise for a number of reasons that should be explored systematically.

To address this need, Mohammed et al. (2004) proposed the **Pyramid Model for Investigation of Special Cause Variation in healthcare**, a systematic approach of hypothesis generation and testing based on five a priori candidate explanations for special cause variation: data, patient casemix, structure or resources, process of care, and carers (Figure 4.1).

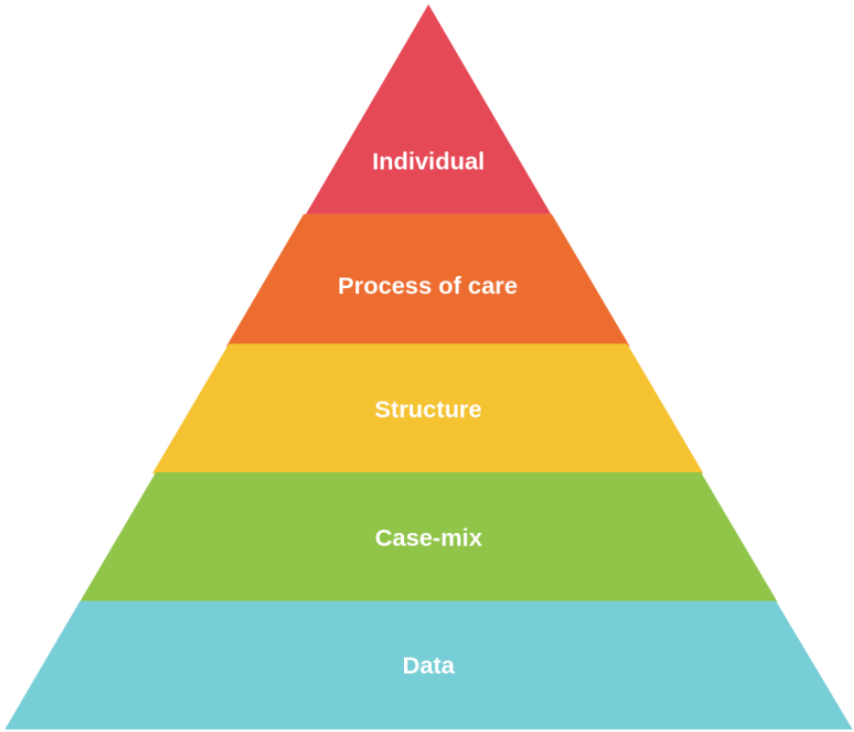

Figure 4.1: Pyramid Model for Investigation

These broad categories are arranged from the most likely explanation at the base of the pyramid to the least likely at the top. In this way, the model provides a roadmap for investigation, beginning with the simplest and most plausible explanations and progressing only as needed. The first two layers, data and casemix, focus on the validity of the data and the adequacy of any casemix adjustment. The upper three layers focus more directly on aspects of care delivery and professional practice.

A proper investigation requires a multidisciplinary team with expertise relevant to each layer of the pyramid. Such a team is also likely to include staff whose outcomes or data are being investigated, so that their experience and insight can inform the enquiry and so that they remain engaged in the process.

The basic steps in using the model are as follows. First, a multidisciplinary team is formed with sufficient expertise to assess each layer of the pyramid and to judge whether a credible explanation has been found. Next, guided by the type of pattern seen in the data – whether freaks, shifts, trends, mixed patterns, or cyclic variation – the team generates and tests candidate explanations, beginning at the lowest level of the pyramid and moving upwards only if the lower levels do not provide an adequate explanation. A credible cause should be supported by both quantitative and qualitative evidence. If no credible explanation can be found, the remaining possibility is that the signal was a false one.

The kinds of questions that may be asked during such an investigation include the following.

Data: At this level the focus is on data quality, including coding accuracy, definitions, completeness, and reliability.

- Are the data coded correctly?
- Has there been a change in coding practice, for example because less experienced coders are involved?
- Is the clinical documentation clear, complete, and consistent?

Casemix: Although differences in casemix may be accounted for in the analysis, some residual confounding may remain.

- Are there factors specific to this hospital that are not captured in the risk adjustment?
- Has the pattern of referrals changed in a way that is not reflected in the adjustment model?

Structure or resources: This includes the availability of staff, beds, equipment, and relevant organisational arrangements.

- Has there been a change in the distribution of patients across the hospital, so that more patients in this specialty are now spread across multiple wards rather than concentrated in one unit?
- Has the physical environment or organisational structure changed?

Process of care: This concerns treatments, pathways, and care policies.

- Has there been a change in the care being provided?
- Have new treatment guidelines been introduced?

Professional staff or carers: This concerns the practice and methods of those delivering care.

- Has there been a change in staffing?
- Has a key staff member received additional training and introduced a new method that may have affected outcomes?

4.2 Using SPC to improve a process

SPC is also used to support efforts to improve a process. In healthcare, this usually involves making small-scale changes and measuring their impact on an SPC chart. In improvement mode, the main aim is to determine whether the changes made to a process have resulted in improvement. Returning to the earlier handwriting example, we might ask whether we produce better letters after switching from pen and paper to a computer. The answer depends on whether the change is followed by signals of special cause variation in the desired direction, assuming we have a credible measure of "letter quality".

The degree of alignment between changes made to the process and subsequent signals on the chart provides a story that describes the impact of those changes, both qualitatively and quantitatively.

Common cause variation can only be reduced by changing a major part of the process. But what do we mean by a major part? The Theory of Constraints (Goldratt and Cox 2022) offers the analogy of a chain, whose strength is determined by its weakest link. Strengthening that weakest link strengthens the whole chain. Strengthening other links while leaving the weakest unchanged does not improve the system as a whole. In this sense, the weakest link represents the constraint on performance, and in most real systems there are only a few such constraints, perhaps only one or two, while the remaining factors are non-constraints.

A widely used framework for improvement in healthcare is the Model for Improvement, developed by Langley et al. (2009). This model consists of three fundamental questions together with the Plan-Do-Study-Act (PDSA) cycle.

The first question is: **What are we trying to accomplish?** This defines the aim of the improvement effort. The aim should be specific, measurable, and time-bound, and it should include a clear rationale.

The second question is: **How will we know that a change is an improvement?** This focuses attention on measurement. The team identifies the key indicators that will be used to determine whether improvement has occurred. This should also include balancing measures, which are intended to detect unintended negative consequences of change.

The third question is: **What changes can we make that will result in improvement?** This concerns the generation and testing of change ideas. These ideas are tested using the PDSA cycle:

Plan: Develop a plan for testing the change, including who will do what, when, and where.

Do: Carry out the change on a small scale.

Study: Examine the results, paying particular attention to the effect of the change.

Act: Decide whether to adopt, adapt, or abandon the change on the basis of what has been learned.

There are, of course, other approaches to improvement in healthcare, including Lean, Six Sigma, and Systems Engineering. SPC charts can support each of these approaches because they provide a robust and informative way of assessing whether a change has had the intended effect.

4.3 Successful use of SPC in healthcare

Successful use of SPC in healthcare requires more than simply producing a chart, especially in a complex adaptive system such as healthcare. Important factors include engaging stakeholders, forming an appropriate team, defining a clear aim, selecting the process of interest, choosing the relevant measures, ensuring that data can be collected and fed back reliably, and establishing a baseline. All of this needs to take place within a culture of continual learning and improvement, supported by leadership. For examples of SPC in healthcare practice, see Mohammed (2024).

At the same time, it is important to recognise that SPC charts are not always easy to construct correctly. After reviewing 64 SPC charts, Koetsier et al. (2012) found that almost half contained technical problems. This suggests a need for better training among those who construct charts and helps explain the motivation for this book.

This concludes Part 1. In Part 2, beginning with Chapter 5, we turn to the practical task of producing SPC charts in R.

Part 2: Constructing SPC Charts

Chapter 5

Your First SPC Charts With Base R

From Part 1 of this book, we now have a good understanding of what SPC is and how SPC charts work. In this chapter, we begin constructing SPC charts using functions from base R. In later chapters, we will introduce functions from *ggplot2* and *qicharts2* (an R package developed by JA).

At its core, an SPC chart is a point-and-line plot of data over time, with a horizontal line representing the centre of the data and – when constructing a control chart – two additional lines representing the estimated upper and lower bounds of natural variation.

5.1 A run chart of blood pressure data

Consider the data from Figure 2.1, which show systolic blood pressure measurements (mm Hg) for a patient recorded each morning over 26 consecutive days (Mohammed et al. 2008).

```
systolic <- c(169, 172, 175, 174, 161, 142,
              174, 171, 168, 174, 180, 194,
              161, 181, 175, 176, 186, 166,
              157, 183, 177, 171, 185, 176,
              181, 174)
```

We begin by plotting a simple point-and-line chart without any additional reference lines (Figure 5.1):

```
# Make point-and-line plot
plot(systolic, type = 'o')
```

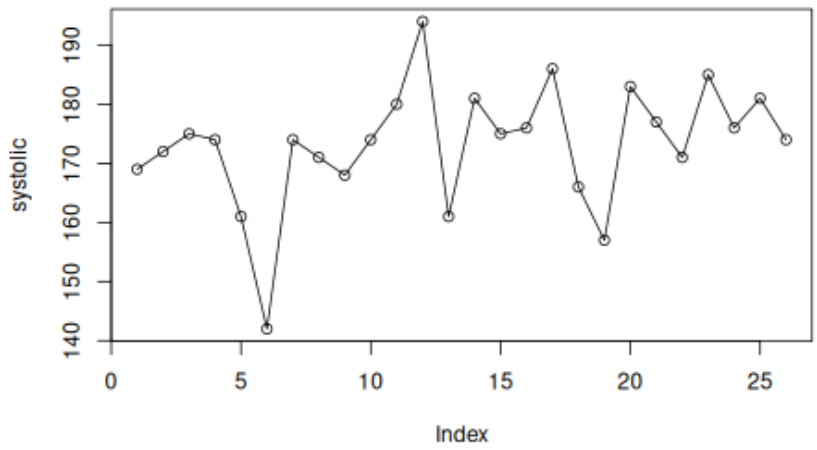

Figure 5.1: Simple run chart

(As a side note, this reminds me (JA) of a manager, who once said to me: "You make such beautiful graphs, but can't you stop them from going up and down all the time.")

To support runs analysis, we add a horizontal centre line. In a run chart, this is typically the median of the data (Figure 5.2):

```
# Create systolic-coordinates for the centre line
cl  <- median(systolic)           # calculate median
cl  <- rep(cl, length(systolic))  # repeat to match the length of y

# Plot data and add centre line
plot(systolic, type = 'o')
lines(cl)
```

From the chart, we observe that the longest run consists of four data points (#14-#17), and the data cross the centre line nine times. Four observations (#4, #7, #10, #26) lie exactly on the centre line, leaving 22 useful observations. Using the runs rules described in Chapter 3, the upper limit for the longest run is 7, which also corresponds to the lower limit for the number of crossings. Since neither limit is exceeded, there is no evidence of sustained shifts or trends in the data.

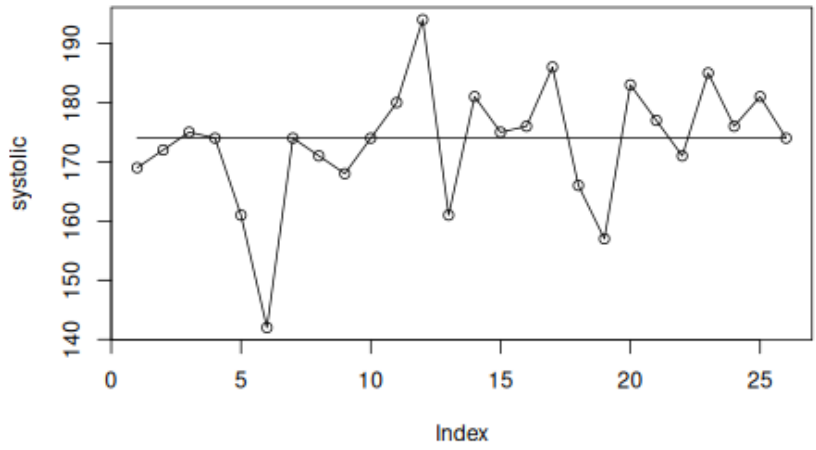

Figure 5.2: Run chart with centre line

5.2 Adding control limits to produce a control chart

We now extend the run chart by adding control limits to produce a control chart.

Recall that control limits are typically defined as CL $\pm$ 3SD, where the centre line (CL) is usually the mean, and the standard deviation (SD) represents the natural variation in the process. Importantly, this is not the overall standard deviation of all observations, but an estimate based on common cause variation.

For individual measurements, we use an I chart (see Chapter 2). In this case, the process standard deviation is estimated from the average moving range divided by a constant (1.128). The moving ranges are the absolute differences between consecutive observations. This will be explained in more detail in Chapter 6.

```
# Calulate the centre line (mean)
cl  <- rep(mean(systolic), length(systolic))

# Calculate the moving ranges of data
mr <- abs(diff(systolic))
```

```r
# Print the moving ranges for our viewing pleasure
mr
```

```
##  [1]  3  3  1 13 19 32  3  3  6  6 14 33 20  6  1 10 20  9 26  6  6 14  9  5  7
```

```r
# Calculate the average moving range
amr <- mean(mr)

# Calculate the process standard deviation
s <- amr / 1.128

# Create y-coordinates for the control limits
lcl <- cl - 3 * s
ucl <- cl + 3 * s
```

When plotting the chart, we must ensure that the y-axis accommodates both the data and the control limits (Figure 5.3):

```r
# Plot data while expanding the y-axis to make room for all data and lines
plot(systolic, type = 'o', ylim = range(systolic, lcl, ucl))

# Add lines
lines(ucl)
lines(cl)
lines(lcl)
```

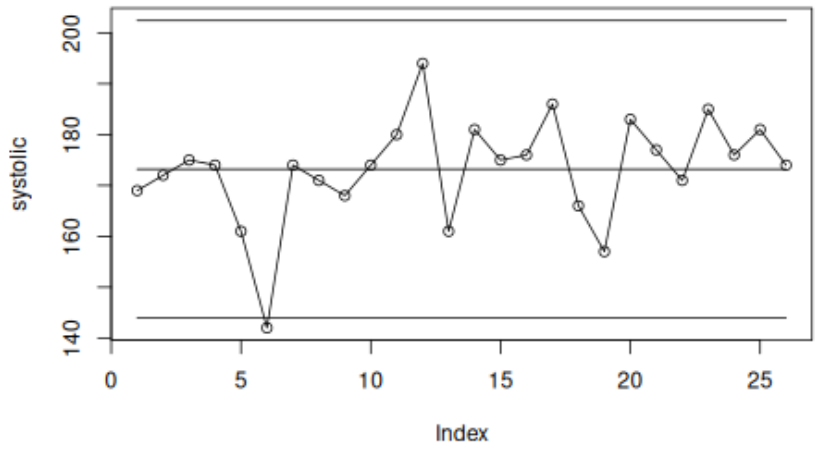

Figure 5.3: Standardised control chart

One data point falls below the lower control limit. This suggests that the observation is unlikely to be due to common cause variation alone and may

reflect a special cause. The chart itself does not explain the cause, but it highlights the need for further investigation with the aim of learning and improvement.

5.3 That's all, Folks!

Constructing an SPC chart in R requires only a few lines of code. Most of the work involves preparing the data and calculating the necessary statistics. The plotting itself follows a simple pattern: first plot the data points, then add the relevant lines.

In later chapters, we will wrap these steps into functions that automate the calculations, highlight signals of non-random variation, and produce more refined visualisations than the basic plots shown here.

In the next chapter, we turn to the construction of the most commonly used SPC charts in healthcare – *the Magnificent Seven*.

Chapter 6

Calculating Control Limits

In the previous chapter, we established the basic steps for constructing SPC charts in R using the I chart as an example. In this chapter, we continue with the rest of *The Magnificent Seven* and show how to calculate their control limits.

6.1 Introducing the spc() function

To avoid repeating the same plotting code, let us begin by creating a simple function to automate the plotting for us.

```
spc <- function(
    x,           # x axis values
    y   = NULL,  # data values
    cl  = NA,    # centre line
    lcl = NA,    # lower control limit
    ucl = NA,    # upper control limit
    ...          # other parameters passed to the plot() function
) {
  # if y is missing, set y to x and make a sequence for x
  if (is.null(y)) {
    y <- x
    x <- seq_along(y)
  }

  # repeat line values to match the length of y
  if (length(cl) == 1)
    cl <- rep(cl, length(y))

  if (length(lcl) == 1)
    lcl <- rep(lcl, length(y))

  if (length(ucl) == 1)
```

```
    ucl <- rep(ucl, length(y))

  # plot the dots and draw the lines
  plot(x, y,
       type = 'o',
       ylim = range(y, lcl, ucl, na.rm = TRUE),
       ...)
  lines(x, cl)
  lines(x, lcl)
  lines(x, ucl)
}
```

The `spc()` function takes five arguments, of which only the first, x, is required. If only x is supplied, a simple point-and-line chart is drawn using those values as the y-values and their sequence numbers on the x-axis. If y is also supplied, then x is used for the x-axis and y for the plotted data values.

The remaining arguments, cl, lcl, and ucl, are used for the centre line and the lower and upper control limits. These may be given either as single values or as vectors of the same length as the data. Additional graphical arguments, such as main, xlab, and ylab, may be passed on to the `plot()` function.

Let us test the function using the blood pressure data from Chapter 5 (Figure 6.1).

```
# create an x variable, not that is it necessary in this case, just because we can
day <- seq_along(systolic)

# plot data
spc(day, systolic, cl, lcl, ucl)
```

With this function in hand, we are now able to construct many different kinds of control charts. All we need to know is how to calculate the centre line and the control limits.

6.2 Formulas for calculation of control limits

The formulas for calculation control limits for The Magnificent Seven introduced in Chapter 2 are shown in Table 6.1. Do not be alarmed by the number of strange symbols. We will translate the formulas to R code one by one as we move through the chapter.

As discussed in Chapter 2, data come in two broad forms: count data and measurement data. Counts are non-negative integers representing events or cases, for example patient falls, surgical complications, or healthy babies. Measurements are values on a continuous scale and may include decimals, such as blood pressure, height, weight, or waiting times.

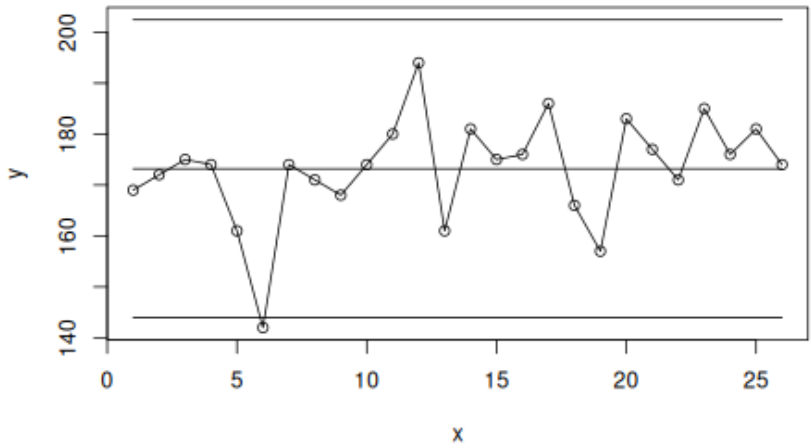

Figure 6.1: Control chart of systolic blood pressure

Table 6.1: Formulas for calculating control limits

Subgroups	Chart type	Control limits	Assumed distribution
Count data			
Counts	C	$\bar{c} \pm 3\sqrt{\bar{c}}$	Poisson
Rates	U	$\bar{u} \pm 3\sqrt{\frac{\bar{u}}{n_i}}$	Poisson
Proportions	P	$\bar{p} \pm 3\sqrt{\frac{\bar{p}(1-\bar{p})}{n_i}}$	Binomial
Measurement data			
Individual measurements	I	$\bar{x} \pm 2.66\overline{MR}$	Normal
Moving ranges of individual measurements	MR	$3.267\overline{MR}$	Normal
Averrages of 2 or more measurements	X-bar	$\bar{\bar{x}} \pm A_3\bar{s}$	Normal
Standard deviations of 2 or more measurements	S	$B_3\bar{s};\ B_4\bar{s}$	Normal

6.3 Count data

For count charts in this chapter we use the Bacteremia data set, which contain monthly counts of infection events and related mortality cases:

```
# read data from file
bact <- read.csv('data/bacteremia.csv',  # path to data file
                 comment.char = '#',     # ignore lines that start with "#"
                 colClasses   = c(       # specify variable types
                   'Date',
                   'integer',
                   'integer',
```

```
                  'integer',
                  'integer'
                ))

# print the first six rows
head(bact)
```

```
##        month ha_infections risk_days deaths patients
## 1 2017-01-01            24     32421     23      100
## 2 2017-02-01            29     29349     22      105
## 3 2017-03-01            26     32981     13       99
## 4 2017-04-01            16     29588     14       85
## 5 2017-05-01            28     30856     17       98
## 6 2017-06-01            16     30544     15       85
```

We use C charts for event counts, U charts for event rates, and P charts for case proportions.

6.3.1 C chart

The C chart (C for counts) is the simplest of all control charts and the easiest to construct. The process standard deviation is estimated simply as the square root of the process mean.

C charts are appropriate when counting events over subgroups that are approximately equal in size, whether these subgroups represent periods of time or units of space.

Figure 6.2 shows a C chart of the monthly number of hospital-acquired bacteremias.

```
with(bact, {
  cl  <- mean(ha_infections)
  lcl <- cl - 3 * sqrt(cl)
  ucl <- cl + 3 * sqrt(cl)

  # print the limits
  cat('UCL =',  ucl, '\n')
  cat('CL  =',   cl, '\n')
  cat('LCL = ', lcl, '\n')

  # plot the chart
  spc(month, ha_infections,
      cl, lcl, ucl,
      ylab = 'Infections', # y-axis label
      xlab = 'Month')      # x-axis label
})
```

```
## UCL = 36.94952
## CL  = 22.66667
## LCL =  8.38381
```

The average monthly number of infections is 22.7, and all data points fall within the control limits, which range from 8.4 to 36.9. If nothing changes,

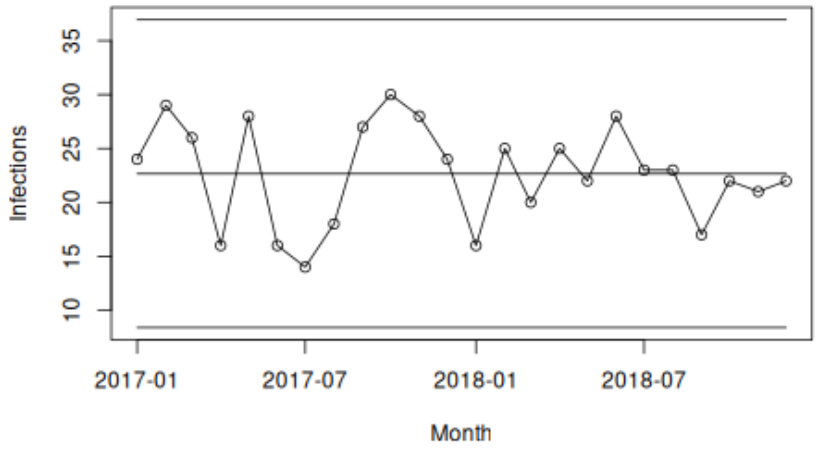

Figure 6.2: C chart of monthly numbers of hospital-acquired bacteremias

we should therefore expect future counts to be around 23, while recognising that from time to time values as low as 9 or as high as 36 may still occur as part of the natural variation in the process.

6.3.2 U chart

U charts are used when events are counted over subgroups of unequal size, resulting in unequal areas of opportunity (U for unequal). In our example, the number of patient-days may vary from month to month, so it makes sense to adjust for this by plotting rates rather than raw counts.

Because events are often rare relative to the area of opportunity, the resulting rates may be very small. It is therefore often helpful to multiply the rates by a constant before plotting. In Figure 6.3, we multiply by 10,000 to present infections per 10,000 risk-days rather than infections per day.

```
with(bact, {
  y   <- ha_infections / risk_days           # rates to plot
  cl  <- sum(ha_infections) / sum(risk_days) # overall mean rate, centre line
  s   <- sqrt(cl / risk_days)                # standard deviation
  lcl <- cl - 3 * s                          # lower control limit
  ucl <- cl + 3 * s                          # upper control limit

  # multiply y axis to present infections per 10,000 risk days
  multiply <- 10000
```

```
  y       <- y * multiply
  cl      <- cl * multiply
  lcl     <- lcl * multiply
  ucl     <- ucl * multiply

  spc(month, y, cl, lcl, ucl,
      ylab = 'Infections per 10,000 risk days',
      xlab = 'Month')
})
```

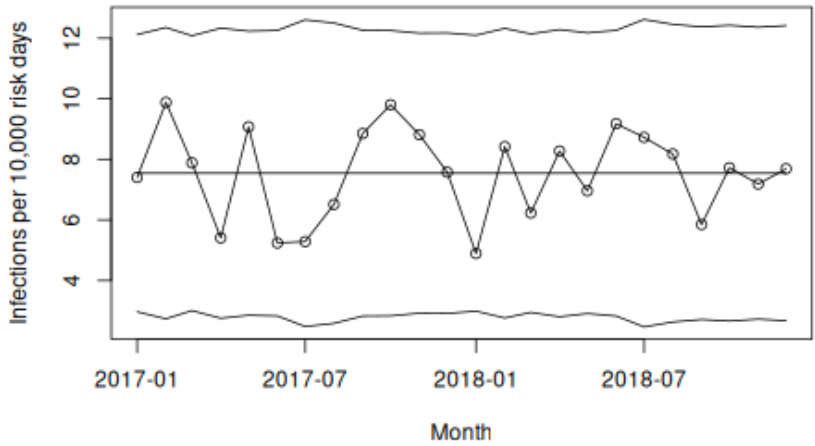

Figure 6.3: U chart of monthly number of infections per 10,000 risk days

The U chart in Figure 6.3 shows an average of 7.5 infections per 10,000 risk days, with all data points within control limits ranging from about 3.5 to 12. Unlike the C chart, the control limits vary from point to point because they depend on the denominator. Large denominators produce narrower limits, whereas small denominators produce wider ones.

In this example, the denominator varies only slightly between months, so the U chart adds little compared with the C chart. For pedagogical purposes, we may even prefer the C chart, because it is easier to relate to 23 infections per month than to 7.5 infections per 10,000 risk days.

6.3.3 P chart

P charts are used for proportions or percentages. Figure 6.4 shows the monthly percentage of patients with bacteremia who died within 30 days.

```
with(bact, {
  y   <- deaths / patients
  cl  <- sum(deaths) / sum(patients)      # process mean, centre line
  s   <- sqrt((cl * (1 - cl) / patients)) # process standard deviation
  lcl <- cl - 3 * s                       # lower control limit
  ucl <- cl + 3 * s                       # upper control limit

  # multiply by 100 to get percentages rather than proportions
  multiply <- 100
  y        <- y * multiply
  cl       <- cl * multiply
  lcl      <- lcl * multiply
  ucl      <- ucl * multiply

  spc(month, y, cl, lcl, ucl,
      ylab = '%',
      xlab = 'Month')
})
```

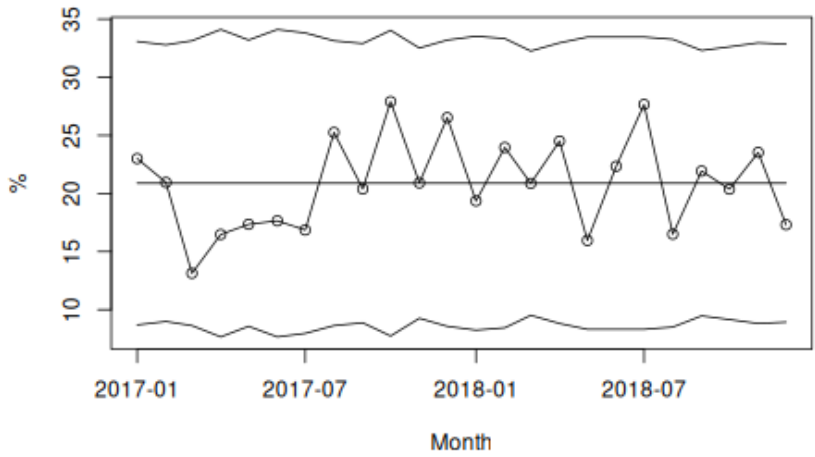

Figure 6.4: P chart of monthly 30-day mortality rates

The average mortality is 21%, and all data points fall within the control limits. As with the U chart, the control limits vary with the size of the denominator.

6.4 Measurement data

For this section, we use the Ceasearian section delay dataset on response times for 208 grade 2 caesarean sections, that is, the time in minutes from

the decision to perform the caesarean section until the baby was delivered. The aim is to keep this delay below 30 minutes.

```
# read raw data
csect <- read.csv('data/csection_delay.csv',
                  comment.char = '#',
                  colClasses   = c('POSIXct',
                                   'Date',
                                   'integer'))

csect <- csect[order(csect$datetime), ]

# show the first 6 rows
head(csect)
```

```
##              datetime      month delay
## 1 2016-01-06 03:55:40 2016-01-01    22
## 2 2016-01-06 20:52:34 2016-01-01    22
## 3 2016-01-07 02:50:43 2016-01-01    29
## 4 2016-01-07 22:32:27 2016-01-01    28
## 5 2016-01-09 14:56:09 2016-01-01    22
## 6 2016-01-09 21:21:24 2016-01-01    20
```

6.4.1 I chart

The "I" in I chart stand for "individuals" because each subgroup consists of a single value. I charts are also often referred to as X charts.

They are useful when measurements are available for individual units, for example waiting times or blood pressure measurements for individual patients.

As we shall see later, the I chart is in fact useful for many types of data because it bases its estimate of variation on the actual variation observed in the data rather than on theoretical parameters from an assumed distribution. For this reason, the I chart is often described as the *Swiss Army knife* of SPC.

When subgroups consist of single values, we estimate within-subgroup variation using the average moving range, that is, the average absolute difference between neighbouring data points. Dividing this value by 1.128 gives an estimate of the process standard deviation. Equivalently, we may multiply the average moving range by (3 / 1.128 = 2.66) to obtain 3 standard deviations.

Let us look at the delay times for the most recent 60 caesarean sections (Figure 6.5).

```
with(tail(csect, 60), {
  xbar <- mean(delay)      # mean value, centre line
  mr   <- abs(diff(delay)) # moving ranges
  amr  <- mean(mr)         # average moving range
  s    <- amr / 1.128      # process standard deviation
```

```
  spc(delay,
      cl  = xbar,
      lcl = xbar - 3 * s,
      ucl = xbar + 3 * s)
})
```

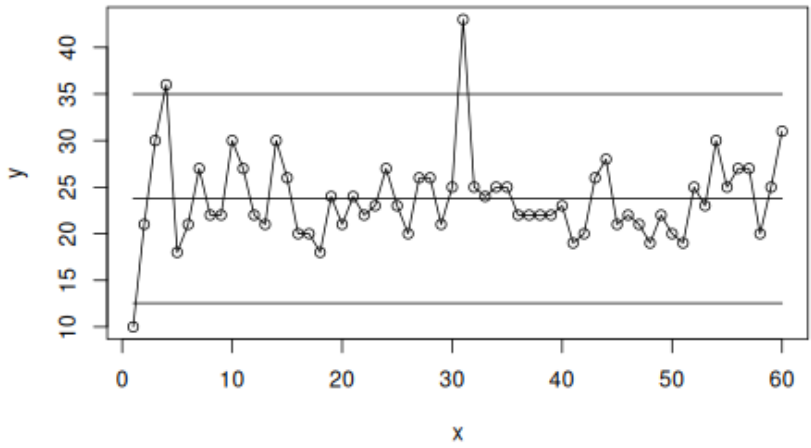

Figure 6.5: I-chart

The average delay for these cases is 24 minutes. Three data points fall outside the control limits (#1, #4, and #31), suggesting that these cases were unusual. It may therefore be useful to look more closely at what went particularly well in case #1 and less well in cases #4 and #31.

6.4.2 MR chart

The MR chart plots the moving ranges, that is, the absolute pairwise differences between neighbouring observations. It is a companion chart to the I chart. Because moving ranges can be zero but never negative, the MR chart has no lower control limit (Figure 6.6).

```
with(tail(csect, 60), {
  mr  <- c(NA, abs(diff(delay)))  # add NA in front to match the length of the I-chart
  amr <- mean(mr, na.rm = TRUE)

  spc(mr,
```

```
      cl   = amr,
      ucl = 3.267 * amr)
})
```

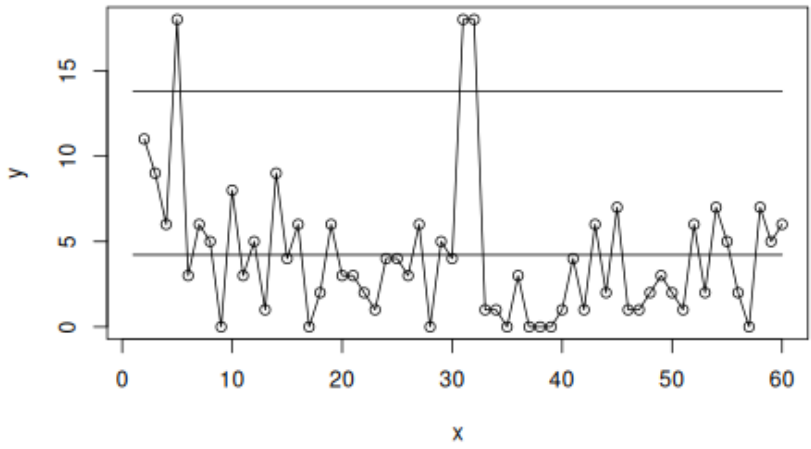

Figure 6.6: MR-chart

There is one fewer moving range than individual values. To keep the MR chart aligned with the I chart, we therefore insert an NA at the beginning.

The two charts may also be plotted together (Figure 6.7):

```
with(tail(csect, 60), {
  xbar <- mean(delay)
  mr   <- c(NA, abs(diff(delay)))
  amr  <- mean(mr, na.rm = TRUE)

  op <- par(mfrow = c(2, 1),           # setting up plotting area
            mar   = c(5, 5, 2, 1))
  spc(delay,
      cl   = xbar,
      lcl  = xbar - 2.66 * amr,
      ucl  = xbar + 2.66 * amr,
      main = 'I-chart',
      ylab = 'Delay (minutes)',
      xlab = '')

  spc(mr,
      cl   = amr,
      ucl  = 3.267 * amr,
      main = 'MR-chart',
      ylab = 'Moving range (minutes)',
      xlab = 'C-section #')
```

```
  par(op)                          # restoring plotting area
})
```

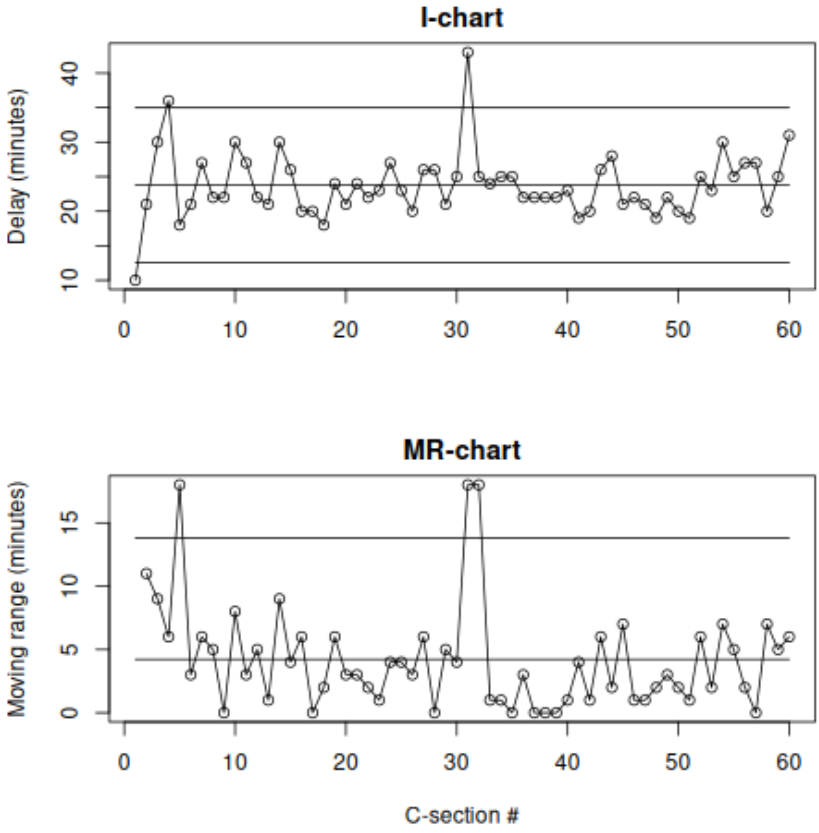

Figure 6.7: I- and MR-charts

The MR chart also identifies three data points outside the control limits. Two of these correspond to special causes identified on the I chart and strengthen the conclusion that these deliveries were unusual. Note that each point on the I chart, except the first and last, contributes to two moving ranges on the MR chart.

6.4.3 X-bar chart

The X-bar chart is appropriate when the subgroups consist of two or more measurements.

To construct a control chart of monthly average delay times, we first need to aggregate the data by month to obtain the subgroup mean, subgroup standard deviation, and subgroup size.

```
# split data frame by month
csect.agg <- split(csect, csect$month)

# aggregate data by month
csect.agg <- lapply(csect.agg, function(x) {
  data.frame(month = x$month[1],
             mean  = mean(x$delay),
             sd    = sd(x$delay),
             n     = nrow(x))
})

# put everything together again
csect.agg <- do.call(rbind, c(csect.agg, make.row.names = FALSE))

# print the first 6 rows
head(csect.agg)
```

```
##        month     mean       sd  n
## 1 2016-01-01 23.85714 3.387653  7
## 2 2016-02-01 24.45455 6.137811 11
## 3 2016-03-01 22.45455 6.638729 11
## 4 2016-04-01 22.66667 3.041381  9
## 5 2016-05-01 22.50000 3.891382  8
## 6 2016-06-01 22.00000 6.204837  5
```

See the section on aggregating data frames in Appendix C for details of this split-apply-combine strategy.

Next, we calculate the centre line and the control limits using the formula in Table 6.1. Here $\bar{\bar{x}}$ (x bar bar) is the weighted mean of the subgroup means, $\bar{s}$ (s bar) is the weighted mean of the subgroup standard deviations, and A_3 is a constant that depends on the subgroup size. The R code for calculating A_3 and other constants is given at the end of the chapter.

With the aggregated data we can now construct the X-bar chart (Figure 6.8).

```
with(csect.agg, {
  xbarbar <- weighted.mean(mean, n)  # centre line
  sbar    <- weighted.mean(sd, n)    # process standard deviation
  a3      <- a3(n)                   # A3 constant

  spc(x   = month,
      y   = mean,
      cl  = xbarbar,
      lcl = xbarbar - a3 * sbar,
      ucl = xbarbar + a3 * sbar)
})
```

Figure 6.8 shows the average delay per month. The overall average delay is 23 minutes, and all data points fall within the control limits, suggesting that the process is stable and predictable.

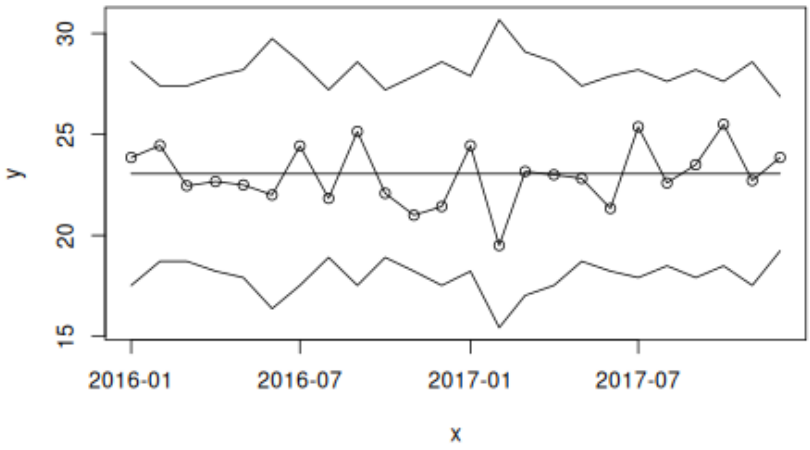

Figure 6.8: X bar chart

As with U and P charts, the control limits vary from month to month because subgroup sizes vary. Small subgroups produce wider limits, while large subgroups produce narrower limits.

It is important not to conclude that everything is acceptable simply because no monthly average exceeds the target of 30 minutes. That target applies to individual cases, not to monthly averages. As the I chart already showed, individual cases may exceed the target even when the monthly means remain comfortably below it.

6.4.4 S chart

The S chart is usually plotted alongside the X-bar chart and shows within-subgroup variation. It is useful for detecting changes in the spread of the data over time.

To calculate the centre line and control limits for the S chart, we need the process standard deviation, $\bar{S}$, together with the constants B_3 and B_4 from Table 6.1.

```
with(csect.agg, {
  sbar    <- weighted.mean(sd, n)  # pooled SD, centre line
  b3      <- b3(n)                 # B3 constant
```

```
  b4      <- b4(n)                # B4 constant

  spc(x   = month,
      y   = sd,
      cl  = sbar,
      lcl = b3 * sbar,
      ucl = b4 * sbar)
})
```

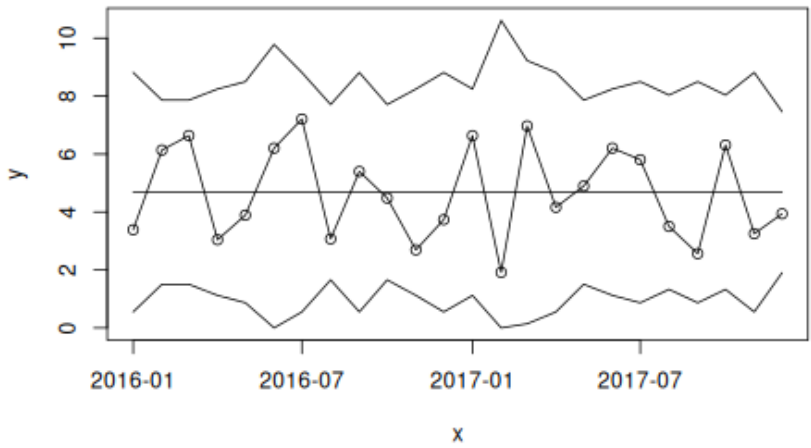

Figure 6.9: S chart

Figure 6.9 shows the monthly standard deviations of delay times. The average standard deviation is 4.7 minutes, and all points lie within the control limits.

We may also plot the X-bar and S charts together (Figure 6.10):

```
with(csect.agg, {
  xbarbar <- weighted.mean(mean, n)  # pooled average
  sbar    <- weighted.mean(sd, n)    # pooled standard deviation
  a3      <- a3(n)                   # A3 constant
  b3      <- b3(n)                   # B3 constant
  b4      <- b4(n)                   # B4 constant

  op <- par(mfrow = c(2, 1),
            mar   = c(3, 5, 2, 1))
  spc(month, mean,
      cl   = xbarbar,
      lcl  = xbarbar - a3 * sbar,
      ucl  = xbarbar + a3 * sbar,
      main = 'X-bar Chart',
      xlab = '')
```

```
  spc(month, sd,
      cl   = sbar,
      lcl  = b3 * sbar,
      ucl  = b4 * sbar,
      main = 'S chart',
      xlab = '')
  par(op)
})
```

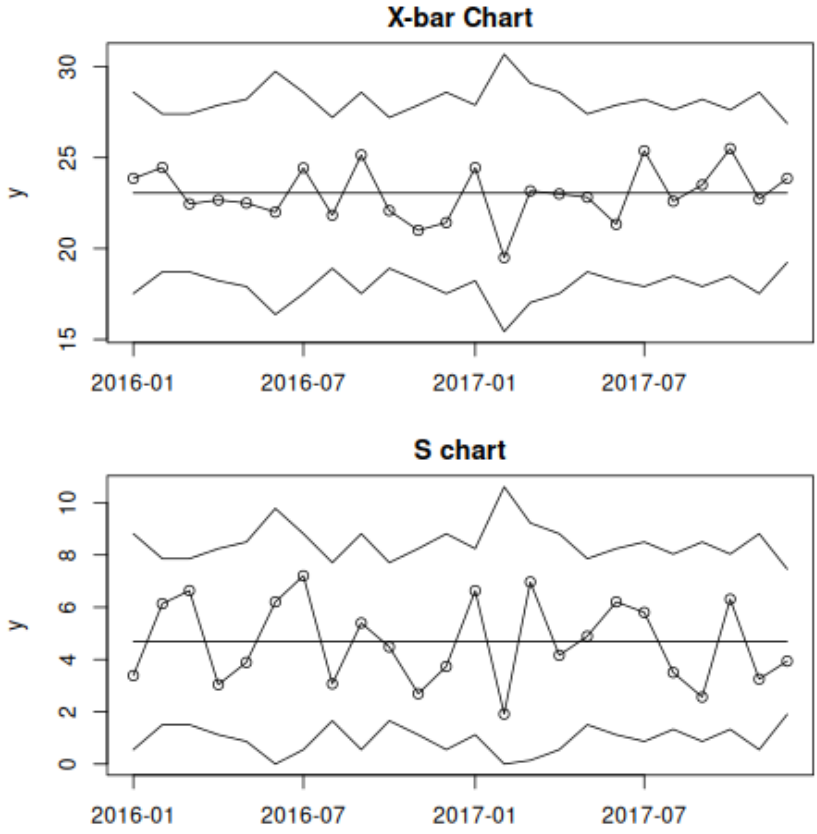

Figure 6.10: X-bar and S charts

6.5 Control limits in short

Control limits are intended to estimate the boundaries of natural common cause variation. They are placed three standard deviations above and below

the centre line, which is usually the mean or weighted mean of the subgroup means.

The exact calculation depends on the type of data and the type of chart, but the interpretation remains the same regardless of chart type.

In the next chapter, we improve our plots by adding visual cues that highlight signs of non-random variation in the data.

Control chart constants

```
a3 <- function(n) {
  3 / (c4(n) * sqrt(n))
}

b3 <- function(n) {
  pmax(0, 1 - 3 * c5(n) / c4(n))
}

b4 <- function(n) {
  1 + 3 * c5(n) / c4(n)
}

c4 <- function(n) {
  n[n <= 1] <- NA
  sqrt(2 / (n - 1)) * exp(lgamma(n / 2) - lgamma((n - 1) / 2))
}

c5 <- function(n) {
  sqrt(1 - c4(n)^2)
}
```

Chapter 7

Highlighting Freaks, Shifts, and Trends

In Chapter 6 we calculated control limits for commonly used control charts. These limits define the boundaries of natural common cause variation. Data points falling outside the control limits – freaks – are therefore signals of special cause variation, that is, unexpected change caused by something outside the usual system.

Control limits are designed primarily to detect relatively large, and possibly transient, changes in a process. However, as discussed in Chapter 3, smaller but sustained changes in the form of shifts or trends may remain undetected by the control limits for long periods. For this reason, a number of supplementary tests, or rules, have been proposed. To balance the need for timely detection of special causes against the risk of false alarms, we recommend using the 3-sigma rule to identify freaks and the two runs rules – unusually long runs and unusually few crossings – to identify shifts and trends.

In this chapter, we will improve the `spc()` function so that it automatically highlights special cause variation in the form of freaks, shifts, and trends.

7.1 Introducing the cdiff data set

The Clostridioides difficile infections data set contains 24 monthly observations of hospital-acquired C. diff. infections from an acute care hospital.

```
# read data from file
cdiff <- read.csv('data/cdiff.csv',
                  comment.char = '#',
                  colClasses = c('Date',
                                 'integer',
                                 'integer'))

# calculate centre line and control limits
cdiff <- within(cdiff, {
  cl  <- mean(infections)
  lcl <- pmax(0, cl - 3 * sqrt(cl)) # censor lcl at zero
  ucl <- cl + 3 * sqrt(cl)
})

# print the first six rows of data
head(cdiff)
```

```
##        month infections risk_days      ucl lcl       cl
## 1 2020-01-01         12     19801 11.77776   0 5.041667
## 2 2020-02-01          7     18674 11.77776   0 5.041667
## 3 2020-03-01          1     15077 11.77776   0 5.041667
## 4 2020-04-01          4     12062 11.77776   0 5.041667
## 5 2020-05-01          4     14005 11.77776   0 5.041667
## 6 2020-06-01          5     14840 11.77776   0 5.041667
```

Figure 7.1 shows these data plotted with the `spc()` function. We can see that one data point (#1) lies above the upper control limit. Looking more closely, we also find an unusually long run of 11 data points below the centre line (#14–#24), and the line crosses the centre line only 7 times. Thus, in addition to the freak, there is also a shift in the data, not large enough to break the control limits, but sustained enough to trigger the runs rules.

```
with(cdiff, {
  spc(month, infections, cl, lcl, ucl)
})
```

Notice that the lower control limit has been censored at zero, even though its exact value is negative. This is done purely for cosmetic reasons. Since the data themselves cannot take negative values, it makes little sense to display negative control limits. For the same reason, P chart limits are often censored at 0 and 1 (or 100%).

To help us identify special cause variation more easily, we now improve the `spc()` function.

7.2 An improved `spc()` function

The revised version of the `spc()` function includes a number of changes.

First, it imports the function `runs.analysis()` from a separate R script in order to test for unusually long runs and unusually few crossings. This function is displayed at the end of this chapter.

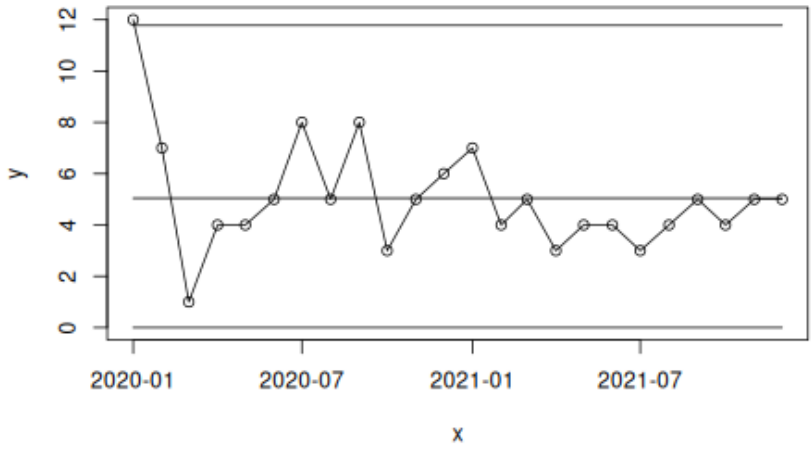

Figure 7.1: C control chart of Clostrioides difficile infections

Second, if no centre line is supplied, the function uses the median of the data.

Third, it creates a logical vector identifying points that fall outside the control limits.

Fourth, it performs runs analysis to check for sustained shifts and trends.

Finally, it changes the plotting so that the plot is first created empty, after which the centre line, control limits, and data are added in a way that allows special cause signals to be highlighted visually.

```
spc <- function(
    x,            # x axis values
    y   = NULL,   # data values
    cl  = NULL,   # centre line
    lcl = NA,     # lower control limit
    ucl = NA,     # upper control limit
    ...           # other parameters passed to the plot() function
) {
  # load runs analysis function from R script
  source('R/runs.analysis.R')

  # if y is missing, set y to x and make a sequence for x
  if (is.null(y)) {
    y <- x
    x <- seq_along(y)
  }

  # if cl is missing use median of y
```

```
  if (is.null(cl))
    cl <- median(y, na.rm = TRUE)

  # repeat line values to match the length of y
  if (length(cl) == 1)
    cl <- rep(cl, length(y))

  if (length(lcl) == 1)
    lcl <- rep(lcl, length(y))

  if (length(ucl) == 1)
    ucl <- rep(ucl, length(y))

  # find data points outside control limits (freaks)
  sigma.signal <- y < lcl | y > ucl
  sigma.signal[is.na(sigma.signal)] <- FALSE

  # check for sustained shifts and trends using runs analysis
  runs.signal <- runs.analysis(y, cl)

  # make empty plot
  plot(x, y,
       type = 'n',
       ylim = range(y, cl, lcl, ucl, na.rm = TRUE),
       ...)

  # add centre line, coloured and dashed if shifts or trends were identified by
  # the runs analysis
  lines(x, cl,
        col = runs.signal + 1,
        lty = runs.signal + 1)

  # add control limits
  lines(x, lcl)
  lines(x, ucl)

  # add data line and points, colour freak data points (outside control limits)
  lines(x, y)
  points(x, y,
         pch = 19,
         col = sigma.signal + 1)
}
```

```
with(cdiff, {
  spc(month, infections, cl, lcl, ucl)
})
```

The chart now makes it much easier to see immediately whether special cause variation is present (Figure 7.2). Freak data points are shown in red, and the centre line turns red and dashed when the runs analysis identifies unusually long runs or unusually few crossings.

It is important to remember, however, that the chart itself does not explain the cause of the signal. Interpretation still depends on people with a good understanding of both the process and the data.

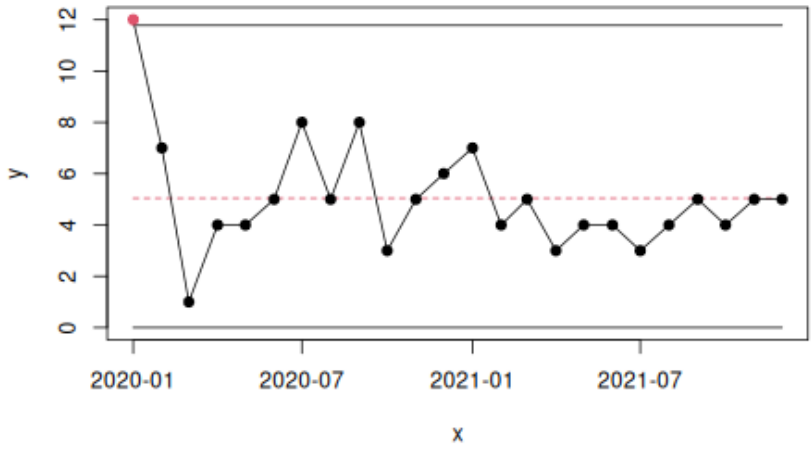

Figure 7.2: Improved control chart with visual clues to highlight special cause variation

7.3 Highlighting special cause variation in short

In this chapter, we improved the `spc()` function so that it automatically highlights signals of special cause variation by using visual cues to signal freaks, shifts, and trends.

In the next chapter, we will improve the spc() function further so that it can automatically aggregate data and calculate control limits.

R function for runs analysis

```
runs.analysis <- function(y, cl) {
  # trichotomise data according to position relative to CL
  # -1 = below, 0 = on, 1 = above
  runs            <- sign(y - cl)

  # remove NAs and data points on the CL
```

```
  runs            <- runs[runs != 0 & !is.na(runs)]

  # find run lengths
  run.lengths     <- rle(runs)$lengths

  # find number of useful observations (data points not on CL)
  n.useful        <- sum(run.lengths)

  # find longest run above or below CL
  longest.run     <- max(run.lengths)

  # find number of times adjacent data points are on opposite sides of CL
  n.crossings     <- length(run.lengths) - 1

  # find upper limit for longest run
  longest.run.max <- round(log2(n.useful)) + 3

  # find lower limit for number of crossing
  n.crossings.min <- qbinom(0.05, n.useful - 1, 0.5)

  # return result, TRUE if either of the two tests is true, otherwise FALSE
  longest.run > longest.run.max | n.crossings < n.crossings.min
}
```

Chapter 8

Core R Functions to Construct SPC Charts

Until now, we have calculated control limits manually before plotting. In this chapter, we introduce a library of R functions that work together to automate the main steps involved in constructing SPC charts.

We will not go through each of these functions in detail, but we encourage you to study them carefully in order to understand how they work individually and how they work together. We also encourage you to adapt and improve them to suit your own needs.

In total, the library consists of 17 functions. However, the user only needs to interact directly with one of them: `spc()`. The source code for the full function library is provided in the section at the end of this chapter.

The main function, `spc()`, is an extended version of the improved `spc()` function introduced in Chapter 7.

Most importantly, it is no longer necessary to calculate control limits manually. Instead, we provide a `chart` argument, which should be one of the following: 'run', 'xbar', 's', 'i', 'mr', 'c', 'u', or 'p'. If no chart argument is supplied, the function produces a run chart.

We also no longer need to use the somewhat clumsy $ notation or the `with()` function to access variables inside a data frame. Instead, we can pass the name of the data frame to the data argument.

Finally, for X-bar and S charts, there is no longer any need to aggregate the data in advance. That task is handled by the `spc.aggregate()` function, which is also responsible for calling the appropriate functions to calculate centre lines and control limits and to perform the runs analysis.

After completing these steps, `spc.aggregate()` returns a data frame containing all the information needed to construct the plot. The actual drawing of the chart is then handled by the `plot.spc()` function.

8.1 Examples

This section gives examples of a run chart and each of the *Magnificent Seven*. Documentation for the datasets used in the examples is provided in Appendix A.

8.1.1 Run chart

```
d <- read.csv('data/blood_pressure.csv',
              comment.char = '#',
              colClasses   = c(date = 'Date'))

head(d)
```

```
##         date systolic diastolic pulse
## 1 2012-06-13      125        77    56
## 2 2012-06-14      118        76    59
## 3 2012-06-15      125        75    59
## 4 2012-06-16      126        73    69
## 5 2012-06-17      124        77    70
## 6 2012-06-18      127        83    63
```

```
spc(date, systolic,
    data = d,
    main = 'Systolic blood pressure',
    ylab = 'mm Hg',
    xlab = 'Date')
```

8.1.2 I and MR charts for individual measurements

```
# Setup plotting area to hold two plots on top of each other and adjust margins
op <- par(mfrow = c(2, 1),
          mar   = c(4, 4, 2, 0) + 0.2)
spc(date, systolic,
    data  = d,
    chart = 'i',
    main  = 'Systolic blood pressure',
    ylab  = 'mm Hg',
    xlab  = NA)
spc(date, systolic,
    data  = d,
    chart = 'mr',
    main  = 'Moving range',
    ylab  = 'mm Hg',
    xlab  = 'Date')
```

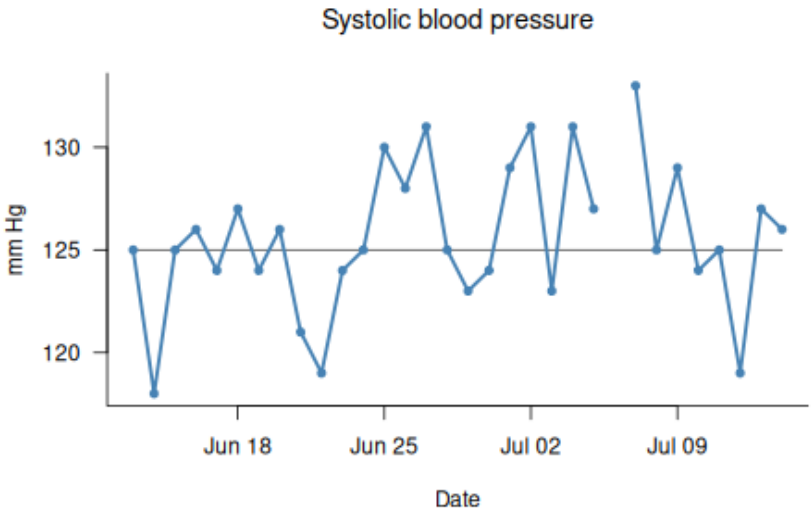

Figure 8.1: Run chart

```
# Reset plotting area to default
par(op)
```

8.1.3 X-bar and S charts for averages and standard deviations of measurements

```
d <- read.csv('data/renography_doses.csv',
              comment.char = '#',
              colClasses   = c(date = 'Date',
                               week = 'Date'))

head(d)
```

```
##         date       week dose
## 1 2014-01-06 2014-01-06   51
## 2 2014-01-06 2014-01-06   48
## 3 2014-01-06 2014-01-06   75
## 4 2014-01-06 2014-01-06   53
## 5 2014-01-06 2014-01-06   71
## 6 2014-01-06 2014-01-06   84
```

```
op <- par(mfrow = c(2, 1),
          mar   = c(4, 4, 2, 0) + 0.2)
spc(week, dose,
```

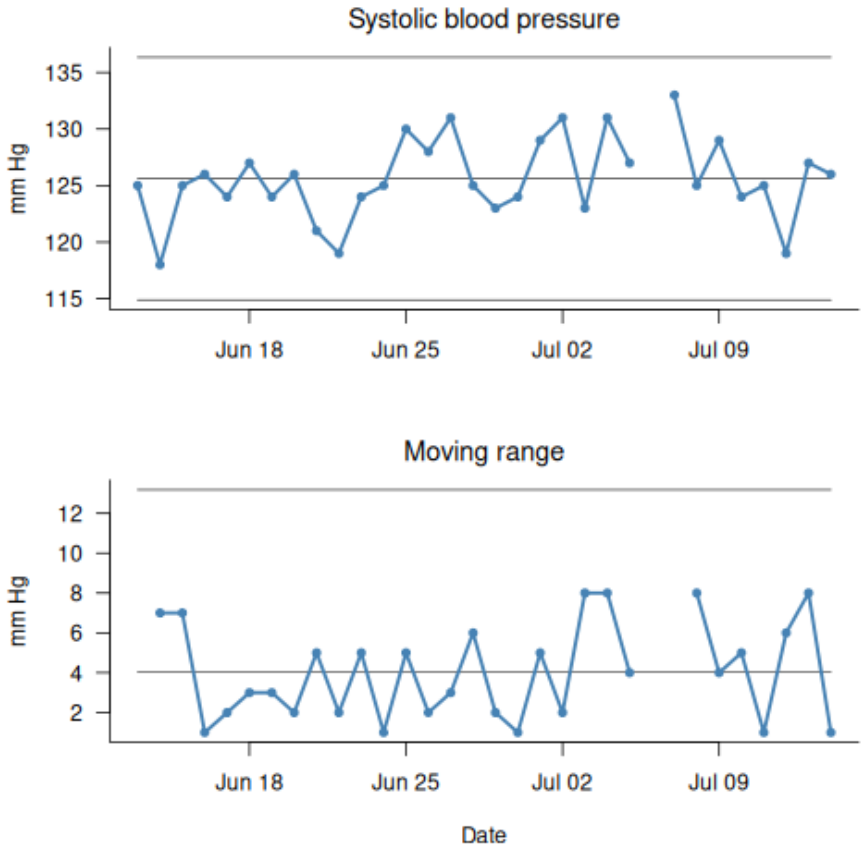

Figure 8.2: I and MR charts

```
    data  = d,
    chart = 'xbar',
    main  = 'Average radiation dose for renography',
    ylab  = 'MBq',
    xlab  = NA)
spc(week, dose,
    data  = d,
    chart = 's',
    main  = 'Standard deviation',
    ylab  = 'MBq',
    xlab  = 'Week')
```

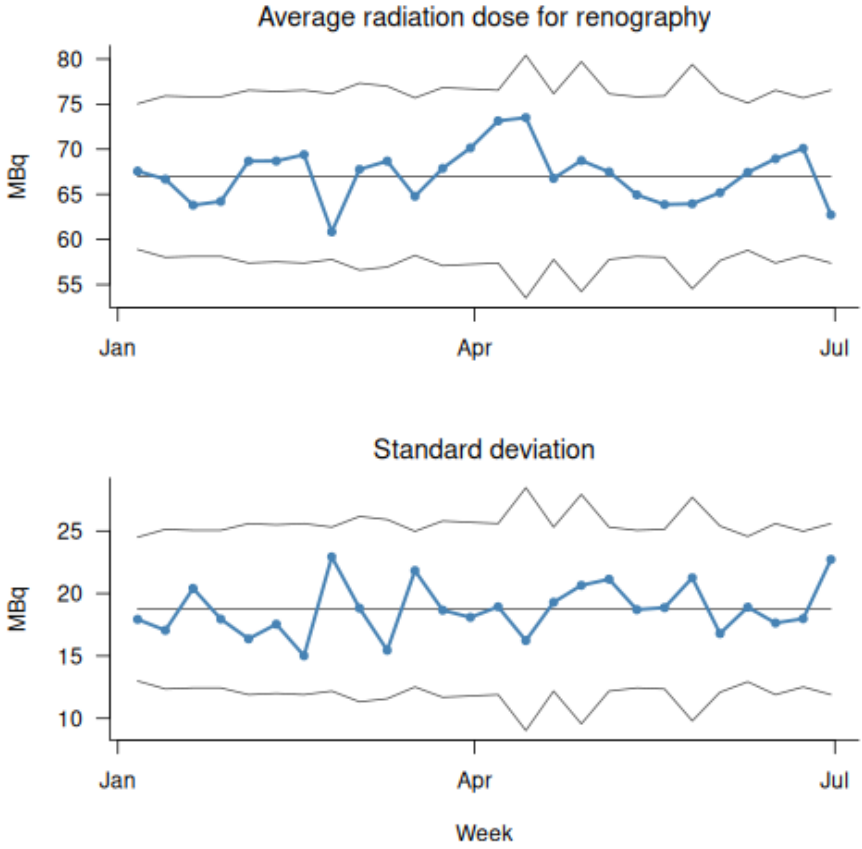

Figure 8.3: X-bar and S charts

```
par(op)
```

8.1.4 C and U charts for counts and rates

```
d <- read.csv('data/bacteremia.csv',
         comment.char = '#',
         colClasses   = c(month = 'Date'))

head(d)
```

```
##        month ha_infections risk_days deaths patients
## 1 2017-01-01            24     32421     23      100
## 2 2017-02-01            29     29349     22      105
## 3 2017-03-01            26     32981     13       99
## 4 2017-04-01            16     29588     14       85
## 5 2017-05-01            28     30856     17       98
## 6 2017-06-01            16     30544     15       85
```

```
op <- par(mfrow = c(2, 1),
          mar   = c(4, 4, 2, 0) + 0.2)
spc(month, ha_infections,
    data = d,
    chart    = 'c',
    main     = 'Hospital-acquired bacteremia',
    ylab     = 'Count',
    xlab     = NA)
spc(month, ha_infections, risk_days,
    data     = d,
    chart    = 'u',
    multiply = 10000,
    main     = NA,
    ylab     = 'Count per 10,000 risk-days',
    xlab     = 'Month')
```

```
par(op)
```

8.1.5 P chart for percentages

```
spc(month, deaths, patients,
    data     = d,
    multiply = 100,
    chart    = 'p',
    main     = '30-day mortality after bacteremia',
    ylab     = '%',
    xlab     = 'Month')
```

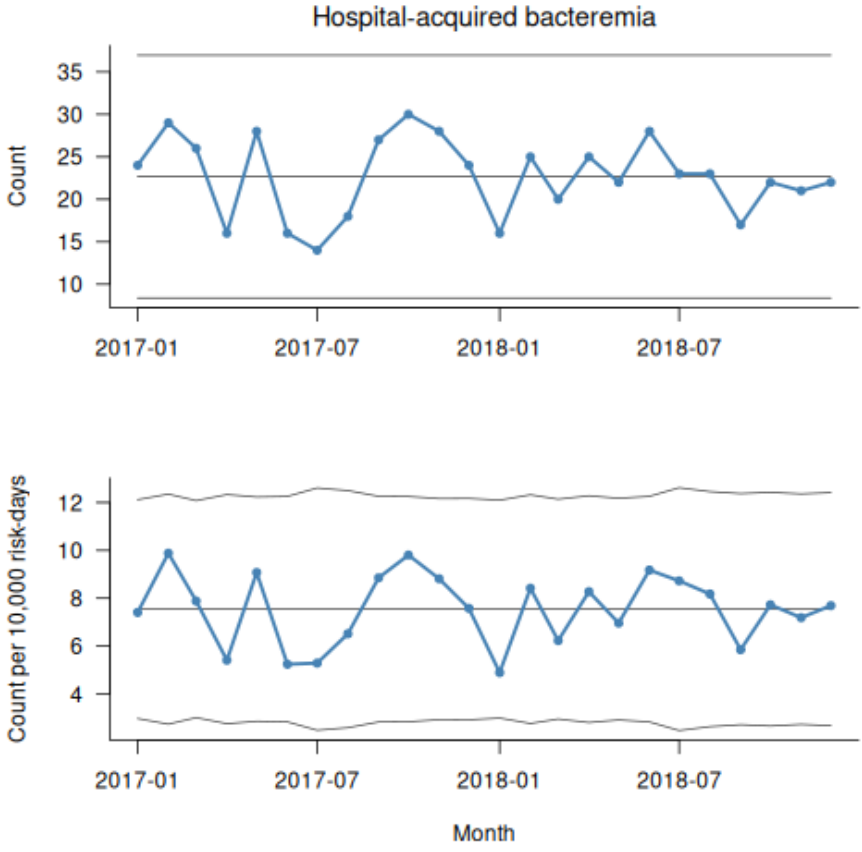

Figure 8.4: C and U charts

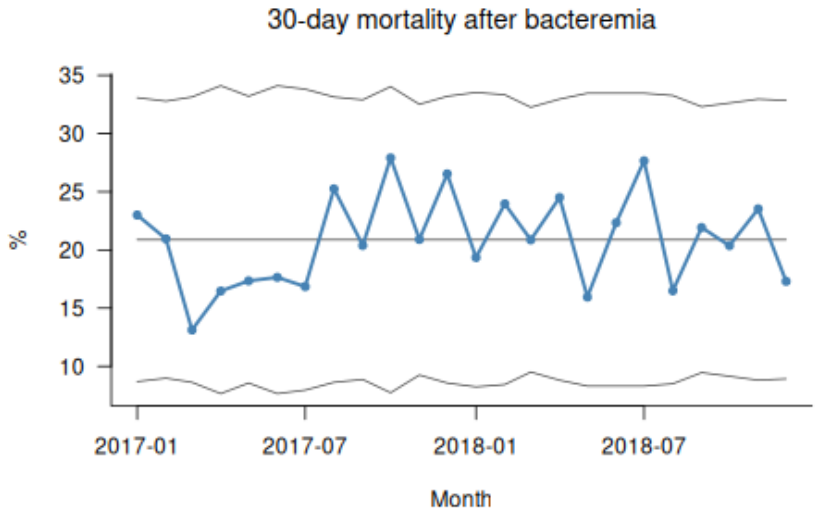

Figure 8.5: P chart

8.2 TODO

One obvious shortcoming of this function library is that it lacks error checking. If you plan to use these functions in a production environment or as part of your own SPC package, you will need to add this yourself. At a minimum, you should check automatically that the inputs are of the expected types and that the x, y, and n arguments have the same lengths. As a starting point, see the `stopifnot()` function.

8.3 Further up, further in

We now have a functioning library of R functions that automates most of the steps involved in constructing SPC charts. There is still plenty of room for improvement, but the library should be enough to get you started and, more importantly, to give you a deeper understanding of the considerations involved in plotting SPC charts.

In the next chapter, we take a brief look at how to use *ggplot2()* for plotting instead of the `plot()` function from base R.

R function library

```
# Master SPC function ########################################################
#
# Constructs an SPC chart.
#
#  Invisibly returns a data frame of class 'spc'.
#
#  x:        Numeric or date(time) vector of subgroup values to plot along the x
#            axis. Or, if y is NULL, x values will be used for y coordinates.
#  y:        Numeric vector of measures or counts to
#            plot on the y axis (numerator).
#  n:        Numeric vector of subgroup sizes (denominator).
#  data:     Data frame containing the variables used in the plot.
#  multiply: Number to multiply y axis by, e.g. 100 to get percentages rather
#            than proportions.
#  chart:    Character value indicating the chart type. Possible values are:
#            'run' (default), 'xbar', 's', 'i', 'mr', 'c', 'u', and 'p'.
#  plot:     Logical, if TRUE (default), plots an SPC chart.
#  print:    Logical, if TRUE, prints a data frame with coordinates.
#  ...:      Other arguments to the plot() function, e.g. main, ylab, xlab.
#
spc <- function(x,
                y        = NULL,
                n        = 1,
                data     = NULL,
                multiply = 1,
                chart    = c('run', 'xbar', 's', 'i', 'mr', 'c', 'u', 'p'),
                plot     = TRUE,
                print    = FALSE,
                ...) {
  # Get data from data frame if data argument is provided, or else get data
  # from the parent environment.
  x       <- eval(substitute(x), data, parent.frame())
  y       <- eval(substitute(y), data, parent.frame())
  n       <- eval(substitute(n), data, parent.frame())

  # Get chart argument.
  chart  <- match.arg(chart)

  # If y argument is missing, use x instead.
  if (is.null(y)) {
    y <- x
    x <- seq_along(y)
  }

  # Make sure that the n vector has same length as y.
  if (length(n) == 1) {
    n <- rep(n, length(y))
  }

  # Make sure that numerators and denominators are balanced. If one is missing,
  # the other should be missing too.
  xna    <- !complete.cases(data.frame(y, n))
  y[xna] <- NA
  n[xna] <- NA

  # Aggregate data by subgroups.
  df <- spc.aggregate(x, y, n, chart)

  # Multiply y coordinates if needed.
  df$y   <- df$y * multiply
  df$cl  <- df$cl * multiply
  df$lcl <- df$lcl * multiply
  df$ucl <- df$ucl * multiply
```

```r
  # Make plot.
  if (plot) {
    plot.spc(df, ...)
  }

  # Print data frame.
  if (print) {
    print(df)
  }

  # Make data frame an 'spc' object and return invisibly.
  class(df) <- c('spc', class(df))
  invisible(df)
}

# Aggregate function ##########################################################
#
#  Calculates subgroup lengths, sums, means, and standard deviations. Called
#  from the spc() function.
#
#  Returns a data frame of x, y, n, and centre line and control limits.
#
#  x:     Numerical, numbers or dates for the x axis.
#  y:     Numerical, measure or count to plot.
#  n:     Numerical, denominator (if any).
#  chart: Character, type of chart.
#
spc.aggregate <- function(x, y, n, chart) {
  df <- data.frame(x, y, n)

  # Get function to calculate centre line and control limits.
  chart.fun  <- get(paste('spc', chart, sep = '.'))

  # Split data frame by subgroups
  df <- split(df, df$x)

  # Calculate subgroup lengths, sums, means, and standard deviations.
  df <- lapply(df, function(x) {
    data.frame(x     = x$x[1],
               n     = sum(x$n, na.rm = TRUE),
               sum   = sum(x$y, na.rm = TRUE),
               mean  = sum(x$y, na.rm = TRUE) / sum(x$n, na.rm = TRUE),
               sd    = sd(x$y, na.rm = TRUE),
               chart = chart)
  })

  # Put list elements back together in a data frame.
  df <- do.call(rbind, c(df, make.row.names = FALSE))

  # Replace any zero length subgroups with NA.
  df$n[df$n == 0] <- NA

  # Calculate the weighted subgroup mean.
  df$ybar <- weighted.mean(df$mean, df$n, na.rm = TRUE)

  # Calculate centre line and control limits.
  df <- chart.fun(df)

  # Add runs analysis
  if (chart == 'mr') {
    df$runs.signal <- FALSE
  } else {
    df$runs.signal <- runs.analysis(df$y, df$cl)
  }

  # Find data points outside control limits.
  df$sigma.signal                         <- (df$y < df$lcl | df$y > df$ucl)
  df$sigma.signal[is.na(df$sigma.signal)] <- FALSE
```

```
  # Return data frame.
  df[c('x', 'y', 'n',
       'lcl', 'cl', 'ucl',
       'sigma.signal',
       'runs.signal',
       'chart')]
}

# Plot function #####################################################################
#
#  Draws an SPC chart from data provided by the spc() function. Is usually
#  called from the spc() function, but may be used as stand-alone for plotting
#  data frames of class spc created by the spc() function.
#
#  Invisibly returns the data frame
#
#  x:   Data frame produced by the spc() function.
#  ...: Additional arguments for the plot() function, e.g. title and labels.
#
plot.spc <- function(x, ...) {
  col1 <- 'steelblue'
  col2 <- 'grey30'
  col3 <- 'tomato'

  # Make room for data and control limits on the x axis.
  ylim <- range(x$y,
                x$lcl,
                x$ucl,
                na.rm = TRUE)

  # Draw empty plot.
  plot(x$x, x$y,
       type = 'n',
       bty  = 'l',
       las  = 1,
       ylim = ylim,
       font.main = 1,
       ...)

  # Add lines and points to plot.
  lines(x$x, x$cl,
        col = ifelse(x$runs.signal, col3, col2),
        lty = ifelse(x$runs.signal, 'dashed', 'solid'))
  lines(x$x, x$lcl, col = col2)
  lines(x$x, x$ucl, col = col2)
  lines(x$x, x$y, col = col1, lwd = 2.5)
  points(x$x, x$y,
         pch = 19,
         cex = 0.8,
         col = ifelse(x$sigma.signal, col3, col1)
  )

  invisible(x)
}

# Runs analysis function ############################################################
#
#  Tests time series data for non-random variation in the form of
#  unusually long runs or unusually few crossings. Called from the
#  spc.aggregate() function.
#
#  Returns a logical, TRUE if non-random variation is present.
#
#  x:  Numeric vector.
#  cl: Numeric vector of length either one or same length as y.
#
runs.analysis <- function(y, cl) {
```

```
  # Trichotomise data according to position relative to CL:
  # -1 = below, 0 = on, 1 = above.
  runs <- sign(y - cl)

  # Remove NAs and data points on the centre line.
  runs <- runs[runs != 0 & !is.na(runs)]

  # Find run lengths.
  run.lengths <- rle(runs)$lengths

  # Find number of useful observations (data points not on centre line).
  n.useful <- sum(run.lengths)

  # Find longest run above or below centre line.
  longest.run <- max(run.lengths)

  # Find number of crossings.
  n.crossings <- length(run.lengths) - 1

  # Find upper limit for longest run.
  longest.run.max <- round(log2(n.useful)) + 3

  # Find lower limit for number of crossing.
  n.crossings.min <- qbinom(0.05, n.useful - 1, 0.5)

  # Return result.
  longest.run > longest.run.max | n.crossings < n.crossings.min
}

# Limits functions ##################################################################
#
#  These functions calculate coordinates for the centre line and control limits
#  of SPC charts. They are not meant to be called directly but are used by the
#  spc.aggregate() function.
#
#  Return data frames with coordinates for centre line and control limits.
#
#  x: A data frame containing the values to plot.
#
spc.run <- function(x) {
  x$y   <- x$mean

  x$cl  <- median(x$y, na.rm = TRUE)
  x$lcl <- NA_real_
  x$ucl <- NA_real_

  x
}

spc.i   <- function(x) {
  x$y   <- x$mean

  xbar  <- x$ybar
  amr   <- mean(abs(diff(x$y)), na.rm = T)
  sss   <- 2.66 * amr

  x$cl  <- xbar
  x$lcl <- xbar - sss
  x$ucl <- xbar + sss

  x
}

spc.mr <- function(x) {
  x$y   <- c(NA, abs(diff(x$mean)))

  amr   <- mean(x$y, na.rm = TRUE)
```

```
  x$cl  <- amr
  x$lcl <- NA_real_
  x$ucl <- 3.267 * amr

  x
}

spc.xbar <- function(x) {
  x$y     <- x$mean

  a3      <- a3(x$n)
  xbarbar <- weighted.mean(x$mean, x$n, na.rm = TRUE)
  sbar    <- weighted.mean(x$sd, x$n, na.rm = TRUE)
  sss     <- a3 * sbar

  x$cl    <- xbarbar
  x$lcl   <- xbarbar - sss
  x$ucl   <- xbarbar + sss

  x
}

spc.s <- function(x) {
  x$y     <- x$sd

  sbar    <- weighted.mean(x$sd, x$n, na.rm = TRUE)
  b3      <- b3(x$n)
  b4      <- b4(x$n)

  x$cl    <- sbar
  x$lcl   <- b3 * sbar
  x$ucl   <- b4 * sbar

  x
}

spc.c <- function(x) {
  x$y   <- x$sum

  cbar  <- mean(x$y, na.rm = TRUE)
  sss   <- 3 * sqrt((cbar))

  x$cl  <- cbar
  x$lcl <- pmax(0, cbar - sss)
  x$ucl <- cbar + sss

  x
}

spc.u <- function(x) {
  x$y <- x$mean

  ubar  <- x$ybar
  sss   <- 3 * sqrt((ubar / x$n))

  x$cl  <- ubar
  x$lcl <- pmax(0, ubar - sss, na.rm = TRUE)
  x$ucl <- ubar + sss

  x
}

spc.p <- function(x) {
  x$y   <- x$mean

  pbar  <- x$ybar
  sss   <- 3 * sqrt((pbar * (1 - pbar)) / x$n)
```

```
  x$cl  <- pbar
  x$lcl <- pmax(0, pbar - sss)
  x$ucl <- pmin(1, pbar + sss)

  x
}

# Constants functions ########################################################
#
#  These functions calculate the constants that are used for calculating the
#  parameters of X-bar and S charts. Called from the spc.xbar() and spc.s()
#  functions
#
#  Return a number, the constant for that subgroup size
#
#  n: Number of elements in subgroup
#
a3 <- function(n) {
  3 / (c4(n) * sqrt(n))
}

b3 <- function(n) {
  pmax(0, 1 - 3 * c5(n) / c4(n), na.rm = TRUE)
}

b4 <- function(n) {
  1 + 3 * c5(n) / c4(n)
}

c4 <- function(n) {
  n[n <= 1] <- NA
  sqrt(2 / (n - 1)) * exp(lgamma(n / 2) - lgamma((n - 1) / 2))
}

c5 <- function(n) {
  sqrt(1 - c4(n) ^ 2)
}
```

Chapter 9

SPC Charts with *ggplot2*

Armed with the set of functions developed in Chapter 8 , we are now able to construct all of the most commonly used SPC charts using base R. Furthermore, the modular structure of the function library makes it straightforward to extend. To add a new type of chart, we simply need to write a corresponding spc.*() function to calculate the centre line and control limits, and then include this function in the chart argument of the main `spc()` function.

Because we have separated the calculation of chart components from the plotting itself – by introducing a dedicated plotting method, `plot.spc()` – it is also easy to replace the base R graphics engine with an alternative like *ggplot2* or *lattice*.

In this chapter, we develop an alternative plotting function based on *ggplot2*. The advantage of *ggplot2* is not that it can do things that base R cannot, but that it often makes common tasks simpler and more consistent.

For example, *ggplot2* automatically handles scaling of axes and layout of labels, so we do not need to adjust these manually when adding elements to a plot. In addition, *ggplot2* provides a powerful theming system that makes it relatively easy to customise the non-data elements of a chart, such as colours, grid lines, and axis formatting.

9.1 Creating an SPC object for later plotting

Using the `spc()` function from the previous chapter, we can create an SPC object and store it for later use:

```
# make spc object
p <- spc(month, infections,
         data  = cdiff,
         chart = 'c',
         plot  = FALSE)
```

Here, we suppress plotting by setting plot = FALSE. The function instead returns a data frame containing all the coordinates and metadata needed to construct the chart. This object is assigned to p, which we can now manipulate like any other R object.

```
# check the class of p
class(p)
```

```
## [1] "spc"        "data.frame"
```

```
# show the first six rows from p
head(p)
```

```
##            x  y n lcl       cl      ucl sigma.signal runs.signal chart
## 1 2020-01-01 12 1   0 5.041667 11.77776         TRUE        TRUE     c
## 2 2020-02-01  7 1   0 5.041667 11.77776        FALSE        TRUE     c
## 3 2020-03-01  1 1   0 5.041667 11.77776        FALSE        TRUE     c
## 4 2020-04-01  4 1   0 5.041667 11.77776        FALSE        TRUE     c
## 5 2020-05-01  4 1   0 5.041667 11.77776        FALSE        TRUE     c
## 6 2020-06-01  5 1   0 5.041667 11.77776        FALSE        TRUE     c
```

Because p is an object of class "spc" we simply call the generic `plot()` function: `plot(p)`.

```
# show the C chart
plot(p) # not necessary to call spc.plot(), just call the generic plot() function
```

The generic `plot()` function automatically dispatches to the specialised `plot.spc()` method (9.1).

9.2 Making a new plot function based on *ggplot2*

We can now define an alternative plotting function that uses *ggplot2*.

```
# Load ggplot2
library(ggplot2)

# Function for plotting spc objects with ggplot()
ggspc <- function(p) {
  # Set colours
  col1    <- 'steelblue'
```

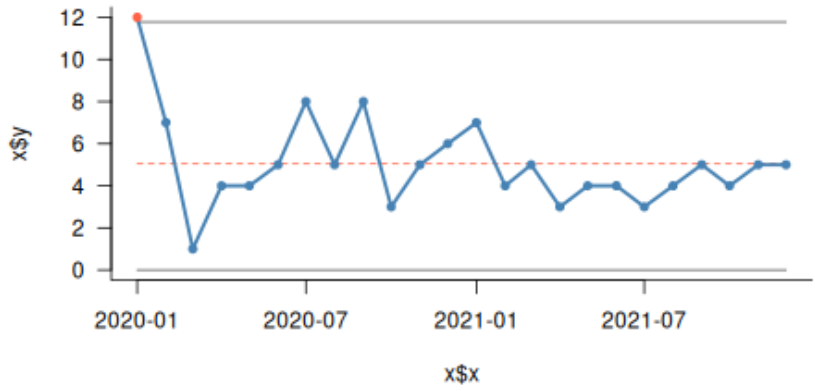

Figure 9.1: SPC chart

```
  col2    <- 'tomato'
  linecol <- 'gray50'
  dotcol  <- ifelse(p$sigma.signal, col2, col1)
  clcol   <- ifelse(p$runs.signal[1], col2, linecol)
  cltyp   <- ifelse(p$runs.signal[1], 'dashed', 'solid')

  # Plot the dots and draw the lines
  ggplot(p, aes(x, y)) +
    geom_line(aes(y = lcl), colour = linecol, na.rm = TRUE) +
    geom_line(aes(y = ucl), colour = linecol, na.rm = TRUE) +
    geom_line(aes(y = cl), colour = clcol, linetype = cltyp, na.rm = TRUE) +
    geom_line(colour = col1, na.rm = TRUE) +
    geom_point(colour = dotcol, na.rm = TRUE)
}
```

We then use this function to plot the SPC object (Figure 9.2).

```
# Plot an spc object
ggspc(p)
```

This function produces a plot equivalent to the base R version, but using *ggplot2*'s grammar of graphics.

We can also convert the SPC object into a ggplot object and then further customise it as in Figure 9.3.

```
# convert to ggplot2 object
p <- ggspc(p)
class(p)
```

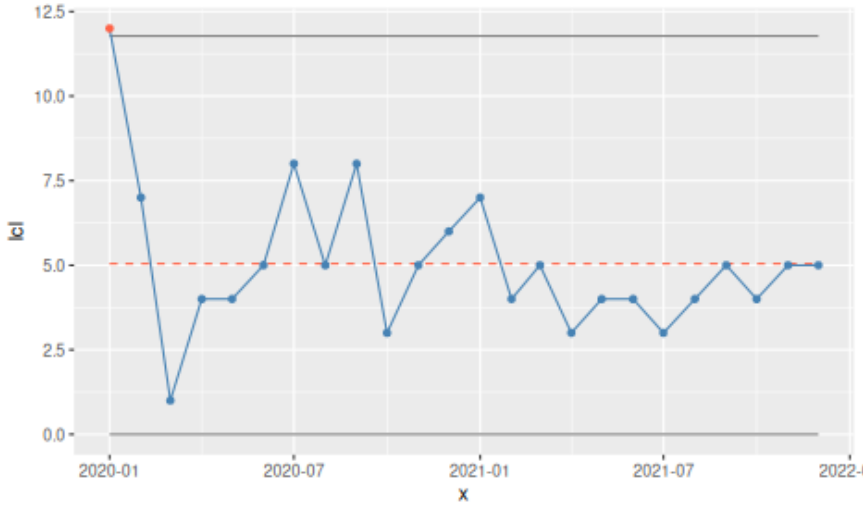

Figure 9.2: SPC chart using the `ggspc()` function

```
## [1] "ggplot2::ggplot" "ggplot"          "ggplot2::gg"     "S7_object"
## [5] "gg"
```

```
# modify theme and labels
p +
  theme_light() +
  theme(panel.grid   = element_blank(),
        panel.border = element_blank(),
        axis.line    = element_line(colour = 'gray')) +
  labs(title = 'CDiff infections',
       y     = 'Count',
       x     = 'Month')
```

At this point, we could replace the existing `plot.spc()` function with this new implementation if desired. The choice between base R and *ggplot2* ultimately comes down to personal preference and use case.

9.3 Customising the plotting theme

The final example in this chapter shows how to define a custom theme and format the y-axis as percentages (Figure 9.4).

```
mytheme <- function() {
  theme_light() +
  theme(panel.grid   = element_blank(),
        panel.border = element_blank(),
```

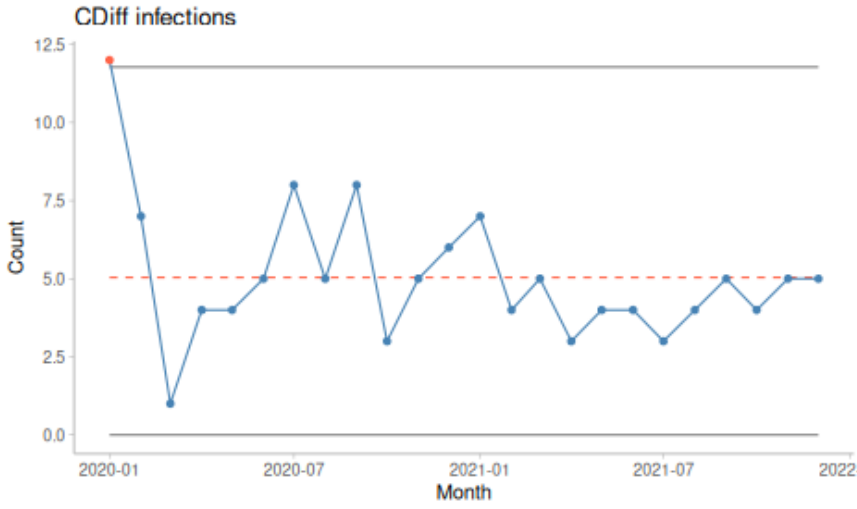

Figure 9.3: SPC chart with modified theme

```
        axis.line    = element_line(colour = 'gray'))
}

p <- spc(month, deaths, patients,
          data = bact,
          chart = 'p',
          plot = FALSE)

ggspc(p) +
  mytheme() +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(title = '30-day mortality',
       y     = NULL,
       x     = 'Month')
```

In practice, repeatedly defining themes and formatting axes for each plot can become tedious. Ideally, these elements would be handled automatically.

9.4 Preparing for *qicharts2*

This is precisely the aim of *qicharts2*, an R package for SPC charts that we introduce in the next chapter. The package builds on the same principles developed so far but provides additional functionality for customisation and automation.

In particular, qicharts2 makes it easy to create multidimensional plots us-

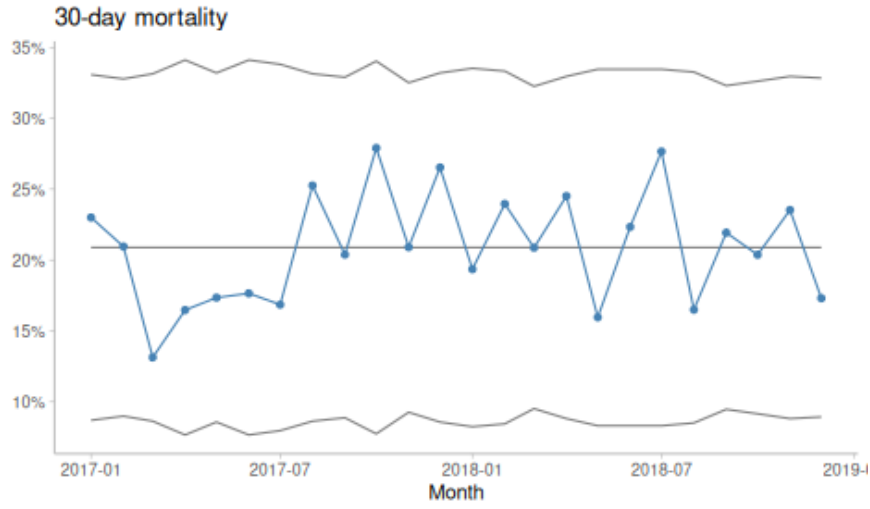

Figure 9.4: SPC chart with labels and custom y-axis

ing *ggplot2*'s faceting capabilities, allowing multiple related charts to be displayed together in a consistent and flexible way.

Chapter 10

SPC Charting with *qicharts2*

qicharts2 (Quality Improvement Charts, Anhøj (2024), Anhøj (2018)) is an R package for statistical process control aimed primarily at healthcare data analysts. It is built on the same principles that we have developed throughout this book and provides functions for constructing run charts and all of the Magnificent Seven. In addition, it includes specialised charts such as Pareto charts and control charts for rare events data.

Full documentation and examples are available on the package website: https://anhoej.github.io/qicharts2/. In this chapter, we focus on a few key features that extend the functionality of the simple function library we developed earlier:

- excluding data points from analysis
- freezing and splitting charts by periods
- multivariate plots (small multiples)

To get started, install and load the package:

```
# install package
install.packages('qicharts2')

# load package
library(qicharts2)
```

Note that installation is only required once, whereas the package must be loaded in each R session.

You may also wish to explore the vignette: `vignette('qicharts2')`.

Alternatively, you can begin using the package immediately.

10.1 A simple run chart

The main function of *qicharts2* is `qic()`. It accepts the same core arguments as the `spc()` function developed earlier, along with many additional options. For a full overview, consult the documentation: `?qic`.

To reproduce our first run chart from Chapter 5:

```
qic(systolic)
```

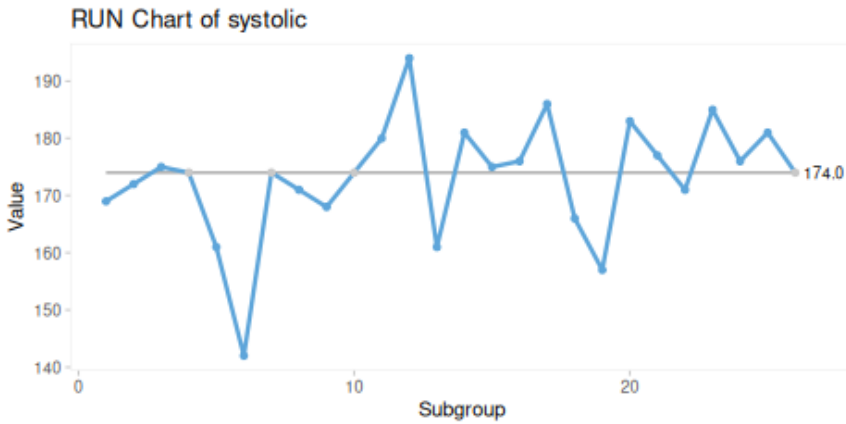

Figure 10.1: Run chart produced with the `qic()` function from the qicharts2 package

Several features are worth noting in Figure 10.1. Default titles and axis labels are generated automatically, though these can be modified using the title, ylab, and xlab arguments. Data points that fall exactly on the centre line are displayed in grey, reflecting that they are not counted as useful observations in runs analysis. The value of the centre line is also displayed directly on the chart.

10.2 A simple control chart

To produce a control chart, we simply specify the chart type (Figure 10.2):

```
qic(systolic, chart = 'i')
```

By default `qic()` uses a grey background area to show the natural process limits.

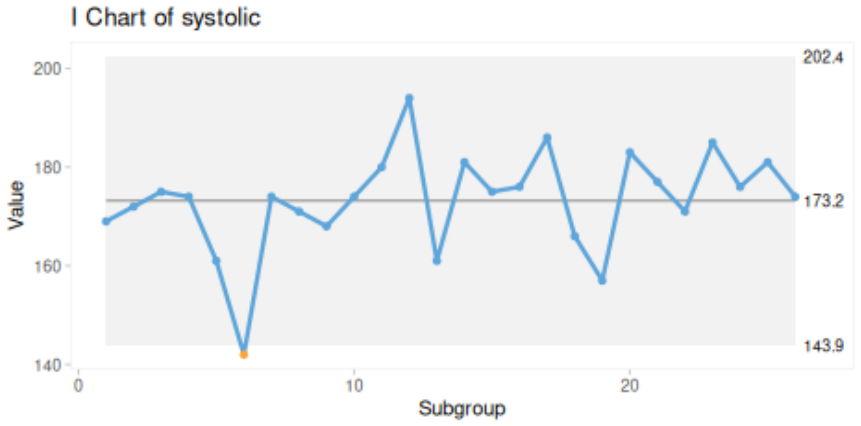

Figure 10.2: I chart produced with `qic()`

10.3 Excluding data points from analysis

In some situations, it may be appropriate to exclude specific data points from the calculation of control limits and runs analysis. This is done using the exclude argument (Figure 10.3):

```
qic(systolic, chart = 'i', exclude = 6)
```

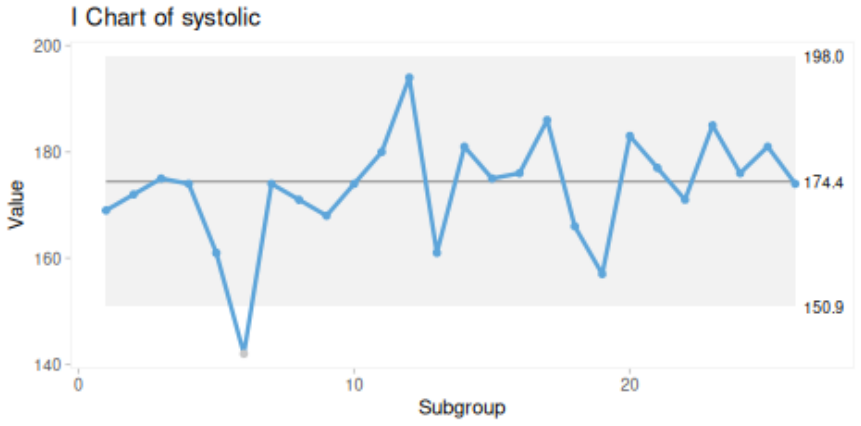

Figure 10.3: I chart with one data point excluded from calculations

Excluding a data point affects the estimated centre line and control limits

– in this example, the limits become slightly narrower.

Such exclusions should be made deliberately and only when there is a clear understanding of why the data point does not represent the underlying process. Excluding points simply because they fall outside the control limits undermines the purpose of SPC, which is to learn from variation.

10.4 Freezing baseline period

When a stable baseline has been established, it is often useful to "freeze" the centre line and control limits and apply them to future data. In industrial settings, this is known as phase I and phase II analysis.

In healthcare, freezing is particularly useful when evaluating the impact of an intervention. The cdi dataset, included with *qicharts2*, contains monthly infection counts before and after an improvement programme:

```
qic(month, n, data = cdi, freeze = 24)
```

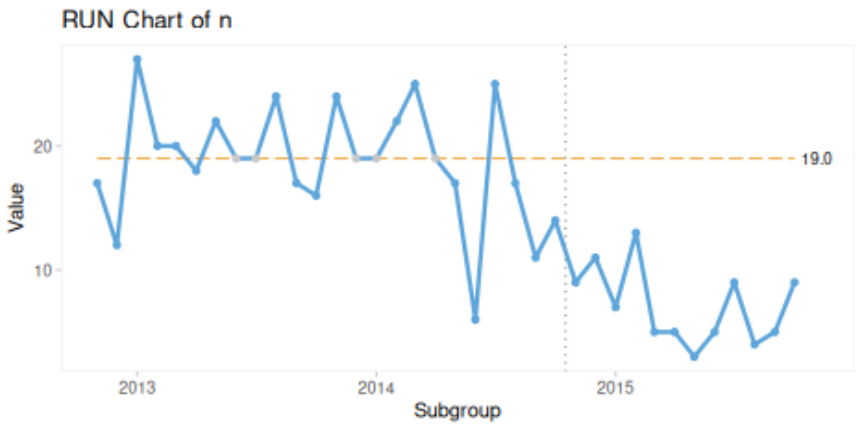

Figure 10.4: Infections before and after intervention

In Figure 10.4, the centre line is calculated from the first 24 months (the baseline period) and applied to subsequent data. This makes it easier to detect sustained changes following the intervention.

10.5 Splitting chart by period

When a sustained shift has been identified and its cause is understood, it may be appropriate to split the chart into distinct periods using the part argument (Figure 10.5):

```
qic(month, n, data = cdi, part = 24)
```

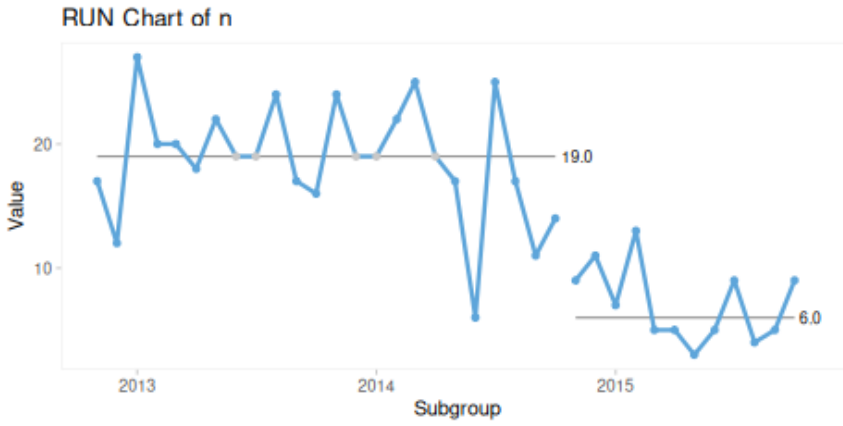

Figure 10.5: Splitting using index

Alternatively, a categorical variable can be used to define periods(Figure 10.6):

```
qic(month, n, data = cdi, part = period)
```

Splitting should be done cautiously and only when justified. It is typically appropriate when:

- a sustained shift is present
- the cause of the shift is known
- the shift is in the desired direction
- the new level is expected to persist

If these conditions are not met, the focus should remain on understanding the underlying causes of variation.

Once distinct stable periods have been identified, control limits may be added: (Figure 10.7).

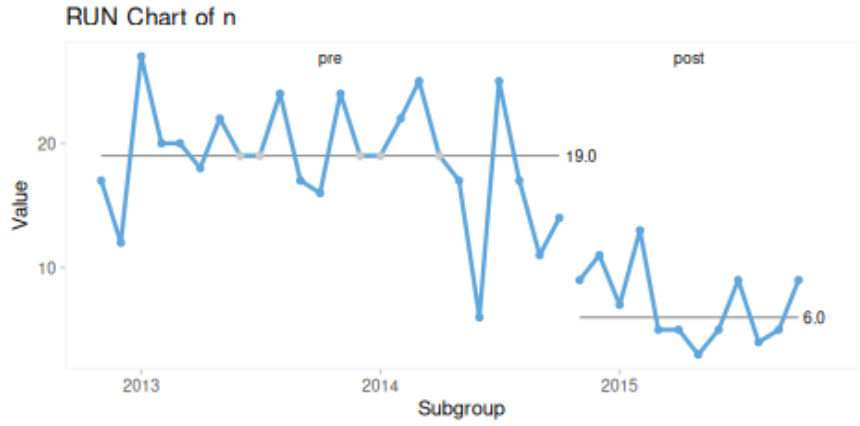

Figure 10.6: Splitting using a period variable

```
qic(month, n, data = cdi, part = period, chart = 'c')
```

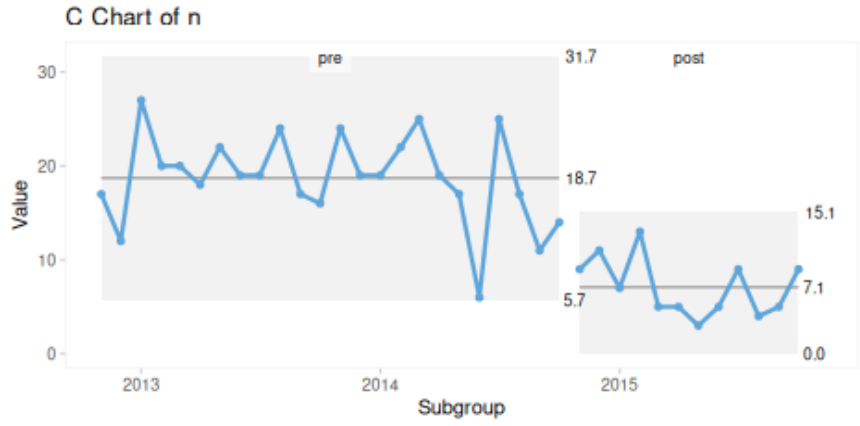

Figure 10.7: Splitting using a period variable

10.6 Small multiple plots for multivariate data

While SPC charts are inherently time-based, many healthcare datasets include additional dimensions that are important for interpretation.

The hospital_infections dataset, included in *qicharts2*, contains infection counts by hospital and infection type:

```
# show the first six rows of the hospitals_infections dataset
head(hospital_infections)
```

```
##   hospital infection      month  n     days
## 1      AHH       BAC 2015-01-01 17 17233.67
## 2      AHH       BAC 2015-02-01 18 15308.25
## 3      AHH       BAC 2015-03-01 17 16883.67
## 4      AHH       BAC 2015-04-01 10 15463.83
## 5      AHH       BAC 2015-05-01 13 15788.96
## 6      AHH       BAC 2015-06-01 14 15660.04
```

In addition to time, the data vary by hospital and infection type.

Figure 10.8 shows aggregated data for urinary tract infections across all hospitals:

```
qic(month, n, days,
    data     = subset(hospital_infections,
                      infection == 'UTI'),
    chart    = 'u',
    multiply = 10000,
    title    = 'Urinary tract infections',
    ylab     = 'Count per 10,000 risk days',
    xlab     = 'Month')
```

The same data can be stratified into separate panels using the facets argument (Figure 10.9):

```
qic(month, n, days,
    data     = subset(hospital_infections,
                      infection == 'UTI'),
    chart    = 'u',
    multiply = 10000,
    facets   = ~hospital,                    # stratify by hospital
    ncol     = 2,                            # two-column arrangement of plots
    title    = 'Urinary tract infections',
    ylab     = 'Count per 10,000 risk days',
    xlab     = 'Month')
```

These small multiple plots (also known as trellis or lattice plots) allow comparison across categories.

Similarly, we can stratify by infection type within a single hospital: (Figure 10.10).

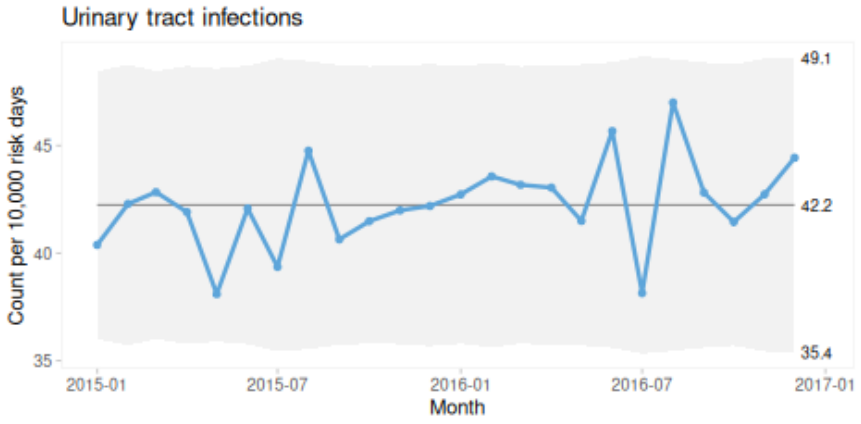

Figure 10.8: Aggregated U chart of urinary tract infections from six hospital

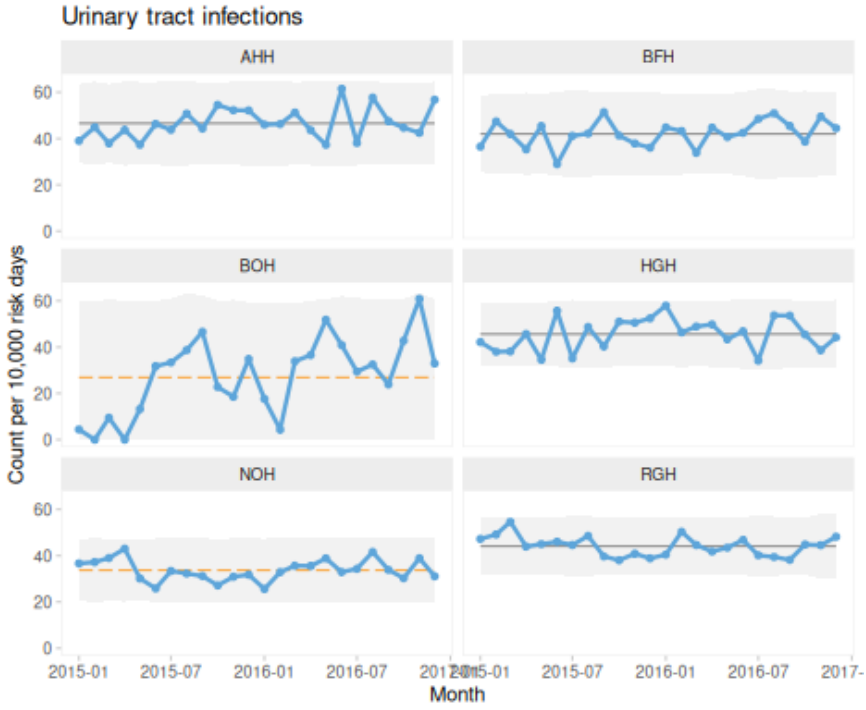

Figure 10.9: Stratified (small multiple) U charts of urinary tract infections from six hospital

```
qic(month, n, days,
    data     = subset(hospital_infections,
                      hospital == 'NOH'),
    chart    = 'u',
    multiply = 10000,
    facets   = ~infection,                    # stratify by infection type
    ncol     = 1,
    title    = 'Hospital infections',
    ylab     = 'Count per 10,000 risk days',
    xlab     = 'Month')
```

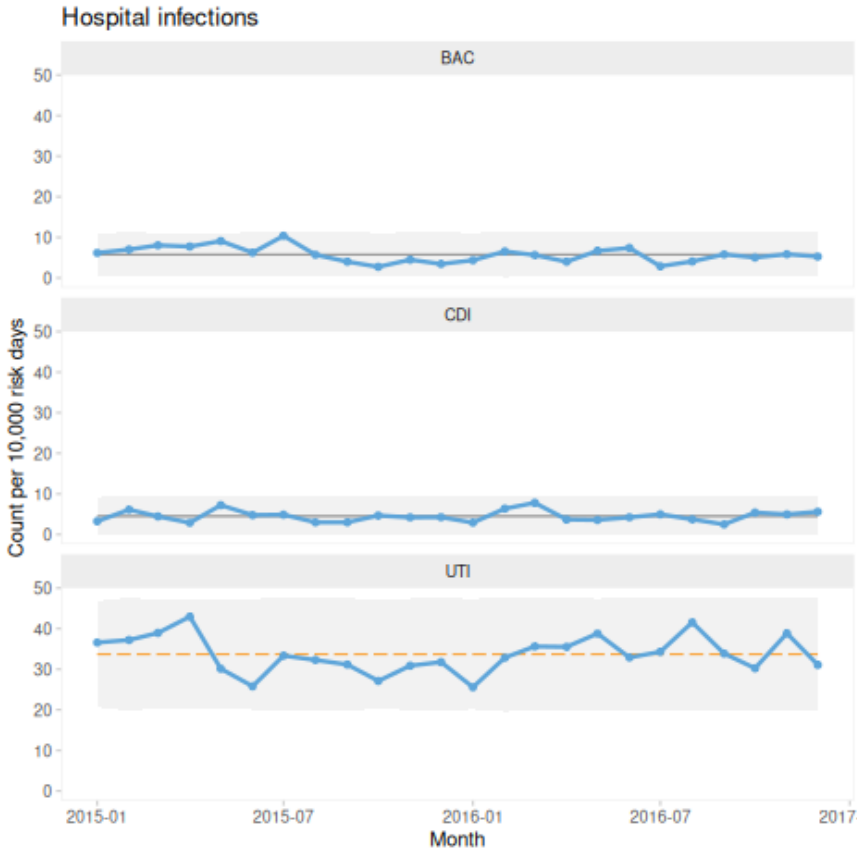

Figure 10.10: U charts from one hospital stratified by infection type

By default, all panels share the same axis scales. This facilitates comparison of levels and patterns. However, when indicators differ greatly in magnitude, it may be more useful to allow separate scales as in Figure 10.11:

```
qic(month, n, days,
    data     = subset(hospital_infections, hospital == 'NOH'),
    chart    = 'u',
    multiply = 10000,
    facets   = ~infection,
    scales   = 'free_y',                    # free y-axes
    ncol     = 1,
    title    = 'Hospital infections',
    ylab     = 'Count per 10,000 risk days',
    xlab     = 'Month')
```

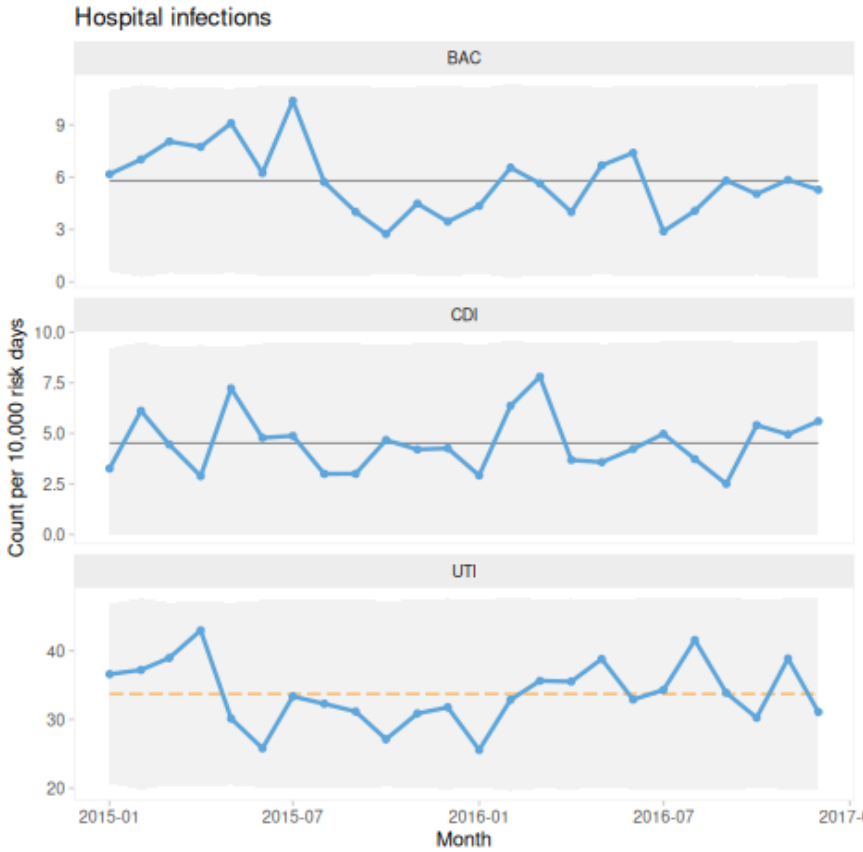

Figure 10.11: Small multiples free y-axes

Finally, both dimensions can be displayed simultaneously as in Figure 10.12:

```
qic(month, n, days,
    data     = hospital_infections,
    chart    = 'u',
    multiply = 10000,
```

```
    facets   = infection ~ hospital,         # two-dimensional faceting
    scales   = 'free_y',
    title    = 'Hospital infections',
    ylab     = 'Count per 10,000 risk days',
    xlab     = 'Month')
```

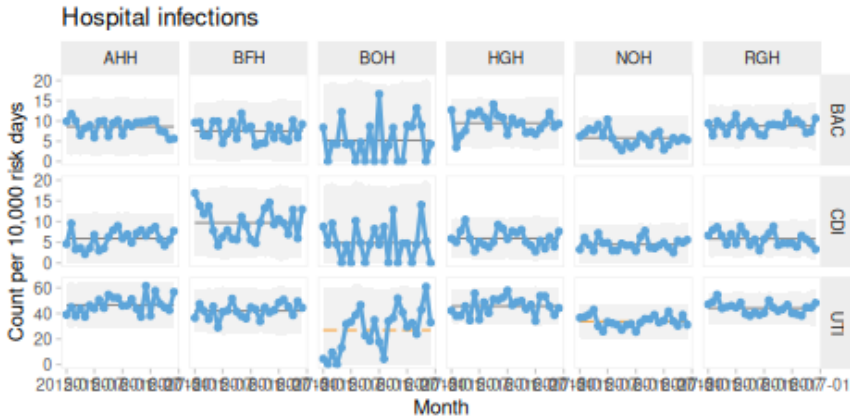

Figure 10.12: Hospital infections stratified by infection and hospital

10.6.1 To aggregate or not to aggregate

When working with data from multiple organisational units, a key decision is whether to aggregate or stratify.

Aggregation can simplify interpretation but may mask important variation. In contrast, stratification can reveal differences between units that would otherwise be hidden. For example, aggregated data may appear stable even when individual units exhibit shifts in opposite directions.

Conversely, small shifts occurring consistently across multiple units may become more visible when data are aggregated.

The choice between aggregation and stratification should therefore be made deliberately, based on an understanding of the context and purpose of the analysis. In many cases, it is useful to examine the data at multiple levels.

10.7 *qicharts2* in short

qicharts2 is a flexible and powerful package for constructing SPC charts, particularly in healthcare settings. It builds on the same principles intro-

duced in this book while providing additional functionality for automation, visualisation, and handling of complex data structures.

In the following chapters, we will explore specialised chart types and further extensions of SPC methodology.

Part 3: Advanced SPC Techniques

Chapter 11

Screening I Charts for Freak Moving Ranges

As discussed in earlier chapters, control limits are designed to represent natural (common cause) variation and thereby enclose almost all data points from a stable process. Estimation of this variation typically relies on within-subgroup variation. However, when subgroups consist of single observations – as in I charts – there is no within-subgroup variation to calculate. Instead, we use moving ranges, defined as the absolute differences between consecutive data points. In effect, pairs of consecutive observations are used to estimate the within-subgroup variation.

This approach has an important implication. If a shift occurs between two consecutive observations, the moving range at that point will be inflated and may produce a signal on the moving range chart.

To illustrate, consider the following 24 values:

```
18 16 8 9 10 11 26 14 15 14 18 19 18 11 28 20 16 17 12 13 24 16 15 11
```

The average moving range (AMR) is 5. From Table 6.1, the natural upper limit for moving ranges is 3.267 x AMR (= 16.3). The moving range between observations 14 and 15 is $|28 - 11| = 17$, which exceeds this limit and is therefore unusually large, as shown in Figure 11.1.

```
qic(y,
    chart = 'mr',
    title = 'Moving range chart')
```

Some authors recommend removing such extreme moving ranges before calculating the control limits for the corresponding I chart, as this may improve

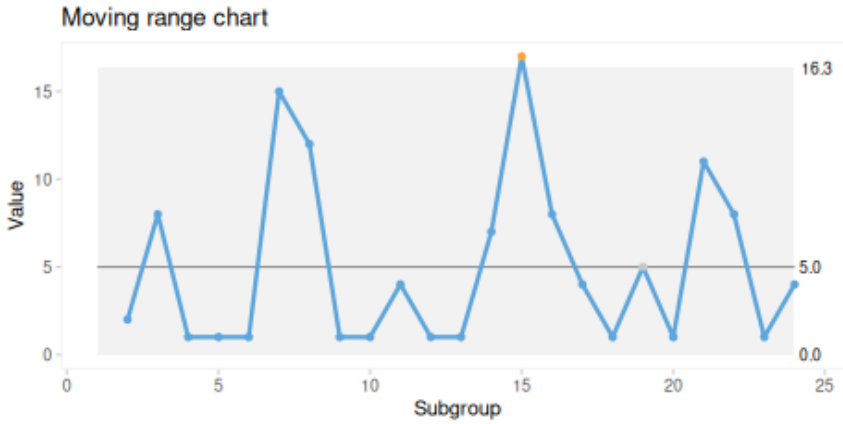

Figure 11.1: Moving range chart with one range above the control limit

sensitivity to meaningful changes (Nelson 1982). By default, qicharts2 applies this approach, as illustrated in Figure 11.2.

```
# default is to screen moving ranges before calculating control limits
qic(y,
    chart = 'i',
    title = 'I chart with screened MRs')
```

The procedure is as follows. First, calculate the AMR from the original data (= 5). Next, remove any moving ranges greater than 3.267 × AMR (= 16.335) and recalculate the AMR (= 4.45). Finally, use this screened AMR to calculate the control limits for the I chart: (= 15.8 ± 2.66 x 4.45).

To suppress this screening step in qicharts2, the qic.screenedmr option can be set to FALSE as in Figure 11.3:

```
# suppress screening of moving ranges
options(qic.screenedmr = FALSE)
qic(y,
    chart = 'i',
    title = 'I chart without screened MRs')
```

```
# reset screening option to default
options(qic.screenedmr = TRUE)
```

As expected, the control limits in the unscreened chart are slightly wider than those in the screened chart – in this example, just wide enough to suppress a signal that would otherwise have been detected.

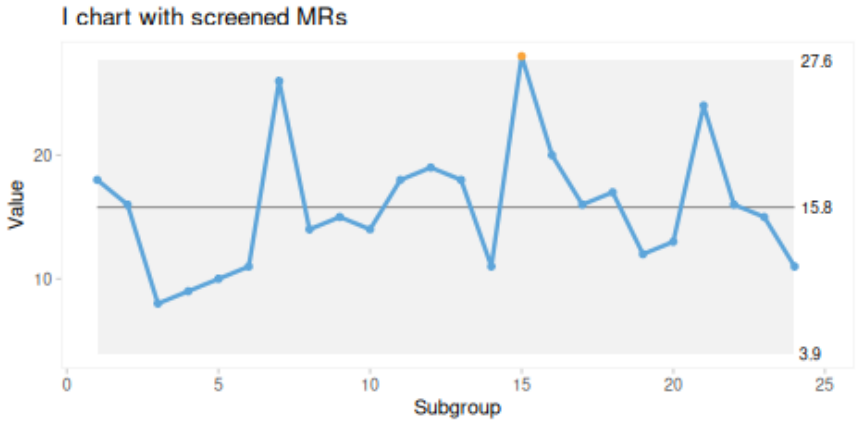

Figure 11.2: I chart with control limits calculated after removing freak moving ranges

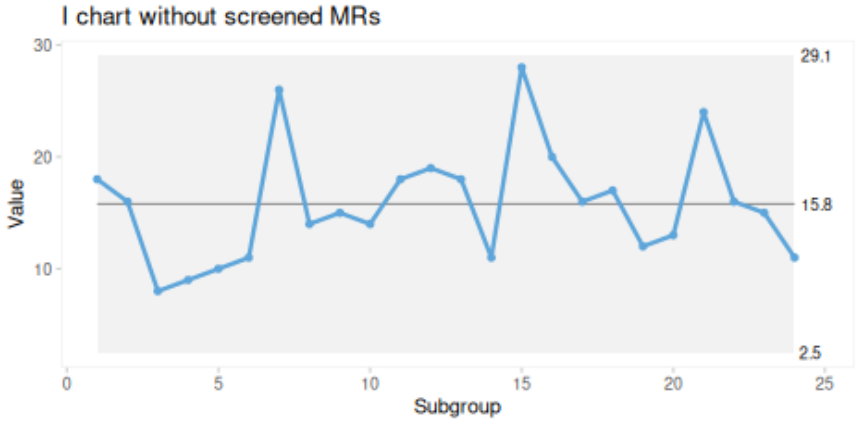

Figure 11.3: I chart with control limits calculated without removing extreme ranges

One might argue that screening moving ranges is unnecessary if the moving range chart is presented alongside the I chart. However, in our experience, moving range charts rarely add value for non-technical users and may even introduce confusion. For this reason, we prefer to increase the sensitivity of I charts by excluding extreme moving ranges before calculating control limits, while reserving moving range charts for more technical analysis.

Chapter 12

SPC Charts for Rare Events

When dealing with potentially serious or fatal events, the number of occurrences is often, fortunately, very low. This creates challenges for traditional SPC charts designed for count data.

The difficulty arises because charts such as the P, C, and U charts assume a sufficiently frequent and reasonably stable occurrence of events. When counts are low, the data become sparse and highly variable, which makes these charts less useful.

Sparse data can produce lower control limits that are censored at zero, thereby invalidating the 3-sigma test in that direction. Run charts may also become problematic when more than half of the data points are zero, because the median then becomes zero as well. This undermines the runs analysis by producing one long run above the median.

One way of dealing with rare events is to plot the number of opportunities or the time between events rather than event proportions or rates. In other words, we turn the indicator around and focus on the gaps between occurrences rather than on the occurrences themselves.

Another approach is to examine the cumulative sums (CUSUM) of binary data.

In this chapter, we introduce the G chart for the number of opportunities between events, the T chart for time between events, and the Bernoulli CUSUM chart for binary data.

12.1 Introducing the birth dataset

The Robson group 1 births dataset is a data frame containing 2193 observations from Robson group 1 deliveries, that is, first pregnancy, single baby, head first, and gestational age of at least 37 weeks.

```
# import data
births <- read.csv('data/robson1_births.csv',
                   comment.char = '#',
                   colClasses   = c('POSIXct',
                                    'Date',
                                    'logical',
                                    'logical',
                                    'factor',
                                    'integer',
                                    'integer',
                                    'integer',
                                    'double',
                                    'logical'))

# make sure the rows are sorted in time order
births <- births[order(births$datetime), ]
# add dummy variable, case, for counting the denominator for the proportion charts
births$case <- 1
# show data structure
str(births)
```

```
## 'data.frame':    2193 obs. of  11 variables:
##  $ datetime: POSIXct, format: "2016-01-04 06:13:00" "2016-01-04 07:20:00" ...
##  $ biweek  : Date, format: "2016-01-04" "2016-01-04" ...
##  $ csect   : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
##  $ cup     : logi  FALSE FALSE FALSE FALSE TRUE TRUE ...
##  $ sex     : Factor w/ 2 levels "female","male": 2 1 2 2 2 2 1 2 2 2 ...
##  $ length  : int  55 52 50 47 55 50 50 47 50 53 ...
##  $ weight  : int  3872 3750 3458 3145 4224 2730 3130 2755 3270 3645 ...
##  $ apgar   : int  10 10 10 10 10 10 10 10 10 10 ...
##  $ ph      : num  7.1 7.26 7.36 7.31 7.16 7.14 7.23 7.21 7.17 7.2 ...
##  $ asphyxia: logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
##  $ case    : num  1 1 1 1 1 1 1 1 1 1 ...
```

In this chapter, we are interested in the asphyxia variable, which is a logical vector that is TRUE when the baby had either a low Apgar score or a low umbilical cord pH, suggesting lack of oxygen during delivery. Asphyxia is a very serious condition that may result in permanent brain damage or death. In total, there are 16 cases corresponding to 0.7%.

Figure 12.1 is a run chart of the biweekly counts.

```
# run chart of counts
qic(biweek, asphyxia,
    data    = births,
    agg.fun = 'sum')  # use sum() function to aggregate data by subgroup
```

```
## Subgroup size > 1. Data have been aggregated using sum().
```

Because more than half of the data points are zero, the centre line (the median) is also zero. This invalidates the runs analysis, because the chart

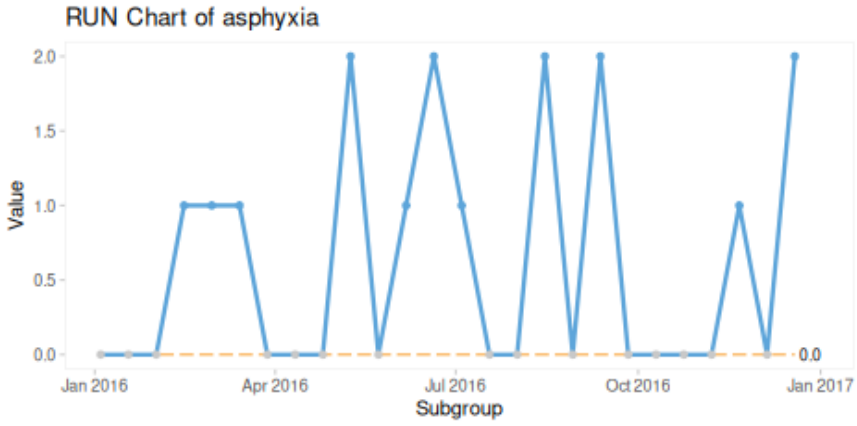

Figure 12.1: Run chart of number of deliveries with neonatal asphyxia

then contains only one very long run. As a result, the chart may falsely suggest one or more shifts in the data.

Figure 12.3 plots proportions rather than counts and leads to the same conclusion.

```
# run chart of proportions
qic(biweek, asphyxia, case,  # use the case variable for denominators
    data = births)
```

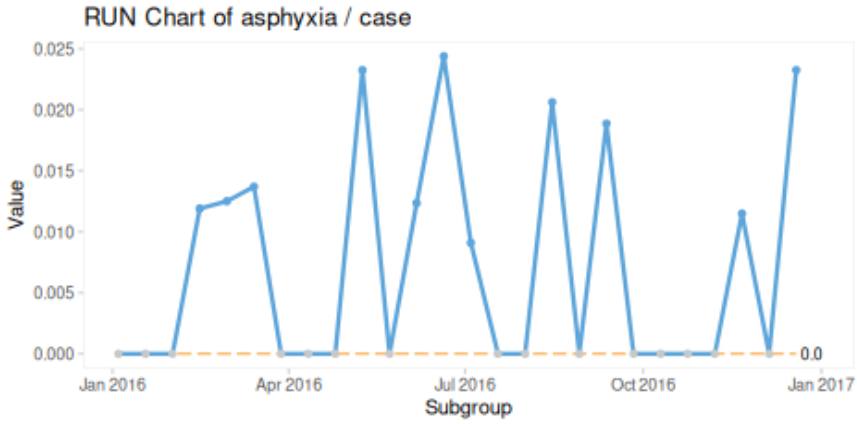

Figure 12.2: Run chart of proportion deliveries with neonatal asphyxia

Plotting the data on a P chart provides control limits and allows for a more meaningful runs analysis, because in this case the average is a better representation of the process centre (Figure 12.3).

```
# P chart
qic(biweek, asphyxia, case,
    data  = births,
    chart = 'p')
```

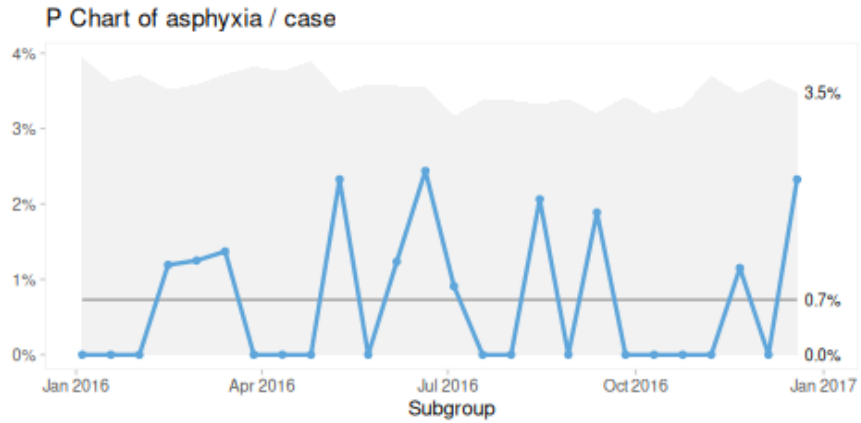

Figure 12.3: P chart of percent deliveries with neonatal asphyxia

While this P chart is still useful, the lower control limit is zero, which makes it impossible to detect a signal of improvement using the 3-sigma rule. We are therefore left to rely on runs analysis alone when looking for improvement.

One obvious solution is to increase subgroup size by aggregating data over longer periods, such as months or quarters. However, this comes at the cost of slower detection of both improvement and deterioration.

As an alternative, we now introduce the G chart.

12.2 G chart for opportunities between cases

The G chart plots the number of opportunities between cases, for example the number of deliveries between cases of neonatal asphyxia. This kind of data often follows a geometric distribution with standard deviation

$$\sigma = \sqrt{\bar{x}(\bar{x}+1)}$$

and control limits

$$\bar{x} \pm 3\sqrt{\bar{x}(\bar{x}+1)}$$

where $\bar{x}$ is the average number of opportunities between cases.

To obtain the number-between variable, we calculate the differences between the indices of the cases. As always, the data must first be sorted in time order.

```
# get indices of asphyxia cases
asph_cases <- which(births$asphyxia)

# calculate the number of deliveries between cases
asph_g <- c(NA, diff(asph_cases) - 1)

# plot G chart
qic(asph_g, chart = 'g')
```

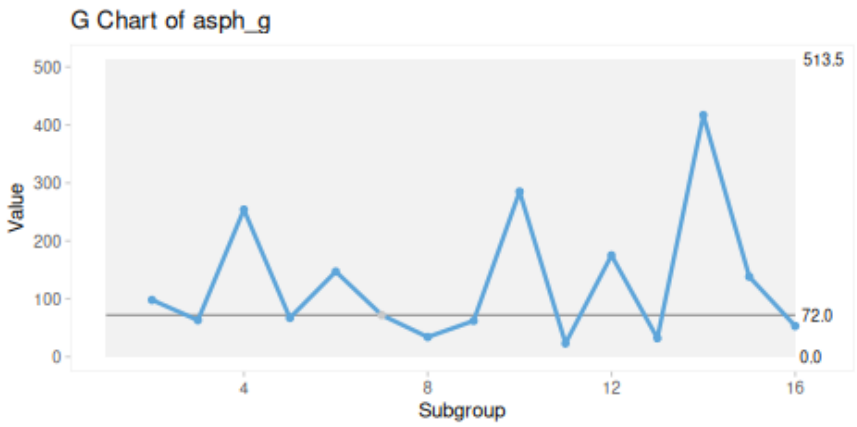

Figure 12.4: G chart of deliveries between neonatal asphyxia

Because the geometric distribution is highly skewed, the average is not ideal for runs analysis, which assumes that the data are distributed symmetrically around the centre. For this reason, qicharts2 uses the median by default in the runs analysis for G charts (Figure 12.4).

As with other count charts, negative lower control limits are rounded up to zero, since values below zero are not possible.

Notice that process improvement, that is, fewer cases, appears as the curve moving upwards. In this example, a 3-sigma signal would require at least 518 consecutive deliveries without asphyxia.

G charts are therefore useful alternatives to P charts when events are rare and the main interest is in detecting improvement. The two charts are complementary rather than competing. The P chart is often more familiar and more useful for signalling deterioration, whereas the G chart is more helpful when looking for improvement.

12.3 T chart for time between events

The T chart plots the time between events, which is a continuous variable. Although such data could in principle be plotted on an I chart, this may not be ideal. If events occur according to a Poisson process, as is often assumed, then the time between events follows an exponential distribution, which is highly skewed. One way to deal with this is to transform the data before plotting them on an I chart. A suitable transformation is

$$x = y^{(1/3.6)}$$

where x is the transformed variable and y is the original time-between-events variable.

Control limits can then be calculated for the transformed data in the usual way. Because transformed values are difficult to interpret, it is often preferable to plot the original values together with back-transformed centre and control lines. The back-transformation is

$$y = x^{3.6}$$

.

```
# get time between events
asph_t <- diff(c(births$datetime[asph_cases]))
# plot T chart
qic(asph_t, chart = 't')
```

T charts do not allow zero time between events. This may become a problem when time is recorded with insufficient precision. For example, if two events occur on the same day and time of day is not recorded, the variable is not truly continuous. In such situations, a G chart plotting days between events may be more appropriate.

In this example, the control limits on the T chart in Figure 12.5 appear rather wide. The upper control limit indicates that about 190 days, or 27 weeks, must pass without any asphyxia before a signal is triggered. This suggests that these asphyxia cases, being discrete cases rather than random events, may not be especially well suited to analysis with a T chart.

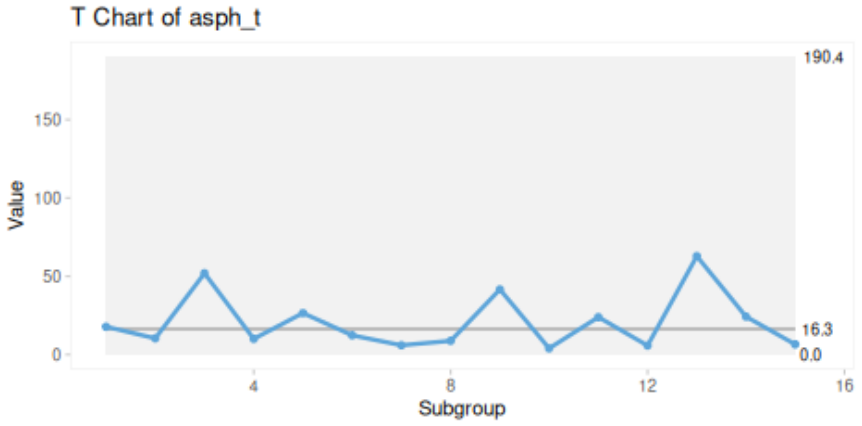

Figure 12.5: T chart of time (days) between deliveries with neonatal asphyxia

12.4 The Bernoulli CUSUM chart for binary data

The Bernoulli CUSUM chart, also known as the B chart, is a specialised control chart for detecting small shifts in the proportion of binary outcomes over time (Neuburger et al. 2017). It operates on a logical vector (TRUE/FALSE), where each observation contributes to a trace statistic calculated as the cumulative sum of deviations from a predefined target proportion.

The trace value s_i at time i is updated recursively. When the current observation x_i is TRUE, the trace increases; when x_i is FALSE, the trace decreases. The size of each increment depends on both the target proportion and the size of the shift the chart is designed to detect. In this way, the B chart accumulates evidence over time and becomes sensitive to sustained deviations from the target.

If x_i = TRUE:

$$s_i = s_{i-1} + logOR - log(1 + p_0(OR - 1))$$

If x_i = FALSE:

$$s_i = s_{i-1} - log(1 - p_0(OR - 1))$$

Here, s_i is the CUSUM statistic at time i, p_0 is the target or baseline proportion, and OR is the odds ratio representing the smallest shift the chart is designed to detect. For example, setting OR = 2 configures the chart to detect a doubling of the target proportion.

If the trace remains close to zero, the process is operating near target.

```
bchart(births$asphyxia, target = 0.007, or = 2, limit = 3.5)
```

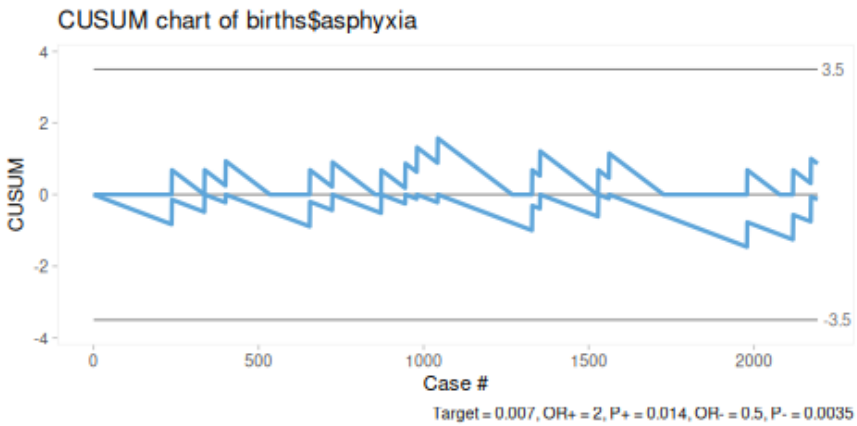

Figure 12.6: B chart of newborns with asphyxia

Figure 12.6 shows the asphyxia data plotted in a B chart using the bchart() function from qicharts2. The target corresponds to the known baseline proportion of asphyxia cases, and the default settings are used for odds ratio (= 2) and control limits (= ±3.5).

Several features are worth noting:

The chart displays two cumulative traces. The upper trace is configured to detect an increase from the target proportion, specifically a doubling in the odds, while the lower trace is configured to detect a decrease, corresponding to an odds ratio of 0.5.

The trace values themselves have no direct meaning beyond their direction of movement. They increase in response to TRUE values and decrease in response to FALSE values. Their magnitude reflects the cumulative departure from target, not the absolute severity or frequency of events.

The traces are reset to zero whenever they cross the horizontal axis or a control limit, so that the chart remains responsive to new patterns of deviation.

The control limits are user-defined rather than statistically derived. Their placement therefore reflects a compromise between sensitivity, that is, the ability to detect true shifts, and specificity, that is, resistance to false alarms. By default, the limits are set at ±3.5.

Figure 12.7 compares the data with a target proportion of 2%.

```
bchart(births$asphyxia, target = 0.02)
```

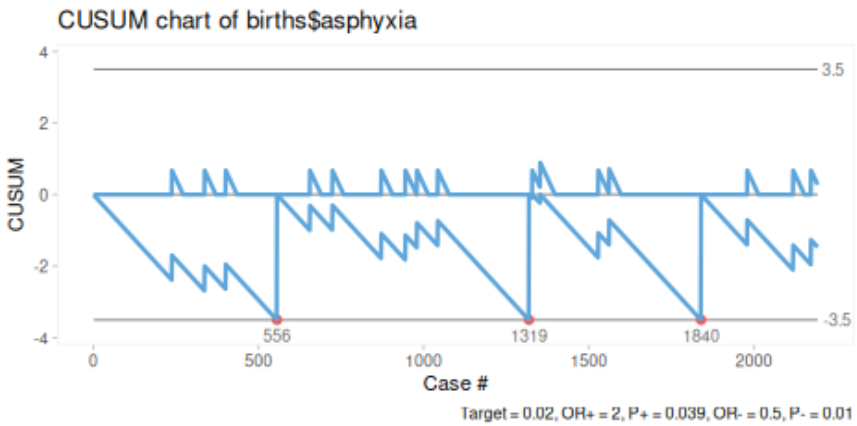

Figure 12.7: B chart of newborns with asphyxia, target = 0.2%

Here, the lower trace signals repeatedly, suggesting that the process is not centred around a target of 2%. More specifically, the chart suggests that the current process is performing better than that target.

Conversely, when the target is set to 0.1%, the chart suggests that the process is performing worse than the target (Figure 12.8).

```
bchart(births$asphyxia, target = 0.001)
```

Control limits for B charts are user-defined and reflect a trade-off between sensitivity and specificity. A common default is ±3.5, which is suitable for detecting a doubling or halving of the event rate from a 1% baseline for roughly 2,000-10,000 observations. Tighter limits and more observations increase sensitivity but tend to raise the false-alarm rate, whereas wider limits and fewer observations reduce false alarms at the cost of missing smaller shifts. The most appropriate choice depends on the context, including the amount of available data, the size of shift of interest, and the consequences of missing a signal or raising a false one.

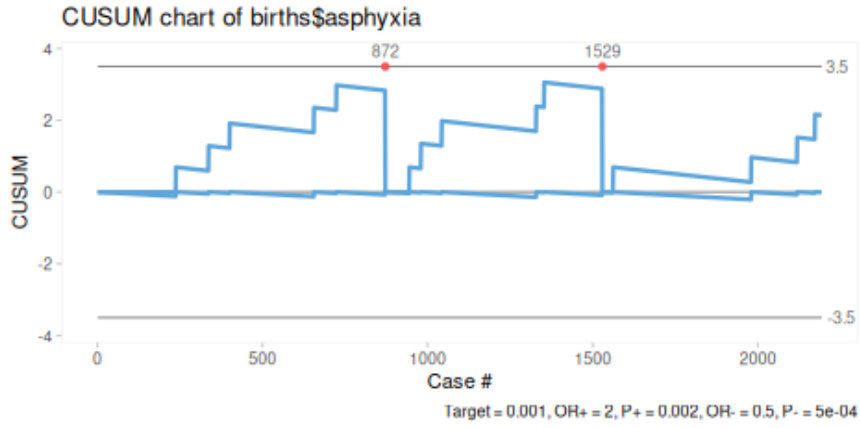

Figure 12.8: B chart of newborns with asphyxia, target = 0.01%

In practice, limits are often chosen on the basis of simulation, historical data, and domain knowledge, especially in relation to the consequences of missed detections and false alerts. See Neuburger et al. (2017) for details on the selection of control limits.

Looking at Figure 12.6, which shows a historically stable process based on more than 2000 observations, with most data points clustered around the centre line, one might judge it reasonable to tighten the control limits to ± 3 or even ± 2.5, as shown in Figure 12.9.

```
bchart(births$asphyxia, target = 0.007, limit = 2.5)
```

12.5 Selecting the right chart for rare events

Having reviewed several alternatives for rare events data – U and T charts for low event rates, and P, G, and B charts for low case proportions – we may ask which chart should be preferred. The answer is that it depends. No single chart is universally best, and in many situations two or more charts provide complementary insights.

In particular, we recommend pairing a T or G chart with its corresponding U or P chart. Together, these pairs provide a fuller picture by helping to detect both deterioration and improvement.

The B chart is a particularly powerful tool, but it requires careful attention and specialist knowledge to construct and interpret. Its diagnostic performance depends strongly on the chosen target, odds ratio, and control limits,

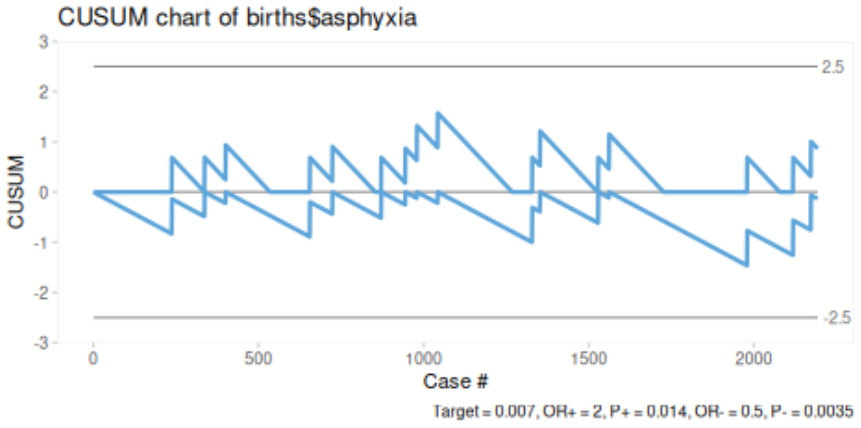

Figure 12.9: B chart of newborns with asphyxia, target = 0.01%

and these choices should be made by people with a good understanding of the underlying process. That said, the default settings – an odds ratio of 2 and control limits of ± 3.5 – are generally reasonable for processes with a baseline rate around 1%, where the aim is to detect a doubling or halving of the event rate relative to that baseline.

Chapter 13

Prime Charts for Very Large Subgroups

With count data involving very large subgroup sizes, SPC charts often produce very tight control limits with many data points falling outside the limits. Figure 13.1 demonstrates this phenomenon, known as overdispersion, using data from from Mohammed et al. (2013) included in *qicharts2* (nhs_accidents). The data are tabulated at the end of this chapter.

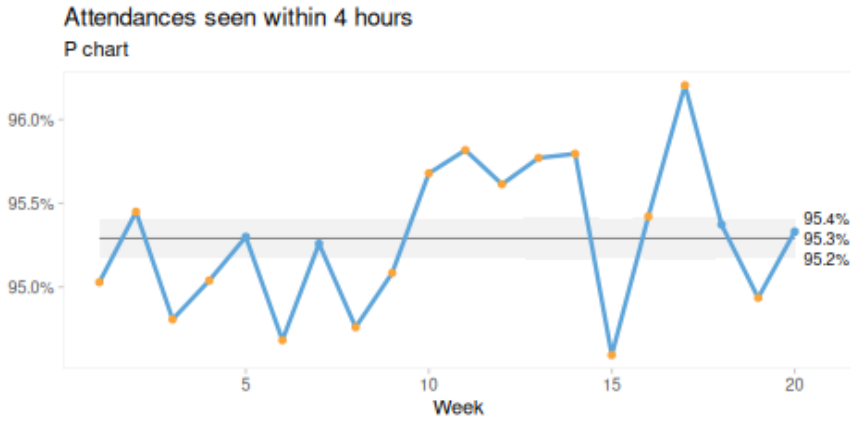

Figure 13.1: P chart of percent emergency attendances seen within 4 hours

At first sight, this may suggest that the process is highly unstable, even when such a conclusion may not be justified. Overdispersion occurs when the natural, common cause variation in the data exceeds what is expected

under the assumed theoretical distribution – binomial for proportions and Poisson for rates.

One way to diagnose this problem is to plot the same data on an I chart, which bases its control limits on the observed variation between successive subgroups rather than on theoretical distributional assumptions. Figure 13.2 is an I chart of the same data as Figure 13.1 and suggests only common cause variation.

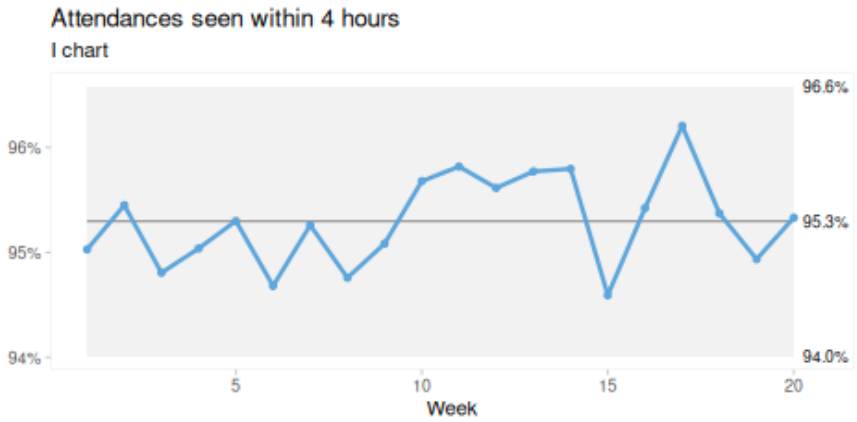

Figure 13.2: I chart of percent emergency attendances seen within 4 hours

As a general rule of thumb, when the control limits from an I chart differ markedly from those of the corresponding P or U chart, this suggests that the observed variation is not consistent with the binomial or Poisson assumptions underlying the P and U charts.

Several solutions to the problem of overdispersion and overly tight control limits have been proposed. One pragmatic solution is simply to use I charts for count data as well as for individual measurements. However, I charts have straight control limits and do not take varying subgroup sizes into account. If subgroup sizes vary only a little, this may not matter. But sometimes – particularly in healthcare – variation in subgroup size does matter.

Laney proposed a solution in which the assumed within-subgroup variation in count data is adjusted by a factor based on the moving ranges of successive standardised data points, σ_z (Laney 2002).

13.1 Laney's prime chart

As shown earlier in Table 6.1, the control limits for a traditional P chart are calculated as:

$$\bar{p} \pm 3\sigma_{pi}$$

where $\bar{p}$ is the average proportion and σ_{pi} is the within-subgroup standard deviation for the i-th subgroup:

$$\sigma_{pi} = \sqrt{\bar{p}(1-\bar{p})/n_i}$$

where n_i is the sample size of the i-th subgroup.

The P' chart, pronounced P prime chart, takes both within and between subgroup variation into account. its control limits are given by:

$$\bar{p} \pm 3\sigma_{pi}\sigma_z$$

where σ_z represents the between-subgroup variation. This is calculated from the moving ranges of the standardised proportions, defined as:

$$z_i = (p_i - \bar{p})/\sigma_{pi}$$

The moving ranges are then:

$$MR_i = |z_i - z_{i-1}|$$

The estimat of between-subgroup variation, σ_z, is the average moving range devided by the constant 1.128:

$$\sigma_z = \overline{MR_z}/1.128$$

Thus, the control limits for the P' chart become:

$$\bar{p} \pm 3\sigma_{pi}\sigma_z$$

The control limits in Figure 13.3 are very close to those of the I chart, though they vary slightly because subgroup sizes vary.

Similarly, control limits for the U' chart are:

$$\bar{u} \pm 3\sigma_{ui}\sigma_z$$

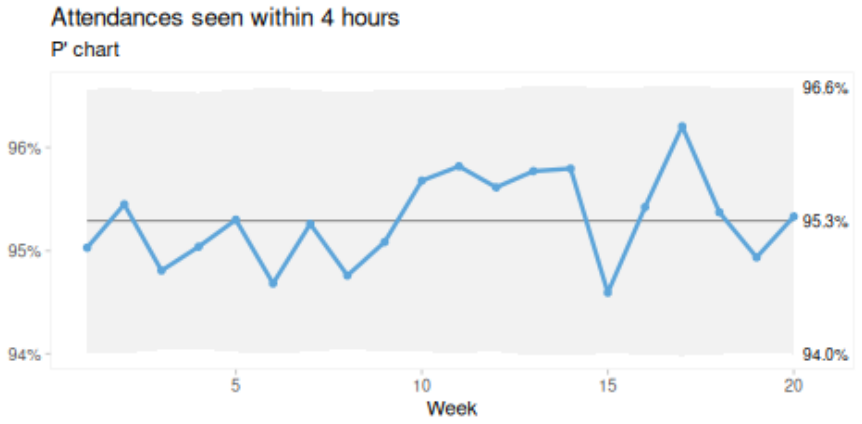

Figure 13.3: P' chart of percent emergency attendances seen within 4 hours

where

$$\sigma_{ui} = \sqrt{\bar{u}/n_i}$$

and

$$\sigma_z = \overline{MR_z}/1.128$$

and the moving ranges are computed from the standardised rates defined as

$$z_i = (u_i - \bar{u})/\sigma_{ui}$$

In *qicharts2*, the moving ranges of the standardised values are, by default, screened for extreme values, following the same approach used for I charts, as described in Chapter 11.

13.2 When to use prime charts

Laney's prime charts were developed to handle overdispersion, and also underdispersion, that is, situations in which the data show more or less variation than expected under a binomial or Poisson distribution. This is often seen when subgroup sizes are very large. But how do we know when to suspect overdispersion?

A warning sign is when a traditional P or U chart shows unusually tight control limits that seem unnatural. If plotting the same data on an I chart produces substantially wider limits, this is a strong indication that the assumptions behind the P or U chart may not hold. In such cases, Laney's P' or U' chart will often be the better choice.

When the adjustment factor (σ_z) is close to one, indicating little or no overdispersion, the P' or U' chart closely resembles the traditional P or U chart. For this reason, Laney recommends prime charts as a generally safe and robust default.

In the next chapter, we introduce a modified I chart – the I', or I prime, chart – which produces control limits for count data that closely match those of Laney's prime charts while also accommodating measurement data with varying subgroup sizes.

Example data for P prime charts

Table 13.1: The number of attendances to major accident and emergency hospital departments in the NHS that were seen within 4 hours of arrival over twenty weeks. Source: [@mohammed2013]

Week (i)	Attendances seen within 4 hours (r)	Number of attendances (n)
1	266,501	280,443
2	264,225	276,823
3	276,532	291,681
4	281,461	296,155
5	269,071	282,343
6	261,215	275,888
7	270,409	283,867
8	279,778	295,251
9	270,483	284,468
10	270,320	282,529
11	267,923	279,618
12	271,478	283,932
13	255,353	266,629
14	256,820	268,091
15	261,835	276,803
16	259,144	271,578
17	255,910	266,005
18	260,863	273,520
19	264,465	278,574
20	260,989	273,772

Chapter 14

I Prime Charts for Variable Subgroup Sizes

The I chart is often regarded as the *Swiss Army knife* of SPC. It is useful in many situations and can often serve as a valid, or even superior, alternative to other Shewhart charts. One reason for this is that the I chart is based on the empirical variation in the data rather than on theoretical distributional assumptions, which may or may not hold.

However, the I chart does not account for variable subgroup sizes. In other words, it does not adjust when the area of opportunity varies between samples, for example when the number of patients fluctuates from month to month. This results in straight control limits, which in some situations may generate false signals or fail to detect true ones. When subgroup sizes vary only slightly, this may not matter. But often – especially in healthcare – size matters.

Several solutions to this problem have been proposed. Here we present the normalised I chart, or, as we prefer to call it, the I prime (I') chart, suggested by Taylor (2018).

In summary, the I' chart accounts for variable subgroup sizes by adjusting the within-subgroup standard deviation using a factor that depends on the size of each subgroup. This produces wavy control limits when subgroup sizes vary. The I' chart is useful for both measurement and count data, with or without denominators. When subgroup sizes are constant, the I' chart is identical to the original I chart. For rates and proportions, the I' chart produces control limits that match those of the U' and P' charts.

Figure 14.1 shows an I' and a P' chart side-by-side illustrating how similar their results are.

```
qic(month, deaths, admissions,
    data      = admis,
    chart     = 'ip',
    y.percent = T,
    title     = "I' chart of hospital mortality",
    xlab      = 'Month')

qic(month, deaths, admissions,
    data  = admis,
    chart = 'pp',
    title = "P' chart of hospital mortality",
    xlab  = 'Month')
```

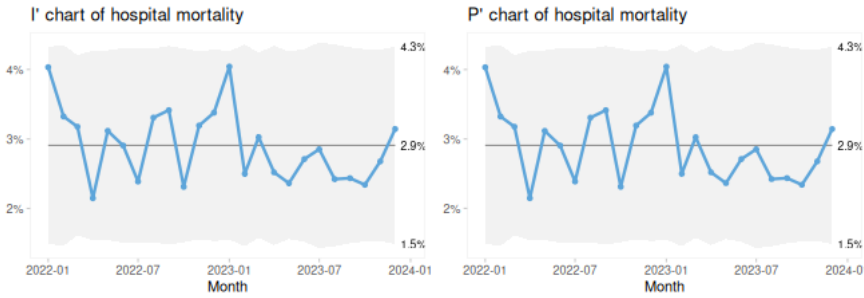

Figure 14.1: Comparing I' and P' charts

14.0.1 Procedure for calculating centre line and control limits

We use the following symbols:

- n = numerators
- d = denominators
- o = number of data values (subgroups)
- i = i$^{\text{th}}$ data value

The values to be plotted are

$$y = \frac{n}{d}$$

The centre line is:

$$CL = \frac{\sum n}{\sum d}$$

The standard deviation of i$^{\text{th}}$ data point is

$$s_i = \sqrt{\frac{\pi}{2}} \frac{|y_i - y_{i-1}|}{\sqrt{\frac{1}{d_i} + \frac{1}{d_{i-1}}}}$$

The average standard deviation is

$$\bar{s} = \frac{\sum s}{o - 1}$$

The control limits are then

$$\text{control limits} = CL \pm 3\frac{\bar{s}}{\sqrt{d_i}}$$

When subgroup sizes are all equal to 1, these limits reduce to

$CL \pm 2.66\overline{MR}$

which is the same as for original I chart.

As with the original I chart, *qicharts2* screens the moving ranges of s_i, removing ranges greater than the theoretical upper natural limit (= 3.2665) before calculating $\bar{s}$.

14.1 I' charts for variable subgroup sizes

Figure 14.1 demonstrates the use of an I' chart with count data. However, the I' chart was originally intended for measurement data with variable subgroup sizes. One example is when, for reasons of privacy, only aggregated patient data are available rather than individual measurements. Traditionally, such data would be plotted on an ordinary I chart. But if the number of patients in each subgroup varies substantially, straight control limits may be inappropriate.

Figure 14.2 shows an I chart of the monthly HbA1c averages from the Diabetes HbA1c dataset. In April 2020, one data point lies above the upper control limit, suggesting a special cause. However, the ordinary I chart does not take subgroup size into account.

```
qic(month, avg_hba1c,
    data  = diabetes,
    chart = 'i',
    title = NULL,
    ylab  = 'mmol / mol',
    xlab  = 'Month')
```

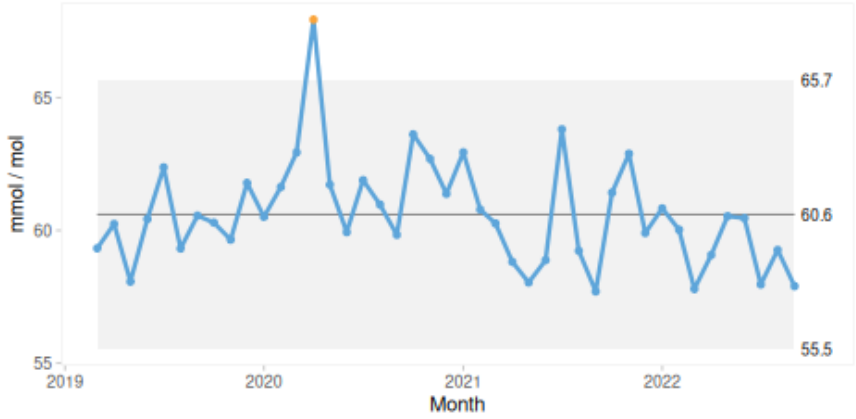

Figure 14.2: I chart of average HbA1c without denominator

Figure 14.3 shows the corresponding I' chart, which incorporates subgroup size, that is, the number of patients, and adjusts the control limits accordingly.

```
qic(month, avg_hba1c * n, n,
    data  = diabetes,
    chart = 'ip',
    title = NULL,
    ylab  = 'mmol / mol',
    xlab  = 'Month')
```

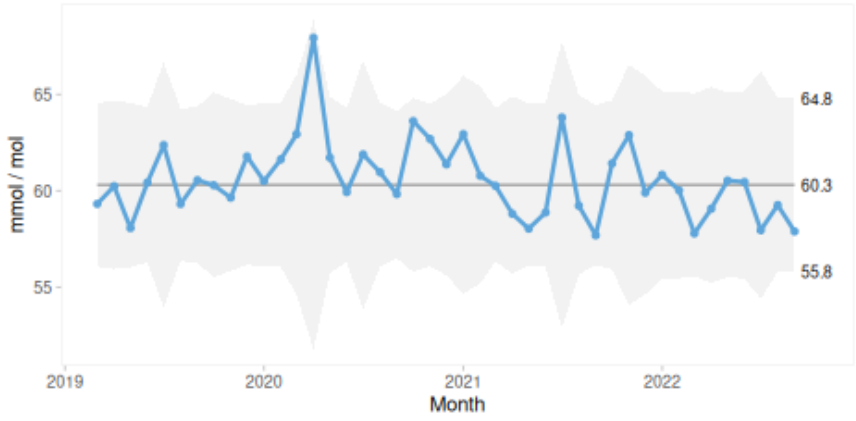

Figure 14.3: I prime chart of average HbA1c with denominator

April 2020 was the first month of lockdown during COVID-19 in Denmark, and the number of patients seen that month was markedly lower than usual. A smaller denominator allows for greater common cause variation in the measurements and therefore leads to wider control limits. When subgroup size is taken into account, the apparent special cause in Figure 14.2 turns out to fall within the limits of expected variation.

To plot averages correctly, we must multiply the subgroup averages (avg_hba1c) by the subgroup sizes (n) in order to recover the sums of individual measurements. Otherwise, the chart would be based on averages of averages.

Notice also that the centre lines differ slightly (60.6 versus 60.3), because the I' chart uses a weighted mean rather than an unweighted mean.

14.2 One chart to rule them all?

The I' chart reproduces the original I chart for individual measurements and matches the P' and U' charts for proportion and count data.

If we accept Laney's argument that prime charts are valid, and perhaps even preferable, substitutes for their non-prime counterparts, it is tempting to view the I' chart as a universal solution to most, if not all, charting needs.

The I' chart is a relatively recent development, but its versatility and its ability to mimic the behaviour of more traditional SPC charts make it a compelling alternative. It has the potential to simplify chart selection by providing a single, robust method that can be used across a wide range of data types and situations. As empirical support for the method grows, the I' chart may well become a preferred tool in modern SPC practice.

Chapter 15

Funnel Plots for Categorical Subgroups

Funnel plots are SPC charts used for categorical subgroups (Spiegelhalter 2005). The subgroups are often ordered by size, producing the characteristic funnel-shaped control limits. Funnel plots are useful for comparing indicator levels across categories, for example complication rates between hospitals.

Figure 15.1 compares the level of an indicator across six organisational units. One unit (E) falls below the lower control limit, suggesting that its results may reflect a fundamentally different process compared with the others.

Because funnel plots do not involve a natural time sequence, the data points are not connected, and run analysis is not applicable.

To demonstrate the construction of funnel plots, we use data on urinary tract infections from the hospital_infections dataset included in *qicharts2*. We begin by plotting the data over time using a traditional U chart:

```
# get urinary tract infections from hospital infections data
uti <- hospital_infections[hospital_infections$infection == 'UTI',]

# plot U chart
qic(month, n, days,
    data     = uti,
    chart    = 'u',
    multiply = 10000,
    facets   = ~hospital,
    ncol     = 2,
    title    = 'Hospital acquired urinary tract infections in six hospitals',
    ylab     = 'Infections per 10,000 risk days',
    xlab     = 'Month')
```

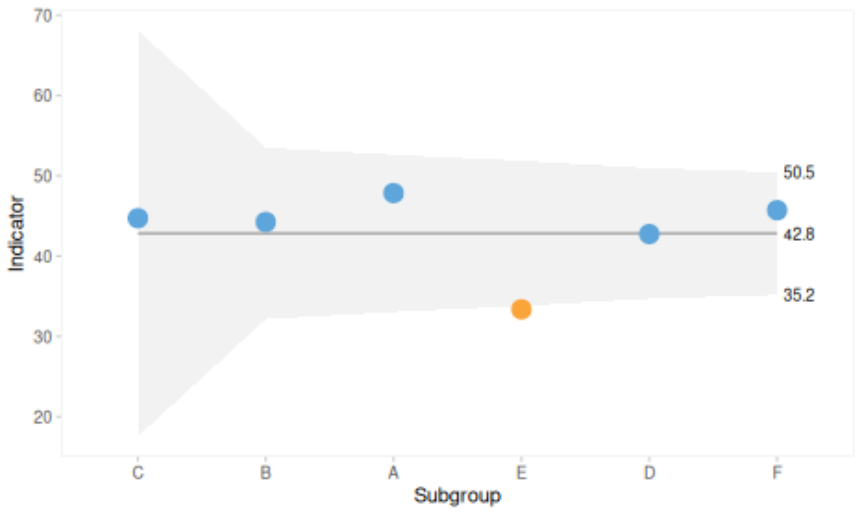

Figure 15.1: Funnel plot

Figure 15.2 shows signals of special cause variation in two hospitals (BOH and NOH). In such cases, direct comparison of infection rates between hospitals is problematic, as it does not make sense to compare levels based on data that are changing over time.

For this reason, we select data from a recent period before constructing the funnel plot. In Figure 15.3, the data are restricted to the most recent quarter and plotted using hospital, rather than time, as the subgrouping variable:

```
# filter data to latest quarter
uti_2016q4 <- uti[uti$month >= '2016-10-01', ]

# plot "funnel" plot
qic(hospital, n, days,
    data     = uti_2016q4,
    chart    = 'u',
    multiply = 10000,
    title    = 'Hospital acquired urinary tract infections in six hospitals',
    ylab     = 'Infections per 10,000 risk days',
    xlab     = 'Hospital')
```

By default, the subgroups are ordered alphabetically along the x-axis. To produce a funnel plot, the subgroups must be ordered by size, as shown in Figure 15.4.

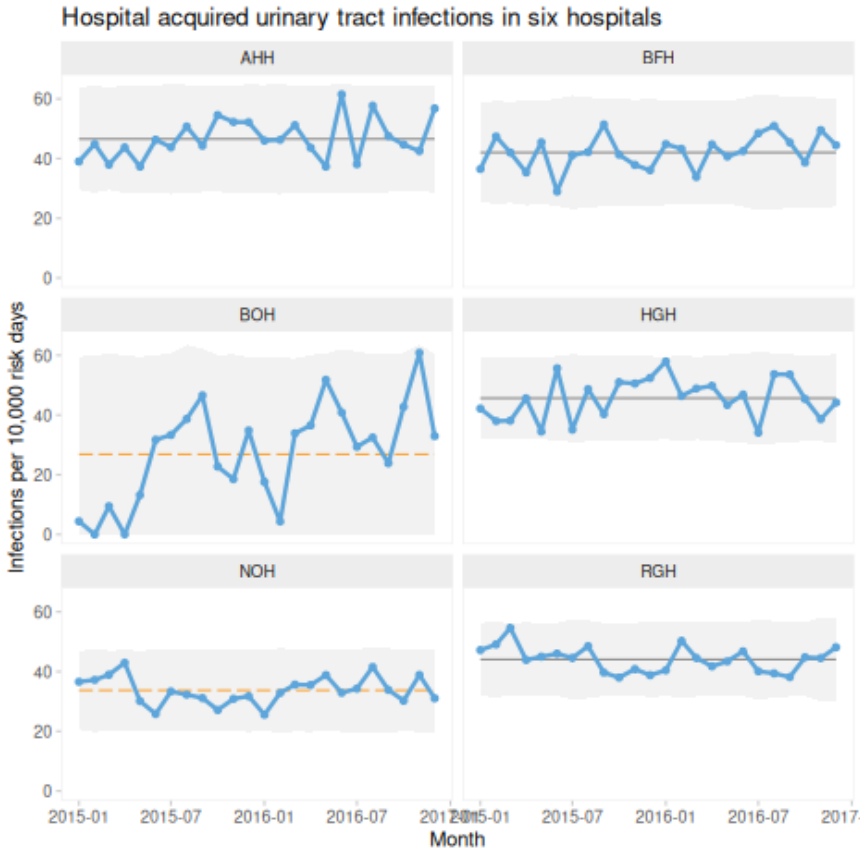

Figure 15.2: U chart of urinary tract infections

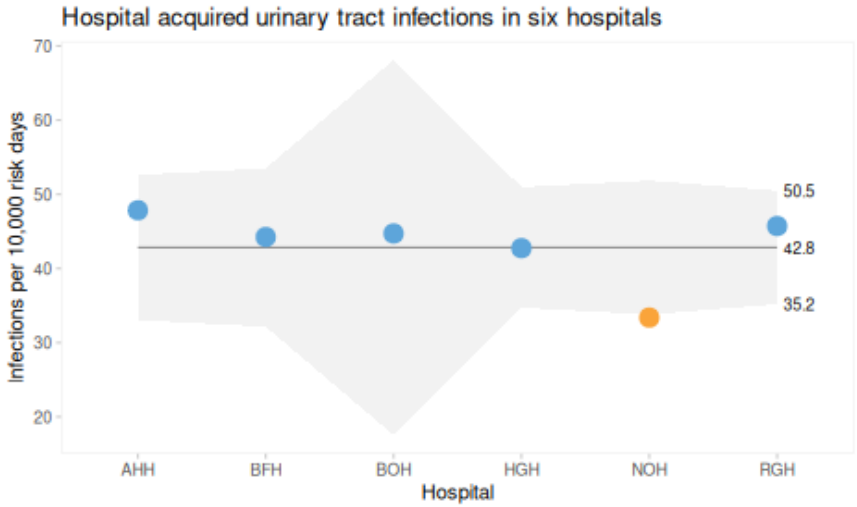

Figure 15.3: Unordered 'funnel' plot of urinary tract infections

```
# order hospitals by denominator (risk days) to produce funnel
qic(reorder(hospital, days), n, days,
    data     = uti_2016q4,
    chart    = 'u',
    multiply = 10000,
    title    = 'Hospital acquired urinary tract infections in six hospitals',
    ylab     = 'Infections per 10,000 risk days',
    xlab     = 'Hospital')
```

In summary, funnel plots are SPC charts for categorical subgroups and are useful for comparing indicator levels across organisational units. To support meaningful comparison, it is important to use data from a recent period before constructing the funnel plot.

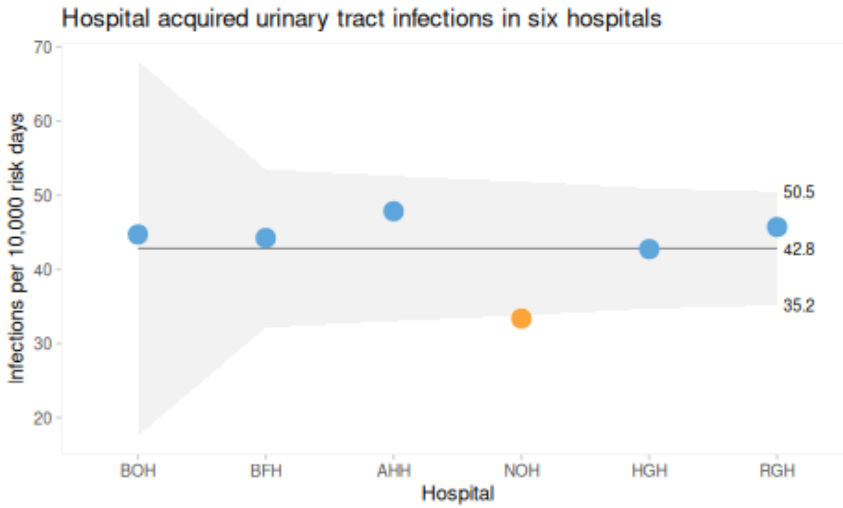

Figure 15.4: Ordered funnel plot of urinary tract infections

Chapter 16

Pareto Charts for Ranking Problems

The Pareto chart, named after Vilfred Pareto, was invented by Joseph M. Juran as a practical tool for identifying the most important causes of a problem. It is widely used in quality improvement to prioritise efforts by highlighting the categories that contribute most to an outcome.

In this example, we use the dataset on adverse events causing harm to patients, collected using the Global Trigger Tool method (Plessen et al. 2012).

```
# print structure of ae data
str(ae)
```

```
## 'data.frame':    131 obs. of  2 variables:
##  $ severity: chr  "E" "F" "E" "F" ...
##  $ category: chr  "Pressure ulcer" "Gastrointestinal" "Infection" "Infection" ...
```

The `paretochart()` function from the *qicharts2* package takes a categorical vector as input and produces a Pareto chart as shown in Figure 16.1.

```
paretochart(ae$category)
```

In a Pareto chart, the bars represent the counts in each category, while the curve shows the cumulative percentage across categories. In this example, almost 80% of harms arise from just three categories: gastrointestinal, infection, and procedure.

Figure 16.2 shows a Pareto chart of harm severity. This chart illustrates that nearly all events resulted in temporary harm (categories E-F).

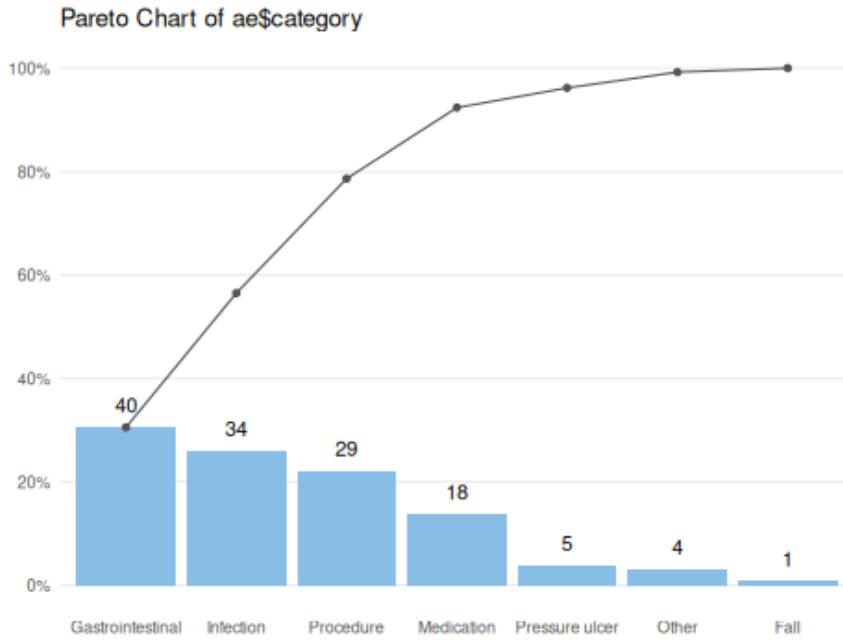

Figure 16.1: Pareto chart of patient harm.

```
paretochart(ae$severity)
```

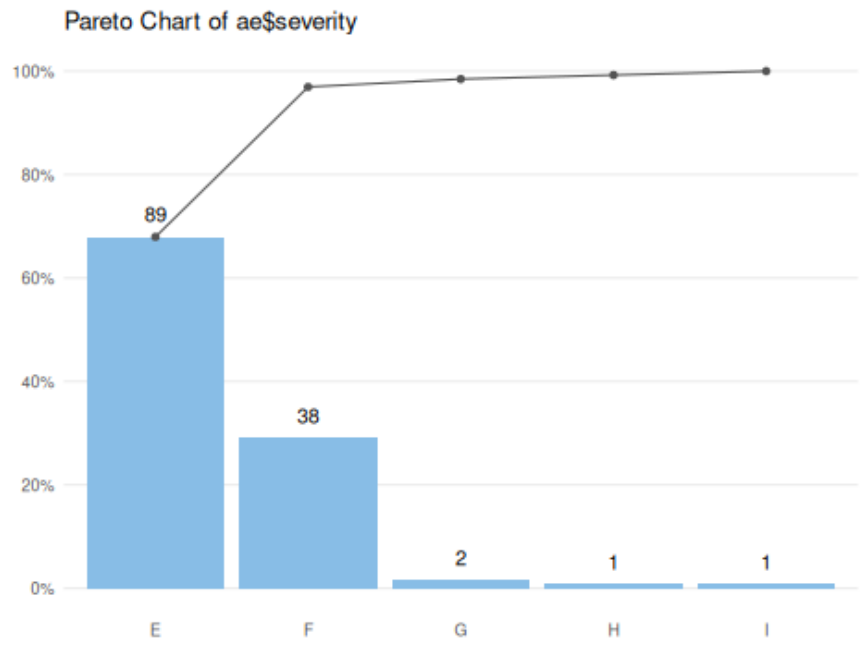

Figure 16.2: Pareto chart of harm severity: E-I, where E-F = temporary harm, G-H = permanent harm, and I = fatal harm.

The `paretochart()` function expects a character or factor vector as input. However, data are often already aggregated into tabular form:

```
ae.tbl
```

```
          category count
1             Fall     1
2 Gastrointestinal    40
3        Infection    34
4       Medication    18
5            Other     4
6   Pressure ulcer     5
7        Procedure    29
```

To construct a Pareto chart from tabulated data, we must first convert the counts back into a vector. This can be done using the `rep()` function, which repeats each category according to its frequency:

```
# make vector from counts
ae.cat <- rep(ae.tbl$category, ae.tbl$count)

# show first six elements of vector
head(ae.cat)
```

```
## [1] Fall             Gastrointestinal Gastrointestinal Gastrointestinal
## [5] Gastrointestinal Gastrointestinal
## 7 Levels: Fall Gastrointestinal Infection Medication Other ... Procedure
```

```
# plot Pareto chart
paretochart(ae.cat)
```

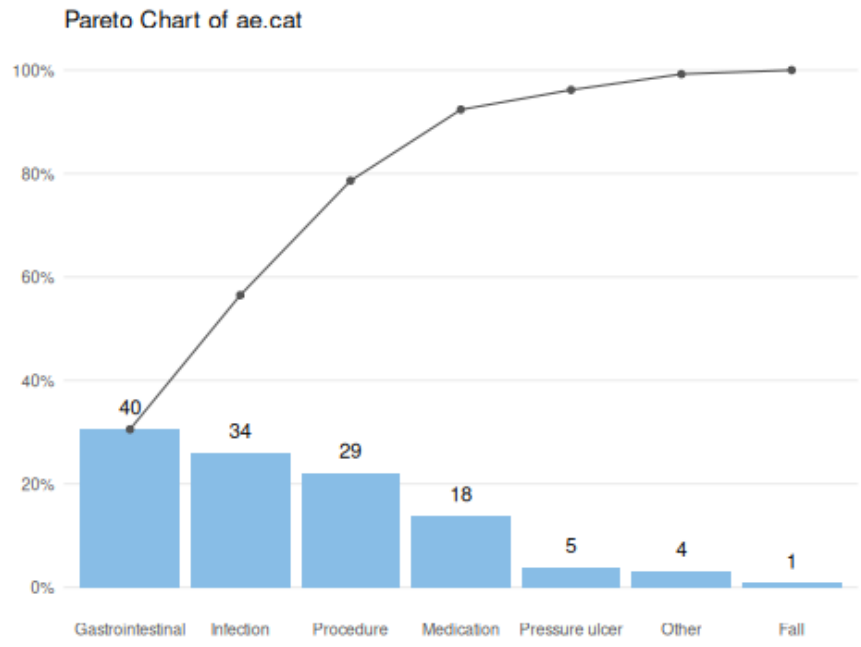

Figure 16.3: Pareto chart constructed from tabular data.

In summary, the Pareto chart is a simple but powerful tool for identifying the most common causes of a problem. In many situations, a large proportion of problems can be attributed to a relatively small number of causes. In the example above, reducing gastrointestinal harm (most often obstipation) and hospital-acquired infections would substantially reduce the overall rate of adverse events.

Part 4: Best Practices, Controversies, and Tips

Chapter 17

Tips for Effective SPC Implementation

Successful implementation of SPC involves far more than plotting control charts. It requires the establishment of a sustainable system that integrates data collection, analysis, interpretation, and action into everyday practice.

Building such a system requires, at a minimum, active engagement of stakeholders at all levels of the organisation, automation of chart production, and continuous evaluation and refinement of the system itself. It also requires early recognition of potential pitfalls and challenges, which are discussed separately in Chapter 18.

17.1 Engaging stakeholders

Although this book is primarily aimed at data scientists and analysts, successful SPC implementation depends on a much broader group of stakeholders. These include front-line staff and team leaders responsible for data collection and interpretation, as well as middle and senior management. In particular, management plays a crucial role: their understanding of SPC principles and their commitment to improvement – rather than blame – are essential for success. Without this support, SPC risks becoming a routine reporting exercise or a box-ticking activity, rather than a meaningful tool for learning and continuous improvement. SPC implementation relies on both knowledge and skills. For successful adoption, the organisation must develop capability in both areas.

- **Knowledge includes**

  - Understanding variation
  - Understanding SPC charts
  - Understanding the Pyramid Model for Investigation

- **Skills include**
  - Developing operational indicator definitions
  - Designing rational sampling plans
  - Constructing SPC charts
  - Creating SPC reports and dashboards
  - Identifying and investigating signals of special-cause variation

However, not all stakeholders require the same depth of knowledge or the same set of skills. Different roles demand different levels of competence.

At a fundamental level, stakeholders should understand key concepts and terminology. At an intermediate level, they should be able to apply and explain SPC principles. At a deeper level, they should be able to integrate, adapt, and extend these concepts in practice.

Front-line staff typically require a fundamental understanding and contribute to data collection and the application of operational definitions. Team leaders require an intermediate level of understanding, enabling them to interpret charts and guide investigations. Data scientists require deep expertise to design systems, construct charts, and analyse patterns. Management, while not requiring technical depth, must understand the principles sufficiently to guide decision-making and foster a culture of learning rather than blame.

Developing this capability across an organisation is challenging. In large healthcare settings, a stepwise approach is often effective: begin with small-scale implementation in selected units, build experience and confidence, and gradually expand as lessons are learned.

17.2 Automating production of SPC charts

Automation is essential for making SPC both effective and sustainable. While small-scale or temporary projects can be managed manually, long-term or system-wide implementation requires systems that handle data efficiently and present results consistently with minimal manual effort.

It is important to distinguish between tasks that are best handled by computers and those that require human judgement. Data management, analysis, and presentation can and should be automated. Interpretation and decision-making, however, remain human responsibilities. Automating routine tasks frees up time for thoughtful analysis and action.

17.2.1 Data collection and storage

Ideally, data collection and storage should be embedded within routine clinical or administrative workflows, adding no extra burden to front-line staff and allowing seamless transfer into analysis systems.

In practice, however, routine data are often insufficient for SPC. Important variables may be missing, inconsistently recorded, or stored as unstructured text, making meaningful analysis difficult. This frequently necessitates supplementary data collection procedures.

A pragmatic approach is to begin by identifying the critical data elements required for SPC. Organisations can then determine which elements are already captured and design additional processes only where necessary. When combined with automated data capture and chart generation, this approach minimises burden while maximising data quality.

In the longer term, it is beneficial to design information systems with SPC and quality improvement in mind. Ideally, such systems should be flexible enough to accommodate future needs that may not yet be known.

17.2.2 Charts, reports, and dashboards

Individual SPC charts provide insight into specific processes or outcomes. However, understanding a system as a whole often requires examining multiple charts together. Reports and dashboards are useful for presenting such information.

Reports are typically static and structured, for example monthly or quarterly summaries that combine SPC charts with explanatory text and interpretation. They are designed for formal review and communication and are usually shared as documents, either on paper or as PDFs.

Dashboards, by contrast, are dynamic and interactive. They present multiple charts or indicators in real time or near real time and allow users to explore the data through filtering and drill-down functionality. Dashboards prioritise timeliness and accessibility, while interpretation is left to the user.

In practice, both formats are valuable. Reports provide context and explanation, while dashboards support ongoing monitoring and rapid response. An effective SPC system often combines both.

Although the design of reports and dashboards is beyond the scope of this book, R provides powerful tools for both. Using R Markdown (Yihui Xie 2023) or Quarto (Allaire et al. 2025), possibly combined with Shiny(Chang et al. 2026), allows organisations to integrate data import, analysis, and presentation into a single, largely automated workflow.

17.3 Continuous evaluation and improvement

Once SPC capability has been established, it is important not to become complacent. SPC itself is a system that requires ongoing maintenance, evaluation, and adaptation.

Indicators should be reviewed regularly to ensure that they remain relevant and useful. This may involve refining definitions, introducing new measures, and removing those that no longer add value.

The burden of data collection should also be continuously minimised. In early stages, manual collection methods are often sufficient and appropriate. As initiatives mature, however, processes should be streamlined and automated wherever possible. Ideally, data collection becomes part of routine workflows and requires minimal additional effort.

Reports and dashboards should likewise be kept simple and focused. Over time, there is a natural tendency to add more metrics, filters, and visualisations. While well intentioned, this can lead to cluttered displays that obscure key messages and overwhelm users.

Effective reporting systems highlight only what matters. They make trends, signals, and actionable insights immediately visible. Each element should serve a clear purpose and be tailored to a specific audience. In this context, simplicity is a strength.

> Perfection is achieved, not when there is nothing more to add,
> but when there is nothing left to take away.
>
> – Antoine de Saint-Exupéry

In summary, implementing SPC is not a one-off task but an ongoing commitment. By continually refining and simplifying the system, organisations ensure that SPC remains focused on its central purpose: learning from variation and supporting meaningful improvement.

Chapter 18

Common Pitfalls to Avoid

While SPC is a powerful framework for data-driven quality improvement, its effectiveness can be undermined by a number of common missteps. Misinterpretation of data, inappropriate use of run and control charts, and faulty assumptions can lead to wasted effort or missed opportunities – and may ultimately erode trust in SPC itself.

This chapter outlines some of the most common pitfalls and how to avoid them.

18.1 Data issues

SPC charts are only as good as the data on which they are based. Poor data quality – such as inaccurate measurements, inconsistent definitions, or data collected under varying conditions across time or location – can significantly compromise their reliability.

Particular attention must be paid to how subgroups are formed. Subgroups consist of the data elements that make up each plotted data point, and the way data are sampled and grouped has a major influence on the chart's ability to distinguish between common cause and special cause variation. Poor subgrouping may generate false signals or obscure meaningful ones. For this reason, we devote an entire chapter (19) to rational subgrouping.

Time spent developing clear operational definitions and rational subgrouping strategies is rarely wasted. Investing effort at this stage ensures that data are meaningful, comparable, and suitable for analysis.

18.2 Signal fatigue from over-sensitive SPC rules

Originally, control charts relied on a single rule: the 3-sigma rule. Over time, numerous supplementary rules have been developed to increase sensitivity to smaller shifts and other patterns of non-random variation. These include the Western Electric rules, Nelson rules, Westgard rules, and various sets of runs rules.

While it may be tempting to apply many rules in order to detect every possible signal, this approach comes at a cost. Additional rules increase sensitivity, but they also increase the risk of false alarms.

Some commonly used rule sets are particularly prone to false signals. For example, a widely used set of run chart rules – including tests for trends, shifts, and run counts (Perla et al. 2011) – has been shown to produce a very high false positive rate. In a run chart with 24 data points, the probability of at least one false signal may approach 50% (Anhøj 2015). A major contributor is the "trend rule", defined as five or more consecutive data points increasing or decreasing. Such short trends are common even in random data and are therefore unreliable indicators of change (Davis and Woodall 1988).

Another related pitfall is the deliberate tightening of control limits in an attempt to increase sensitivity. This often arises from an incorrect use of the overall standard deviation when calculating limits (Wheeler 2010, 2016). As emphasised throughout this book, control limits should be based on within-subgroup variation, not overall variation. Using the overall standard deviation incorporates special cause variation and inflates the limits, which may then prompt misguided attempts to narrow them artificially.

To avoid signal fatigue, practitioners should use a small, carefully selected set of rules. The three rules recommended in this book – the 3-sigma rule and the two runs rules – provide a balanced approach, offering reasonable sensitivity while maintaining an acceptable false alarm rate.

18.3 Confusing the voice of the customer with the voice of the process

Effective SPC requires listening to two distinct "voices": the voice of the customer and the voice of the process. Confusing the two can lead to inappropriate actions and poor decisions.

The voice of the customer reflects desired performance and is typically expressed through specification limits or targets. The voice of the process,

by contrast, reflects actual performance and capability, as revealed by the data and the control limits.

For example, a target may specify that an emergency caesarean section should be completed within 30 minutes. It may be tempting to treat any case exceeding this threshold as a special cause requiring investigation. However, without considering whether the process is stable and what it is capable of, such an approach is misleading.

Unacceptable outcomes may arise from stable processes, and acceptable outcomes may occur in unstable ones. Before acting on individual observations, we must determine whether variation is due to common causes or special causes. This distinction is crucial, as the appropriate response differs fundamentally.

Common cause variation requires changes to the system itself, while special cause variation requires investigation of specific events. The ultimate goal is to establish stable processes that consistently deliver acceptable outcomes. The relationship between these two "voices" can be summarised as in Table 18.1.

Table 18.1: The interplay between the voice of the customer and the voice of the process, with the four appropriate actions for each scenario.

	Process Stable	**Process Unstable**
Customer Satisfied	Maintain and monitor	Stabilise the process
Customer Unsatisfied	Redesign the process	Redesign and stabilise

18.4 Automatic rephasing

Rephasing involves splitting an SPC chart and recalculating the centre line and control limits after a sustained shift has been identified.

When done deliberately and with understanding of the underlying process, rephasing can be useful. Figure 18.1, for example, correctly separates two stable periods before and after an intervention.

```
qic(month, n,
    data  = cdi,
    chart = 'c',
    part  = period,
    title = 'Hospital associated C. diff.-infections',
    ylab  = 'Count',
    xlab  = 'Month')
```

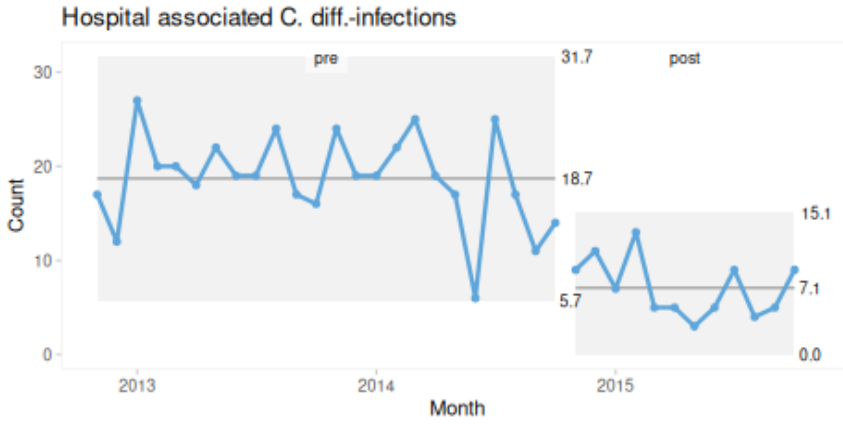

Figure 18.1: Hospital infections – rephasing done right!

However, some software systems perform automatic rephasing whenever a shift is detected. This can lead to misleading results.

In Figure 18.2, automatic rephasing occurs too early, incorrectly assigning data points to the post-intervention period. This distorts the centre lines and control limits and may even create false signals of instability.

```
qic(month, n,
    data        = cdi,
    chart       = 'c',
    part        = 22,
    part.labels = c('pre', 'post'),
    title       = 'Hospital associated C. diff.-infections',
    ylab        = 'Count',
    xlab        = 'Month')
```

Automatic rephasing removes essential human judgement from the process. For this reason, we strongly discourage its use. Rephasing should always be a deliberate decision, based on a clear understanding of the process and any associated interventions.

Rephasing *may* be appropriate when the following conditions are met:

- there is a sustained shift in data,
- the cause of the shift is known,
- the shift is in the desired direction, and
- the shift is expected to persist.

If these conditions are not met, the focus should remain on understanding the causes of variation.

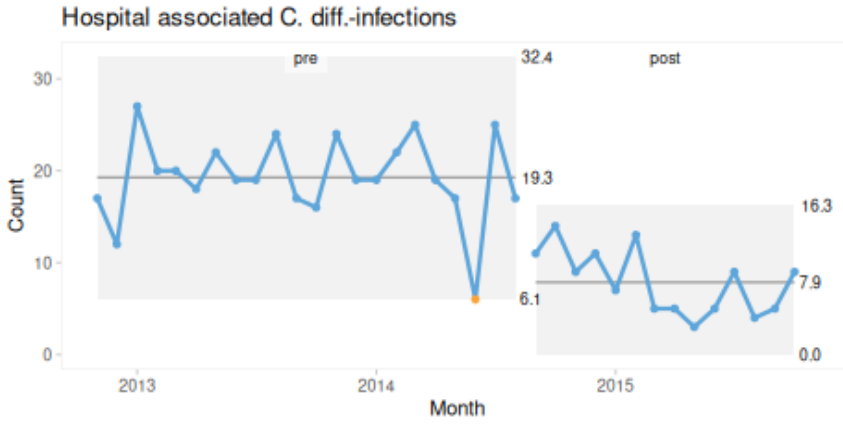

Figure 18.2: Hospital infections – rephasing done wrong!

18.5 The control chart vs run chart debate

A common misconception is that control charts are inherently superior to run charts (e.g. Carey (2002)). Similarly, the two are sometimes presented as fundamentally different methods (e.g. Perla et al. (2011)). We disagree with both views.

Run charts and control charts are best understood as variations of the same underlying concept: SPC charts – time series charts that use statistical rules to detect non-random variation.

The main difference is that control charts include control limits, while run charts do not. Control limits help detect large, sudden changes and provide a visual representation of process variation. Runs analysis, on the other hand, is particularly effective for detecting smaller, sustained shifts and trends.

These approaches are complementary rather than competing. In fact, some traditional control chart rules – such as the Western Electric rule for runs – are based on the same principles as runs analysis.

Run charts also offer practical advantages. They are simpler to construct, especially without specialised software, and they rely on fewer assumptions. Using the median as the centre line ensures a balanced division of data and reduces sensitivity to distributional assumptions.

For improvement work, where the goal is to detect sustained changes, run charts are often sufficient. Control limits can be added later when the focus shifts to monitoring stability and detecting large, sudden changes.

18.6 Assuming a one-to-one link between PDSA cycles and data points

A final source of confusion arises from the relationship between SPC charts and Plan-Do-Study-Act (PDSA) cycles. Because the two are often taught together, it is sometimes assumed that each data point on an SPC chart corresponds directly to a specific PDSA cycle.

In practice, this is rarely the case. PDSA cycles typically operate on short timescales – minutes, hours, or days – while SPC charts often reflect aggregated data over longer periods – days, weeks, or months.

As a result, signals on an SPC chart cannot always be directly attributed to individual PDSA cycles, and the absence of signals does not necessarily imply that a change has failed.

SPC and PDSA serve complementary but distinct purposes. SPC provides a broader view of process behaviour over time, while PDSA supports rapid, iterative testing and learning. Recognising this distinction helps ensure that both methods are used effectively in practice.

Chapter 19

The Forgotten Art of Rational Subgrouping

A subgroup consists of the data elements that make up a single data point, for example, the waiting times used to calculate the average waiting time for a particular period.

Rational subgrouping is one of the most important – and most frequently misunderstood – aspects of statistical process control. Poor subgrouping is a common reason why SPC charts fail in practice.

Rational subgrouping is the intentional and intelligent sampling and grouping of data into data points for SPC charts, with the aim of maximising the chances of detecting special cause variation while minimising the risk of false alarms. In other words, rational subgrouping is about maximising the signal-to-noise ratio.

A rational subgroup consists of a set of measurements or counts that are:

- produced under conditions that are as similar as possible,
- taken close together in time and space, and
- likely to show only common cause variation.

The underlying logic is simple but powerful: when subgroups are rationally formed, variation *within subgroups* reflects common cause variation, while variation *between subgroups* that exceeds what is expected from within-subgroup variation indicates the presence of special causes.

In our experience – particularly in healthcare – rational subgrouping is something of a forgotten art. Too often, we rely on whatever data are readily available, typically pre-aggregated into monthly, quarterly, or yearly

summaries for administrative purposes rather than for improvement. This often leads to suboptimal charts that obscure meaningful signals.

19.1 Too large subgroups – masking meaningful signals

A common mistake is to form subgroups across overly broad spans of time or space (e.g. large organisational units). This blends common and special cause variation and effectively hides the latter.

As an example, figures 19.1 and 19.2 display the number of C. diff. infections subgrouped by monthly and two-monthly periods respectively.

```
qic(month, infections,
    data  = cdiff,
    chart = 'c',
    title = 'C. diff. infections',
    ylab  = 'Count',
    xlab  = 'Months')
```

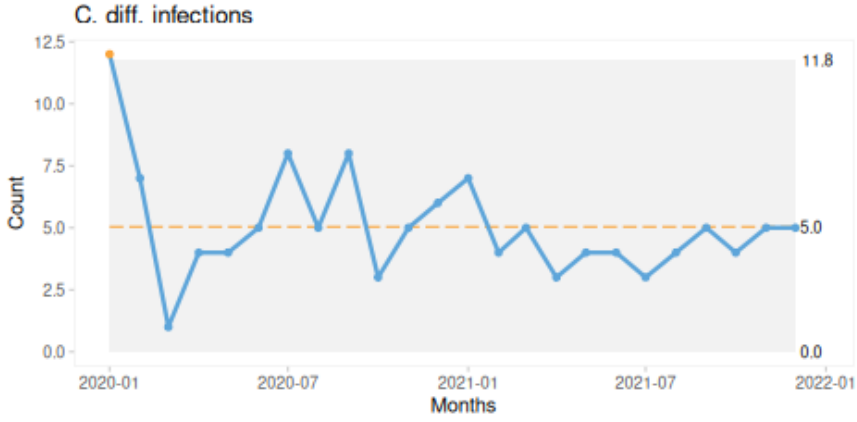

Figure 19.1: Monthly C. diff. infections.

```
qic(month, infections,
    data     = cdiff,
    chart    = 'c',
    x.period = '2 months',
    title    = 'C. diff. infections',
    ylab     = 'Count',
    xlab     = 'Two-months')
```

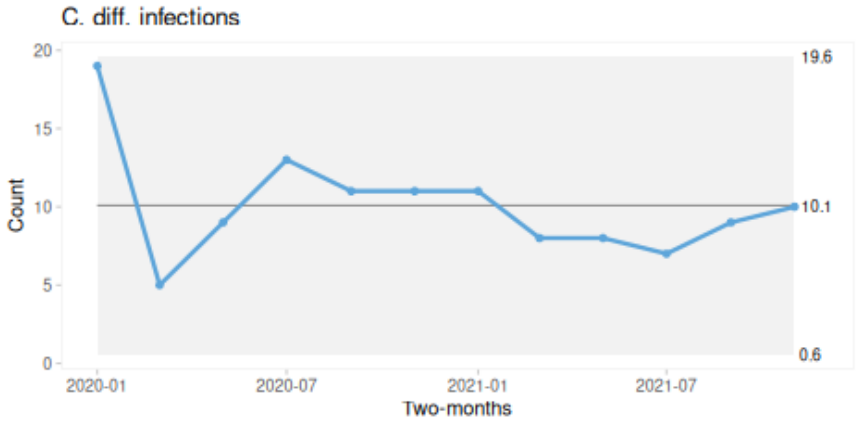

Figure 19.2: Two-monthly C. diff. infections.

By using larger subgroups, we mask two important signals – a freak value and a sustained shift – suggesting that the process is improving. While this trend may eventually become visible even with coarser data, detection is delayed, postponing learning and potential intervention.

A useful rule of thumb is: Collect data at a resolution at least as fine as the rate of change you want to detect.

High-resolution data can always be aggregated later if needed. Low-resolution data cannot be disaggregated.

Notice that the `qic()` function from *qicharts2* includes an argument, x.period, which allows us to aggregate data into larger subgroups as demonstrated in the code producing Figure 19.2.

19.2 Too small subgroups – revealing unimportant noise

A less common – but equally important – mistake is to use subgroups that are too small. In this case, natural process variation may be misinterpreted as special cause variation.

Imagine weighing yourself three times each morning for a couple of weeks and plotting the results on an X-bar chart, using within-day variation to calculate control limits.

Figure 19.3 illustrates this with simulated data.

```r
# lock random number generator for reproducibility
set.seed(5)

# subgroups, 12 subgroups of three values
x <- rep(1:12, each = 3)

# random values, each repeated three times
y <- rep(rnorm(12, mean = 80, sd = 0.5), each = 3)

# add a bit of random noise within subgroups
y <- jitter(y, amount = 0.2)

# plot Xbar-chart
qic(x, y, chart = 'xbar')
```

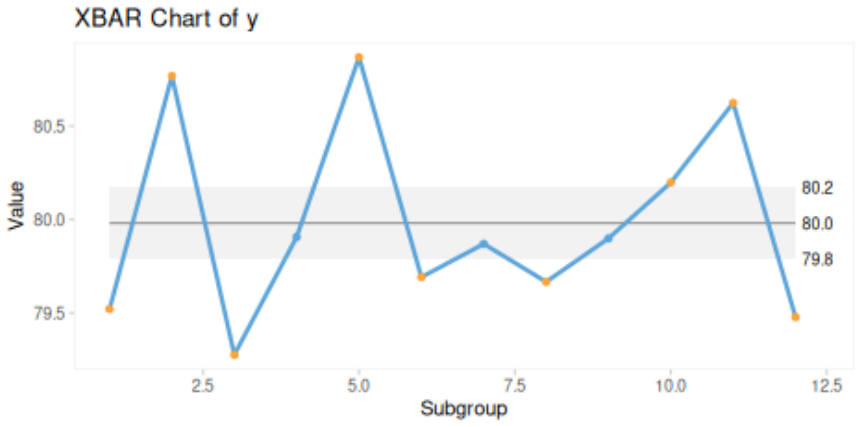

Figure 19.3: Xbar-chart from too small subgroups.

Here, within-subgroup variation is very small, while between-subgroup variation appears large – suggesting instability. In reality, this reflects normal day-to-day physiological variation, not a meaningful signal.

A better approach is to use an I chart of daily averages as in Figure 19.4:

```r
qic(x, y, chart = 'i')
```

```
## Subgroup size > 1. Data have been aggregated using mean().
```

Alternatively, we could define subgroups differently – for example, weekly averages – depending on our goal.

A practical guideline:

- Monitoring stability → larger subgroups (e.g. weekly)
- Detecting change → smaller subgroups (e.g. daily)

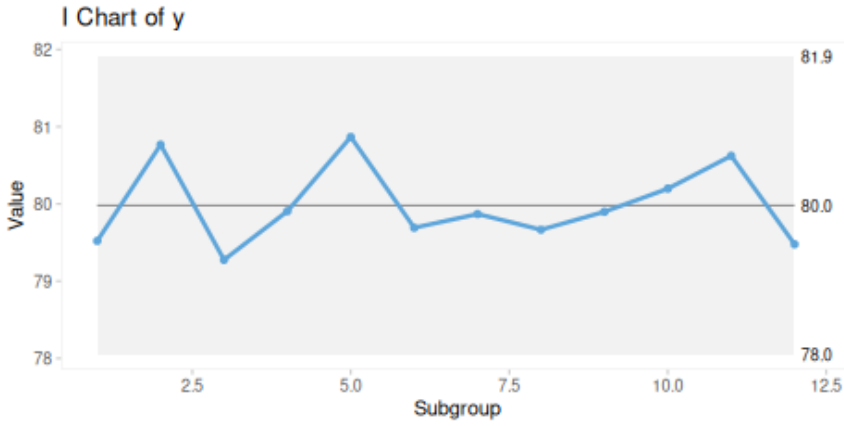

Figure 19.4: I-chart of average daily body weight.

19.3 Striking the balance

Rational subgrouping is both an art and a science. It presents a fundamental challenge: To understand a process, we need rational subgroups – but to form rational subgroups, we need to understand the process.

In practice, subgrouping is often iterative. We experiment with different timeframes, organisational units, or groupings until the data begin to reflect meaningful process behaviour.

This challenge is particularly pronounced in healthcare. Unlike manufacturing, where inputs and processes can be tightly controlled, healthcare processes are inherently complex. Outcomes are influenced by patient characteristics, clinical decisions, staffing, and system-level factors.

However, limited control does not mean no control. By carefully selecting:

- patient groups
- clinical pathways
- organisational units
- time intervals

we can still design subgrouping strategies that reduce noise and improve signal detection.

19.4 A practical approach for rational subgrouping

There is no single correct method for defining rational subgroups. In practice, subgrouping is best approached as an iterative process – often aligned with Plan-Do-Study-Act cycles.

The following principles can guide your decisions:

- Work with process experts Rational subgrouping is not purely statistical – it requires contextual understanding.
- Map the process Identify inputs, outputs, and sources of variation.
- Clarify your purpose Are you monitoring stability or trying to detect improvement?
- Choose appropriate granularity Decide whether to aggregate or stratify by time, location, or population.
- Match sampling to expected change Sample more frequently than the rate at which change is expected.
- Ensure sufficient data per subgroup Count data: aim for at least five events or cases. Proportion data: ensure sufficient denominator.
- Be prepared to adapt Revise your approach as you learn more about the process.

These principles often involve trade-offs. For example, increasing temporal resolution may reduce subgroup size. In such cases, alternative approaches – such as different chart types (e.g. rare event charts) – may be more appropriate.

19.5 Rational subgrouping in summary

Rational subgrouping is one of the most challenging aspects of SPC – and one of the most important. There is no universal recipe. It requires judgement, experimentation, and a deepening understanding of the process over time.

But one thing is clear: Poor subgrouping produces misleading charts. Good subgrouping reveals the truth about the process.

Getting subgrouping right is not optional - it is what separates SPC charts that inform improvement from those that obscure it.

Part 5: Conclusion and Final Thoughts

Chapter 20

Closing Remarks

20.1 From charts to practice: what it really takes

Throughout this book, we have focused on the practical application of statistical process control in healthcare – how to construct charts, calculate limits, and interpret signals.

But as should now be clear, producing charts is only a small part of the task.

SPC charts are only as good as the data behind them. Poor data quality, inconsistent definitions, or inappropriate subgrouping can distort signals or obscure meaningful patterns. Even when charts are constructed correctly, misinterpretation remains a risk – particularly when too many rules are applied or when signals are taken at face value without understanding the underlying process.

Improving data quality is therefore not a preliminary step that happens before SPC – it is an integral part of SPC practice itself. In many cases, the act of using SPC reveals weaknesses in data definitions, collection processes, and recording systems. These insights should not be seen as obstacles, but as opportunities to improve the data system alongside the process being studied.

More fundamentally, SPC does not eliminate uncertainty. Every chart involves a balance between detecting real signals and avoiding false alarms. Mistaking common cause variation for special cause, or overlooking genuine signals, can both lead to inappropriate action.

In healthcare, these challenges are amplified by the complexity of the systems being studied. Outcomes are influenced by many interacting factors,

and approaches such as risk adjustment – while often necessary – can introduce additional sources of error if applied uncritically.

In short, doing SPC well requires more than technical competence. It requires judgement – and a continual commitment to improving both processes and the data used to understand them.

20.2 Building SPC into everyday systems

SPC is often introduced through charts, but its real value emerges only when it becomes part of everyday work.

Charts do not improve systems – people do. SPC provides a structured way of learning from data, but it is only effective when embedded within a broader system of inquiry, dialogue, and action.

Successful use of SPC therefore depends on:

- clear aims and meaningful indicators,
- reliable and well-defined data,
- appropriate sampling and subgrouping,
- engagement of stakeholders at all levels, and
- a culture that supports learning rather than blame.

Without these elements, SPC risks being reduced to a reporting exercise or box-ticking activity, rather than a tool for improvement.

The paradox we introduced at the beginning of this book remains central here: SPC is simple in concept, but difficult to apply well. The tools themselves are straightforward, but using them correctly within real systems requires coordination, discipline, and shared understanding.

20.3 Working with variation: discipline over reaction

At its core, SPC is about changing how we respond to data.

Rather than reacting to individual data points, SPC encourages us to understand processes over time. Rather than asking “Is this good or bad?”, we ask “What does this tell us about the system?”. And rather than seeking immediate fixes, we focus on learning and improvement.

This requires discipline. It means resisting the urge to respond to every fluctuation, and instead paying attention to patterns that are unlikely to

occur by chance. It means investigating signals systematically, using both data and contextual knowledge. And it means recognising that improvement is rarely the result of a single intervention, but of sustained effort over time.

Importantly, SPC supports a culture of curiosity rather than judgement. Signals of special cause variation are not, in themselves, evidence of failure or success – they are invitations to learn.

20.4 Scaling SPC in a data-rich world

Looking ahead, the context in which SPC is applied is changing rapidly.

Healthcare systems are generating increasing amounts of data across clinical care, operations, and population health. At the same time, SPC is being implemented at larger organisational scales. This creates new challenges.

As the number of charts grows from a handful to hundreds or thousands, the problem is no longer how to construct charts, but how to organise, interpret, and act on them. There is a real risk of losing clarity – of not seeing the wood for the trees.

The rise of so-called big data introduces further complexity. With high-frequency, high-volume data, we must reconsider fundamental questions:

- How often should we sample and analyse data?
- How should data be aggregated into meaningful subgroups?
- How do we maintain acceptable false alarm rates when monitoring many variables simultaneously?

Without careful design, more data can lead to more noise, more false signals, and ultimately less trust in the system.

At the same time, increasing data availability places even greater demands on data quality. Large volumes of poorly defined or inconsistently collected data do not strengthen SPC – they weaken it. Ensuring data quality at scale will therefore be one of the central challenges for the future of SPC in healthcare.

The future of SPC will depend not only on statistical methods, but also on better data systems, automation, visualisation, and – crucially – user understanding.

20.5 A shared future: principles, practice, and community

Despite these changes, the fundamental idea of SPC remains unchanged.

All processes exhibit variation, and this variation arises from two sources: common cause and special cause. This simple distinction provides a powerful lens through which to understand, monitor, and improve systems. With this insight comes a set of visual tools – run charts and control charts – that make variation visible and interpretable.

And yet, as we have emphasised throughout this book, there is a paradox: while these tools are conceptually simple, applying them correctly is not. Constructing charts, defining measures, forming rational subgroups, and interpreting signals all require care and experience. SPC is, at the same time, both simple and difficult.

This book reflects our current understanding and experience of what is most readily applicable in healthcare practice. Inevitably, this means that it is selective. There are many additional topics that we have not covered in detail – including case-mix adjusted SPC charts, Tukey charts, multivariate SPC methods, and other specialised approaches.

These areas offer important opportunities for further development and application. Their omission here is not a judgement of their value, but a reflection of our focus on methods that are practical, accessible, and immediately useful in most healthcare settings.

We therefore see this book not as a finished product, but as part of an evolving body of work – and as a community resource.

We invite you, as a practitioner, to contribute to its ongoing development. This may include:

- identifying errors or ambiguities,
- suggesting clarifications or alternative explanations,
- sharing examples from practice, and
- proposing new methods, tools, or applications.

By doing so, you help ensure that SPC continues to evolve in response to real-world challenges.

Ultimately, the value of SPC lies not in the charts themselves, but in how they are used – to inform thinking, guide action, and support learning.

If there is one message to take from this book, it is this: Improvement begins with understanding variation.

By developing the skills, systems, and culture needed to learn from variation – and by continuously improving the quality of the data on which that learning depends – organisations can move beyond reactive management towards more effective and sustainable improvement.

The journey does not end here.

Appendix A

Data Sets

Datasets for this book are provided as comma separated values (csv) in text files. For details on how to read data from csv-files, see the Importing data from text files in Appendix C.

Adverse Events

Patient harm found with the Global Trigger Tool

File: adverse_events.csv

Variables:

- severity (character): Harm severity: E = minor transient, I = fatal
- category (character): Harm category

Bacteremia

Hospital acquired and all cause bacteremias and 30 days mortality

File: bacteremia.csv

Variables:

- month (date): month of infection
- ha_infections (numeric): number of hospital acquired infections
- risk_days (numeric): number of patient days without infection
- deaths (numeric): 30-day mortality after all-cause infection
- patients (numeric): number of patients with all-cause infection

Blood pressure

Daily measurements of blood pressure and resting pulse.

File: blood_pressure.csv

Variables:

- date (date): date of measurement
- systolic (numeric): systolic blood pressure (mm Hg)
- diastolic (numeric): diastolic blood pressure (mm Hg)
- pulse (numeric): resting pulse (beats per minute)

Clostridioides difficile infections

Hospital acquired C. diff. infections

File: cdiff.csv

Variables:

- month (date): first day of month
- cases (numeric): number of cases
- risk_days (numeric): number of patient days without infection

Ceasearian section delay

Time to grade 2 C-section

File: csection_delay.csv

Variables:

- datetime (datetime): date and time of delivery
- month (date): first day of month
- delay (numeric): time in minutes between decision and delivery

Diabetes HbA1c

HbA1c measurements in children with diabetes

File: diabetes_hba1c.csv

Variables:

- month (date): month of measurements
- avg_hba1c (numeric): average of HbA1c measurements
- n (integer): number of patients who visited the clinic

Emergency admission mortality

7-day mortality after emergency admission

File: emergency_admission.csv

Variables:

- month (date): first day of month
- deaths (numeric): number of deaths within 7 days after emergency admission
- admissions (numeric): number of emergency admissions

On-time CT

Patients with acute abdomen CT scanned within 3 hours after arrival

File: ontime_ct.csv

Variables:

- month (date): first day of month
- ct_on_time (numeric): number of patients scanned within 3 hours
- cases (numeric): number of patients with acute abdomen

Radiation doses

Radiation doses used for renography

File: renography_doses.csv

Variables:

- date (date): date of renography
- week (date): first day of week
- dose (numeric): radiation dose in megabequerel

Robson group 1 births

Outcomes and complications of Robson group 1 births: first time pregnancy, single baby, head first, gestational age at least 37 weeks.

File: robson1_births.csv

Variables:

- datetime (datetime): data and time of birth
- biweek (date): first day of biweekly period
- csect (logical): delivery by C-section
- cup (logical): delivery by vacuum extraction
- sex (character): sex of baby
- length (numeric): baby length in cm
- weight (numeric): baby weight in kg
- apgar (numeric): apgar score at 5 minutes
- ph (numeric): arterial umbilical chord pH
- asphyxia (logical): ph < 7 or missing ph and apgar < 7

Appendix B

Basic Statistical Concepts

Understanding data is the foundation of effective decision-making in quality improvement. While this chapter is not intended as a comprehensive statistics course, it introduces key concepts related to probability, data types, summarisation, and visualisation.

These concepts are essential for selecting appropriate SPC charts, interpreting patterns correctly, and ensuring that conclusions drawn from data are valid.

B.1 Probability

Probability is a measure of how likely a given event is to occur and is central to statistics.

Probabilities range from 0 to 1, where 0 means that the event cannot happen, and 1 means that the event is certain to happen. Any number between 0 and 1 represents the expected proportion of times the event would occur if the experiment were repeated indefinitely. For example, the probability of a fair coin landing on heads is $P(H) = 0.5$ or 50%.

Probabilities sum up to 1, such that the probability of a coin landing on *either* heads *or* tails is

$$P(H \text{ or } T) = P(H) + P(T) = 0.5 + 0.5 = 1$$

The probability of multiple independent and mutually exclusive events happening is the product of their individual probabilities. For example, the probability of two coins both landing on heads is

$$P(H \text{ and } H) = P(H) \times P(H) = 0.5 \times 0.5 = 0.25$$

Based on these principles, probability allows us to predict the likelihood of certain events – such as special cause patterns in an SPC chart – and thereby distinguish common cause from special cause variation.

For example, if the probability that a single data point falls outside the three-sigma limits is $P(\text{signal}) = 0.003$, then the probability that all 20 data points in a control chart fall within the limits is

$$P(\text{no signal}) = (1 - 0.003)^{20} = 0.94$$

In other words, a control chart with no three-sigma signals is more likely to represent common cause variation than special cause variation.

Most importantly, it follows that we can never be absolutely certain that signals always represent special causes, or that the absence of signals always represents common causes. Statistics is not a tool for certainty. Rather, it is a tool for making informed judgements under uncertainty by quantifying how likely different explanations are in light of the available data.

SPC is fundamentally concerned with prediction rather than hypothesis testing, p-values and retrospective description alone. A stable process allows us to make probabilistic statements about the range within which future observations are likely to fall if the underlying system remains unchanged. Control limits therefore represent operational expectations about future process behaviour, not merely summaries of past data.

> [...] prediction within limits means that we can state, at least approximately, the probability that the observed phenomenon will fall within the given limits.
>
> Shewhart (1931), p. 6

In this context, *probability* may be informed by empirical observations, theoretical considerations, or both, regarding the distribution and variability of data collected over time. The remainder of this appendix describes how such knowledge can be obtained and used in practice.

B.2 Data types

Data can broadly be classified into categorical and numerical types. Distinguishing between these is crucial, as it determines how data can be analysed and visualised.

B.2.1 Categorical data

Categorical (qualitative) data describe characteristics rather than quantities. Examples include: colours [red, green, blue], specialities [medicine, surgery, psychiatry], and risk levels [low, medium, high].

These data cannot be meaningfully combined using arithmetic operations. Categorical data can be further divided into:

- **Nominal** data: Categories without inherent order, e.g. [red, green, blue].
- **Ordinal** data: Categories with a meaningful order but unequal intervals, e.g. [low < medium < high].

If only two categories exist (e.g. yes/no, alive/dead), the data are binary (or binomial).

Categorical data are often converted into numbers by counting frequencies. In some cases, ordinal categories may be assigned numerical scores, but this should only be done when the intervals are meaningful. For example, converting cancer stages [I, II, III] to [1, 2, 3] is misleading unless the progression is truly proportional.

Finally, some data may appear numerical but are actually categorical identifiers (e.g. postal codes, ID numbers). A simple test is to ask: does it make sense to calculate an average? If not, the data are categorical.

B.2.2 Numerical data

Numerical (quantitative) data represent measurable quantities and can be analysed using arithmetic operations.

They are divided into:

- **Discrete** data: counts (whole numbers), e.g. number of admissions, infections.
- **Continuous** data: measurements (any value within a range), e.g. height, weight, waiting time.

Numerical data can also be grouped into categories (e.g. blood pressure or body weight), but doing so reduces information and should be done with care.

B.3 Summarising categorical data

Categorical data are typically summarised using counts or proportions. Using the Adverse events dataset assigned to the variable ae:

```
# count number of adverse events in each category
(ae.tbl <- table(ae$category))
```

```
##
##             Fall Gastrointestinal        Infection       Medication
##                1               40               34               18
##            Other   Pressure ulcer        Procedure
##                4                5               29
```

Visualisation of categorical data is often done using bar charts:

```
# plot frequencies
barplot(ae.tbl)
```

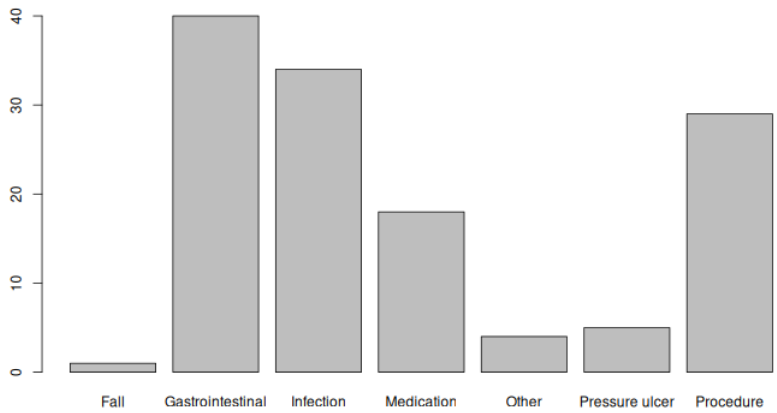

Figure B.1: Bar chart of adverse event frequencies

```
# plot proportions
barplot(prop.table(ae.tbl))
```

The only difference between figures B.1 and B.2 is the y-axis scale.

For binary data, we often summarise using proportions:

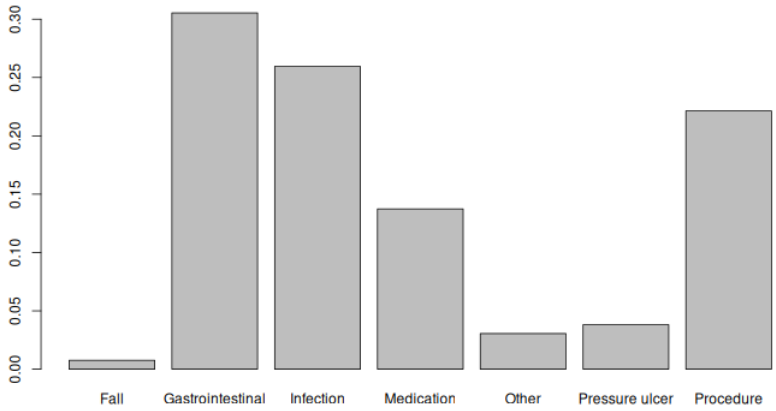

Figure B.2: Bar chart of adverse event proportions.

```
# proportion of fatal adverse events (severity = 'I').
mean(ae$severity == 'I')
```

```
## [1] 0.007633588
```

B.4 Summarising numerical data

Numerical data are typically described using:

- **Centre** (central tendency)
- **Shape** (distribution)
- **Spread** (variation)

A quick overview can be obtained using the `summary()` function:

```
# summarise the length of newborn babies
summary(births$length)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
##    35.0    50.0    52.0    51.7    53.0    60.0       3
```

B.4.1 Centre

The centre describes the "typical" value in the data.

- **Mean**: average of all values
- **Median**: middle value (after sorting)

The mean is sensitive to extreme values, whereas the median is robust.

Example:

- [1, 2, 3]: mean = 2, median = 2
- [1, 2, 999]: mean = 334, median = 2

The median is preferred in run charts because it divides the observations into two equally probable halves, which is essential for probability-based runs analysis.

The mean is usually used in control charts – particularly when the data are approximately symmetric (see below). Because the mean reflects the value of all observations, it may better reflect the overall level of the process and may be more sensitive to sustained shifts in the data.

B.4.2 Shape

The shape describes how values are distributed.

Key features include:

- **Symmetry / skewness**
  - symmetric: mean $\approx$ median
  - right-skewed: long tail to the right, mean $>$ median
  - left-skewed: long tail to the left, mean $<$ median
- **Modality**
  - unimodal: one peak
  - bimodal: two peaks

Histograms provide a clear visualisation (Figure B.3):

```
hist(births$length)
```

Bimodal (and multimodal) data often indicate different underlying processes and should prompt investigation and possible stratification.

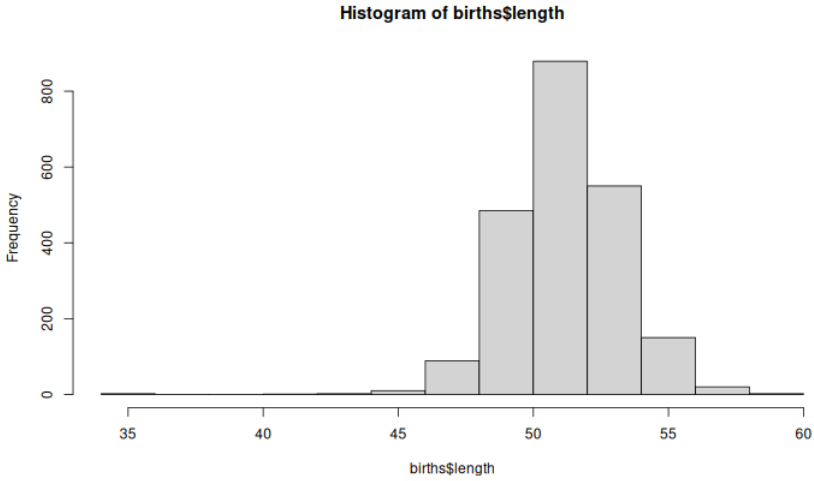

Figure B.3: Histogram of birth lengths.

B.4.3 Spread

Spread describes how much the data vary.

Common measures:

- **Range**: max – min
- **Interquartile range** (IQR): middle 50%
- **Standard deviation** (SD): average distance from the mean

A useful visual summary of centre, shape, and spread is the boxplot (Figure B.4):

```
boxplot(births$length)
```

The box in a boxplot represents the interquartile range (IQR), while the line inside the box indicates the median. The whiskers extend from either end of the box to the smallest and largest values that lie within 1.5 times the IQR from the quartiles. Dots beyond the whiskers represent more extreme values – often referred to as outliers, though there is often nothing truly outlandish about them.

In SPC, **standard deviation** (SD) is central because it defines control limits. SD can be loosely described as a measure of the average distance from the centre.

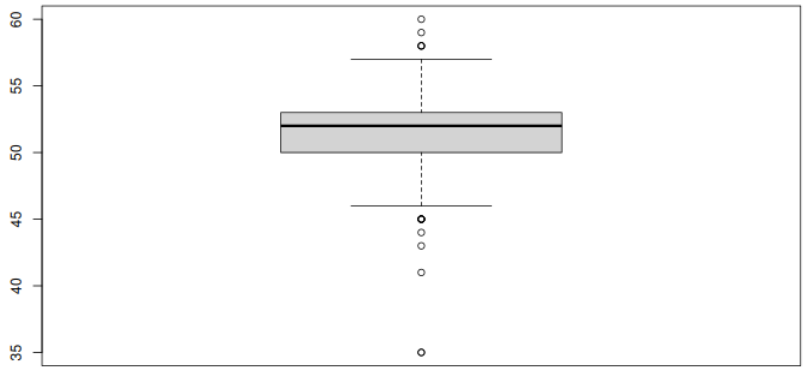

Figure B.4: Boxplot of birth lengths.

For many symmetrical distributions:

- ~68% within ±1 SD
- ~95% within ±2 SD
- ~99% within ±3 SD

Especially, for any unimodal distribution (regardless of symmetry), over 96% of points are expected within ±3 SDs. This supports the use of three-sigma limits even when data are not perfectly normal.

The method used to estimate SD depends on the type and the assumed theoretical distribution of data.

B.5 Theoretical distributions

While SPC is primarily empirical, theoretical distributions help define expected variation. We focus on three:

- Poisson (event counts/rates), e.g. daily emergency department arrivals
- Binomial (case counts/proportions), e.g. proportion of patients receiving treatment within a target time
- Gaussian (continuous data), e.g. patient blood pressure measurements

B.5.1 Poisson distribution – predicting the number of events

The Poisson distribution – named after the French mathematician Siméon Poisson – describes the probability of a given number of events occurring within a fixed time interval, assuming that the events happen independently of one another and at a constant average rate, commonly denoted by lambda (λ). It is used to estimate the control limits for C- and U-charts.

Based on the birth data, we estimate the expected birth rate to be about 6 births per day.

I R, we can use the `dpois()` function to calculate the probability (or density) of any given number of births in a day. For example, the probability of having exactly 9 births in one days is `dpois(9, lambda = 6) = 0.0688385`.

The standard deviation of a Poisson distribution (event counts) conveniently equals the square root of lambda:

$$SD = \sqrt{\lambda}$$

For event rates:

$$SD = \sqrt{\lambda / n_i}$$

where n_i is the size of i^{th} time interval (the area of opportunity).

To visualise discrete probability distributions, we use stick charts as demonstrated in Figure B.5.

```
# plot Poisson probability
x <- 0:16 # x-axis values
plot(x, dpois(x, lambda = 6),
     type = 'h',
     lwd  = 2,
     ylab = 'Probability',
     xlab = 'Number of births')
```

Note that the Poisson distribution is censored at zero – it cannot produce negative counts – but, in principle, it extends towards infinity. As with all probability distributions, the sum of all probabilities is equal to 1.

For low values of lambda, the Poisson distribution is right-skewed; however, as lambda increases, the distribution becomes increasingly symmetric and begins to resemble the Gaussian distribution.

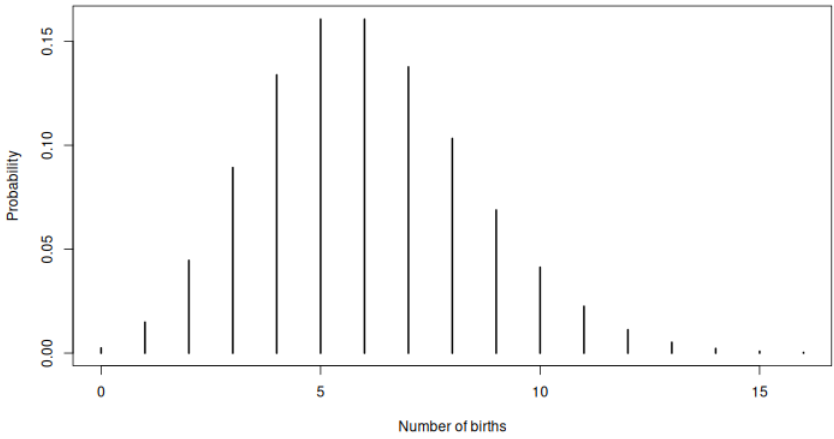

Figure B.5: Poisson probability plot of the number of births in a day (mean = 6).

B.5.2 Binomial distribution – predicting the number of cases of "success" or "failure"

The binomial distribution describes the probability of obtaining a given number of cases – often referred to as successes or failures – in a fixed number of independent trials (or opportunities), each with two possible outcomes (such as yes/no or pass/fail), and a constant probability of success/failure.

The standard deviation of a binomial distribution (case counts) is:

$$SD = \sqrt{np(1-p)}$$

where n is the sample size and p the success proportion.

For case proportions:

$$SD = \sqrt{p(1-p)/n}$$

For example, if we consider each birth as a trial and define a case as a caesarean section (C-section), the binomial distribution can be used to model the probability of observing a specific number of C-sections within a set number of births.

In R, the `dbinom()` function is used to calculate binomial probabilities. In addition to the number of cases of interest, the function requires the total number of opportunities (sample size) and the average probability of a case.

There are approximately 42 births per week, with about 9% resulting in a C-section. To calculate the probability of observing exactly 6 C-sections in a given week, we use: `dbinom(6, size = 42, prob = 0.09)` = `0.093488`.

```
# plot binomial probability
x <- 0:42
plot(x, dbinom(x, size = 42, prob = 0.09),
     type = 'h',
     lwd  = 2,
     ylab = 'Probability',
     xlab = 'Number of C-sections')
```

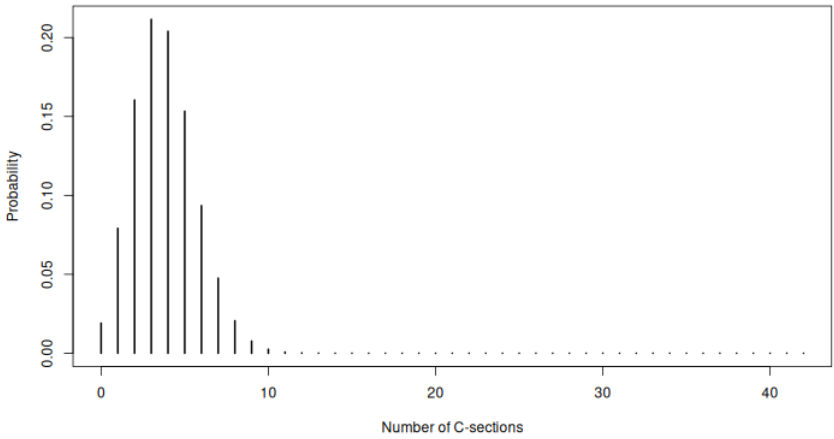

Figure B.6: Binomial probability plot of the number of C-sections in a week (size = 42, prob = 0.09).

As with the Poisson distribution, the binomial distribution is censored at zero. However, unlike the Poisson, its upper limit is fixed and equal to the sample size, since it models a finite number of opportunities (Figure B.6).

The shape of the binomial distribution is perfectly symmetric – resembling the Gaussian distribution – when the success probability is close to 50% and grows increasingly skewed when this approaches 0% (right-skew) or 100% (left-skew). The degree of skewness depends largely on the sample size – bigger samples, less skew and more symmetry.

These considerations are important when designing a sampling plan. To

quickly gather enough data points for a control chart, smaller samples are preferable. However, to maintain symmetry – which is important for the reliability of the P-, C-, and U-charts – larger samples are needed.

B.5.3 Gaussian distribution – predicting the probability of continuous outcomes

The Gaussian distribution, named after the German mathematician Carl Gauss and often referred to as the normal distribution – though there is nothing inherently "normal" about it – is a continuous probability distribution widely used to model many real-world phenomena.

To calculate SD, we use:

$$SD = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

where n is the number of observations, x_i is the i^{th} observation, and $\bar{x}$ is the average of all observations.

Figure B.7 shows a histogram of birth weights, where each bin represents the number of weights that fall within a specific range.

```
hist(births$weight, breaks = 20)
```

In Figure B.8, we have overlaid a curve representing the theoretical Gaussian distribution, using the empirical mean and standard deviation of the birth weights.

```
avg_w <- mean(births$weight)
std_w <- sd(births$weight)
hist(births$weight, breaks = 20, freq = FALSE)
curve(dnorm(x,
            mean = avg_w,
            sd   = std_w),
      add = TRUE)
```

Note that the y-axis now represents density rather than frequency. The density curve doesn't assign probabilities to individual points – it represents the overall shape of the distribution – and probabilities are found by calculating the area under the curve over an interval. The total area under the curve – which, in theory ranges from minus infinity to plus infinity – also equals 1.

Using the `pnorm()` function, we can calculate the probability of outcomes within any interval. To find the probability of data falling within ±3 SD from the mean, we calculate the area under the curve like this:

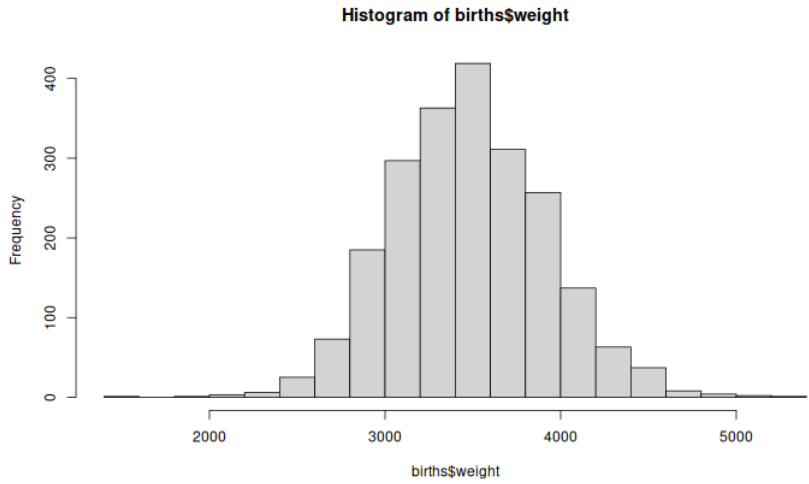

Figure B.7: Histogram of birth weights

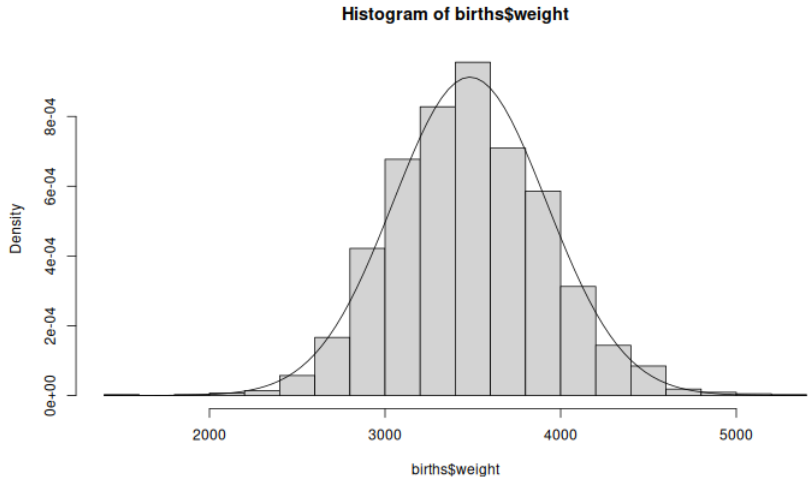

Figure B.8: Histogram of birth weights with overlayed Gaussian density curve.

```
pnorm(avg_w + 3 * std_w,    # area below mean + 3 SD
      mean = avg_w,
      sd   = std_w) -
  pnorm(avg_w - 3 * std_w,  # aread below mean - 3 SD
        mean = avg_w,
        sd   = std_w)
```

```
## [1] 0.9973002
```

B.5.4 A word of caution

In practice, healthcare data rarely fit theoretical statistical distributions perfectly. Processes may show extra variation, seasonal patterns, changing definitions or mixtures of different patient groups and pathways.

However, SPC remains useful because its main purpose is to support operational learning and understanding of processes rather than perfect statistical modelling.

B.6 Basic statistical concepts in summary

In this chapter, we have introduced key concepts for understanding data:

- Types of data (categorical vs numerical).
- Methods for summarising data (counts, proportions, centre, spread).
- Visual tools (bar charts, histograms, boxplots).
- Theoretical distributions used in SPC.

Before constructing SPC charts, it is good practice to:

- visualise the data,
- assess centre, shape, and spread, and
- check for anomalies or data quality issues.

These steps help ensure that SPC methods are applied appropriately and interpreted correctly.

Ultimately, good SPC depends not only on statistical methods, but on understanding the data and the process that generated them.

Appendix C

R Notes

This is not an R tutorial. Rather, this appendix aims to explain and clarify some of the tools and techniques we use in this book to import and prepare data for plotting using only base R functions. We are very well aware of new and modern approaches to data manipulation from the *tidyverse* and *data.table* packages – we use these tool ourselves every day – but we find it valuable to be able to handle data in base R. Especially when sharing code or building R packages it is generally a good strategy to avoid unnecessary dependencies on external packages.

First, we will look at some data structures and types that are essential to this book. Next, we will discuss some principles for importing data from text files into data frames and how to manipulate these to make data ready for plotting. Finally, we give some useful tips and tricks

C.1 Data structures and classes

The simplest data structure in R is the **vector**: zero or more data elements of the same type. Data come in several basic types (**classes**). We are mainly concerned with logical, numeric and character values. Date and datetime values, which are especially important in SPC, are basically just numeric values representing the number of days (dates) or seconds (datetimes) since the beginning of the year 1970.

The **data frame** is (for our purpose) probably the most important data structure in R. Data frames are collections of vectors that may be of different types but are all of the same length. Think matrix or table with rows and columns where each row represents an observation, each column represents a variable, and each cell represents a data value. All rows and

all columns have the same length, and all cells have values (including NA for missing data).

C.2 Plot-ready data frames

When plotting time series data, we need two vectors, one for the x axis representing the subgroups, which in its simplest form may be a sequence of numbers or dates and one for the y axis representing the indicator values to be plotted. To correctly calculate the control limits in spc charts, we often need a third variable representing the denominator of the count data in the y variable. Thus, a generic plot-ready data set for making an spc chart may look like this:

```
           x  y  n
1  2026-08-09 13 32
2  2026-08-10 14 29
3  2026-08-11 16 33
4  2026-08-12 19 30
5  2026-08-13 13 32
6  2026-08-14 18 35
7  2026-08-15 19 29
8  2026-08-16 16 33
9  2026-08-17 16 34
10 2026-08-18 11 27
11 2026-08-19 13 32
12 2026-08-20 12 26
```

To plot a P chart from these data with the qic() function from the qicharts2 package, we may do this, where "d" is the name of the data frame containing the three variables x, y, and n (Figure C.1):

```
qic(x, y, n,
    data  = d,
    chart = 'p')
```

Notice that to correctly plot dates or datetimes on the x axis, it is important that the x variable is of the correct class (Date for dates or POSIXct for datetimes).

C.3 Importing data from text files

When importing data from a text file into R using one of the base R read.*() functions, data are returned as a data frame.

For this book, we provide all data sets as comma separated values (csv) in text files that can be read using the read.csv() function. Each data file begins with a number of commented lines that explains the content and the

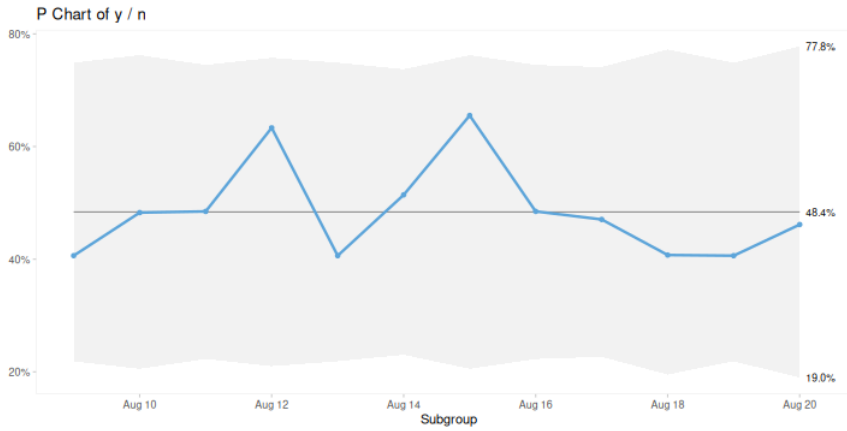

Figure C.1: P chart from data frame

variables in data. For example, the first 18 lines of the "bacteremia.csv" file looks like this:

```
# Bacteremia
#
# Hospital acquired and all cause bacteremias and 30 days mortality
#
# Variables:
#   month (date): month of infection
#   ha_infections (numeric): number of hospital acquired infections
#   risk_days (numeric): number of patient days without infection
#   deaths (numeric): 30-day mortality after all-cause infection
#   patients (numeric): number of patients with all-cause infection

month,ha_infections,risk_days,deaths,patients
2017-01-01,24,32421,23,100
2017-02-01,29,29349,22,105
2017-03-01,26,32981,13,99
2017-04-01,16,29588,14,85
2017-05-01,28,30856,17,98
2017-06-01,16,30544,15,85
...
```

Lines beginning with a hash symbol (#) are comments. The first non-blank line after the comments holds the variable names, and the the rest of the file contains the data values separated by commas (,).

Note how the dates in the first column are formatted using the only unmistakeable way of writing dates: year-month-day (yyyy-mm-dd). We highly recommend to always store dates in this format, which also happens to be the international ISO standard for writing dates. Also, ISO dates, when used in file names, sort correctly and also have the advantage of being easily recognised as dates by R (and other statistical software).

If dates are stored in any other format (e.g. dd-mm-yyyy), we may need to import them as character values and later convert them to dates using the as.Date() function.

When reading data in R programmes for use in production environments, we recommend that you specify the data type (class) of each column using the colClasses argument.

```
# read data from file and assign to variable named d
d <- read.csv('data/bacteremia.csv',
              comment.char = '#',
              colClasses   = c(month         = 'Date',
                               ha_infections = 'integer',
                               risk_days     = 'integer',
                               deaths        = 'integer',
                               patients      = 'integer'))

# print the first six lines of data
head(d)
```

```
      month ha_infections risk_days deaths patients
1 2017-01-01            24     32421     23      100
2 2017-02-01            29     29349     22      105
3 2017-03-01            26     32981     13       99
4 2017-04-01            16     29588     14       85
5 2017-05-01            28     30856     17       98
6 2017-06-01            16     30544     15       85
```

```
# print the data structure
str(d)
```

```
'data.frame':   24 obs. of  5 variables:
 $ month        : Date, format: "2017-01-01" "2017-02-01" ...
 $ ha_infections: int  24 29 26 16 28 16 14 18 27 30 ...
 $ risk_days    : int  32421 29349 32981 29588 30856 30544 26482 27637 30495 30600 ...
 $ deaths       : int  23 22 13 14 17 15 15 25 21 24 ...
 $ patients     : int  100 105 99 85 98 85 89 99 103 86 ...
```

C.4 Manipulating data frames

We will import the C-section data set to demonstrate adding variables and aggregating data.

```
d <- read.csv('data/csection_delay.csv',
              comment.char = '#',
              colClasses   = c(datetime = 'POSIXct',
                               month    = 'Date',
                               delay    = 'integer'))

head(d)
```

```
             datetime      month delay
1 2016-01-06 03:55:40 2016-01-01    22
2 2016-01-06 20:52:34 2016-01-01    22
```

```
3 2016-01-07 02:50:43 2016-01-01    29
4 2016-01-07 22:32:27 2016-01-01    28
5 2016-01-09 14:56:09 2016-01-01    22
6 2016-01-09 21:21:24 2016-01-01    20
```

The C-section data contains 208 rows representing individual C-sections. Our aim for this exercise is to reduce data to a plot ready data frame with one row per month and the number of C-section that were on target and the total number of C-sections.

C.4.1 Adding variables to data frames

The delay variable is the number of minutes from decision to perform a C-section to delivery of the baby. The standard target value for grade 2 C-sections is less than 30 minutes. If we want to plot the proportion of C-sections that are on time (i.e. less than 30 min.), we first need to dichotomise the delay variable into a logical variable, ontime, that is TRUE when delay is less than 30. We add this new variable to the data frame using the $-notation.

```
d$ontime <- d$delay < 30  # Dichotomise delays.

head(d)
```

```
             datetime      month delay ontime
1 2016-01-06 03:55:40 2016-01-01    22   TRUE
2 2016-01-06 20:52:34 2016-01-01    22   TRUE
3 2016-01-07 02:50:43 2016-01-01    29   TRUE
4 2016-01-07 22:32:27 2016-01-01    28   TRUE
5 2016-01-09 14:56:09 2016-01-01    22   TRUE
6 2016-01-09 21:21:24 2016-01-01    20   TRUE
```

C.4.2 Aggregating data frames

Then we aggregate data to one row per month. For this, we use the split-apply-combine strategy by first splitting the data frame into a list of data frames, one per month. Next, we apply the same summary function to all elements (months) of this list collapsing into a one-row data frame. Finally, we combine the elements back into a data frame again.

```
d2 <- split(d, d$month)                        # Split data frame by month.
d2 <- lapply(d2, function(x) {                 # Apply summaries to each group.
  data.frame(month     = x$month[1],
             avg_delay = mean(x$delay),
             n         = nrow(x),
             n_ontime  = sum(x$ontime))
})
d2 <- do.call(rbind,                            # Combine groups into data frame.
              c(d2, make.row.names = FALSE))

head(d2)
```

```
       month avg_delay  n n_ontime
1 2016-01-01  23.85714  7        7
2 2016-02-01  24.45455 11        9
3 2016-03-01  22.45455 11       10
4 2016-04-01  22.66667  9        9
5 2016-05-01  22.50000  8        8
6 2016-06-01  22.00000  5        4
```

```
str(d2)
```

```
'data.frame':   24 obs. of  4 variables:
 $ month    : Date, format: "2016-01-01" "2016-02-01" ...
 $ avg_delay: num  23.9 24.5 22.5 22.7 22.5 ...
 $ n        : int  7 11 11 9 8 5 7 12 7 12 ...
 $ n_ontime : int  7 9 10 9 8 4 5 12 6 11 ...
```

The three functions split(), lapply(), and do.call() may be unfamiliar to many R users, but it pays to get to know them – read the documentation.

The split-apply-combine strategy may be performed a lot easier with functions from the *tidyverse* or *data.table* packages. But, as mentioned, we find it useful to know the base R ways of doing things, not the least to maintain independence from external packages when creating our own functions.

C.5 Tips and tricks

C.5.1 Cutting dates and datetimes

In the C-section data set the month variable represents the time period of each delivery used for subgrouping data, that is counting events and cases per month. The month variable was created by "cutting" the datetime variable into monthly chunks.

Converting dates and datetimes into time period – for example weeks, months or quarters – is a common task, not the least in SPC. It is easily done using the cut() function, which is a generic function with many uses. However, the syntax for cutting dates is poorly documented. To cut dates we use two arguments: first, the vector of date(time)s to be cut; second, the interval we want.

```
# R's date of birth (using parentheses to print the result)
(x <- as.Date('2000-02-29'))
```

```
[1] "2000-02-29"
```

```
# The first day of the month of R's date of birth
cut(x, breaks = 'month')
```

```
[1] 2000-02-01
Levels: 2000-02-01
```

Notice that cut() returns a factor. We need to convert it back to a date:

```
as.Date(cut(x, breaks = 'month'))
```

```
[1] "2000-02-01"
```

The breaks argument accepts a character string that qualifies for a time period, for example:

```
# Cutting to the first day of a week (monday)
as.Date(cut(x, breaks = 'week'))
```

```
[1] "2000-02-28"
```

```
# Quarter
as.Date(cut(x, breaks = 'quarter'))
```

```
[1] "2000-01-01"
```

We can even specify multiples of time periods in the breaks argument:

```
# The first day of biweekly periods
as.Date(cut(x, breaks = '2 weeks'))
```

```
[1] "2000-02-28"
```

Cutting dates is such a common task that we may want to make our own function for the purpose:

```
cutdate <- function(x, breaks = 'month') {
  as.Date(cut(x, breaks = breaks))
}

cutdate(x)
```

```
[1] "2000-02-01"
```

```
cutdate(x, 'week')
```

```
[1] "2000-02-28"
```

```
cutdate(x, 'quarter')
```

```
[1] "2000-01-01"
```

C.5.2 Getting age from date of birth

In healthcare we often need to know the patient's age on a certain date. By "age" we usually mean a whole number of years since the patient's date of birth. However, year is not a well defined unit of time, and age is really a continuous variable. Therefore, we are usually better off knowing the patient's age in days, which in turn can always be converted to decimal or integer years if necessary.

How old is R today?

```
today <- as.Date('2025-04-24')
r_dob <- as.Date('2000-02-29')

# calculate age in days
(age_in_days <- today - r_dob)
```

```
Time difference of 9186 days
```

When subtracting dates we get back a difftime object. Usually we want to convert this to an integer:

```
# convert from difftime object to integer:
(age_in_days <- as.integer(age_in_days))
```

```
[1] 9186
```

A calendar year is on average a little less than 365.25 days. We can converts age in days to age in years:

```
(age_in_years <- age_in_days / 365.25)
```

```
[1] 25.1499
```

Putting everything together:

```
(age_in_years <- as.integer(today - r_dob) / 365.25)
```

```
[1] 25.1499
```

Notice that this calculation may be off by up to one day depending on the number and position of leap years. However, this is far better than relying on age expressed in whole years.

C.5.3 Naming files and variables

Book have been written about how to and how not to name variables and files. Here is our take on a minimal list of dos and don'ts. Feel free to make your own naming style – and stick to it. Mixing styles is a source of confusion not only for others but also for your future self.

Always:

- use lower case ascii letters (a-z) and integers (0-9), e.g. "myvariable", "myfile.R";
- separate words with "-", "_", or".", e.g.: "my_variable", "my-variable", "my.variable";
- use ISO format (yyyy-mm-dd) for dates, e.g.: "2001-02-03";
- when numbering files, use use a suitable number of leading zeros, e.g. "myfile_01", "myfile_02", … "myfile_10", "myfile_11".

Newer:

- use spaces, e.g.: "my variable", "my file.R"; instead, use underscores, e.g.: my_file.R;
- versionise files using words like "final", "finalfinal", "new", "draft", etc.; instead, use ISO dates, e.g.: "my_file_2001-02-03.R".

Appendix D

Critical Values for Runs and Crossings

In SPC charts, a **run** is one or more consecutive data points on the same side of the centre line. Data points that lie directly on the centre line are not counted, nor do they break a run if the next data point is on the same side as the previous one.

A **crossing** occurs when two consecutive data points lie on opposite sides of the centre line, ignoring any data points directly on the centre line.

To perform the runs analysis, first count the number of useful observations in the chart. Then count the number of data points in the longest run and the number of crossings, and compare these values with the critical values in the table.

For example, in an SPC chart with 24 useful observations, a run of *more* than eight data points on the same side of the centre line or *fewer* than eight crossings of the centre line would suggest one or more sustained shifts in the data.

In random data sequences, these two rules used together have a false signal rate of around 5%, regardless of the number of useful observations (Anhøj and Olesen 2014; Anhøj 2015; Anhøj and Wentzel-Larsen 2018).

Useful observations	Longest run (upper limit)	Number of crossings (lower limit)
10	6	2
11	6	2
12	7	3

(continued)

Useful observations	Longest run (upper limit)	Number of crossings (lower limit)
13	7	3
14	7	4
15	7	4
16	7	4
17	7	5
18	7	5
19	7	6
20	7	6
21	7	6
22	7	7
23	8	7
24	8	8
25	8	8
26	8	8
27	8	9
28	8	9
29	8	10
30	8	10
31	8	11
32	8	11
33	8	11
34	8	12
35	8	12
36	8	13
37	8	13
38	8	14
39	8	14
40	8	14
41	8	15
42	8	15
43	8	16
44	8	16
45	8	17
46	9	17
47	9	17
48	9	18
49	9	18
50	9	19

(continued)

Useful observations	Longest run (upper limit)	Number of crossings (lower limit)
51	9	19
52	9	20
53	9	20
54	9	21
55	9	21
56	9	21
57	9	22
58	9	22
59	9	23
60	9	23
61	9	24
62	9	24
63	9	25
64	9	25
65	9	25
66	9	26
67	9	26
68	9	27
69	9	27
70	9	28
71	9	28
72	9	29
73	9	29
74	9	29
75	9	30
76	9	30
77	9	31
78	9	31
79	9	32
80	9	32
81	9	33
82	9	33
83	9	34
84	9	34
85	9	34
86	9	35
87	9	35
88	9	36

(continued)

Useful observations	Longest run (upper limit)	Number of crossings (lower limit)
89	9	36
90	9	37
91	10	37
92	10	38
93	10	38
94	10	39
95	10	39
96	10	39
97	10	40
98	10	40
99	10	41
100	10	41

References

Allaire, J. J., Charles Teague, Carlos Scheidegger, Yihui Xie, Christophe Dervieux, and Gordon Woodhull. 2025. *Quarto*. V. 1.8. Released September. https://doi.org/10.5281/zenodo.5960048.

Anhøj, Jacob. 2015. "Diagnostic Value of Run Chart Analysis: Using Likelihood Ratios to Compare Run Chart Rules on Simulated Data Series." *PLoS ONE*, ahead of print. https://doi.org/10.1371/journal.pone.0121349.

Anhøj, Jacob. 2018. "qicharts2: Quality Improvement Charts for R." *JOSS*, ahead of print. https://doi.org/10.21105/joss.0069.

Anhøj, Jacob. 2024. *Qicharts2: Quality Improvement Charts*. https://CRAN.R-project.org/package=qicharts2.

Anhøj, Jacob, and Anne Vingaard Olesen. 2014. "Run Charts Revisited: A Simulation Study of Run Chart Rules for Detection of Non-Random Variation in Health Care Processes." *PLoS ONE*, ahead of print. https://doi.org/10.1371/journal.pone.0113825.

Anhøj, Jacob, and Tore Wentzel-Larsen. 2018. "Sense and Sensibility: On the Diagnostic Value of Control Chart Rules for Detection of Shifts in Time Series Data." *BMC Medical Research Methodology*, ahead of print. https://doi.org/10.1186/s12874-018-0564-0.

Carey, Raymond G. 2002. "How Do You Know That Your Care Is Improving? Part 2: Using Control Charts to Learn from Your Data." *J Ambulatory Care Manage* 25(2): 78–88.

Chang, Winston, Joe Cheng, JJ Allaire, et al. 2026. *Shiny: Web Application Framework for r*. https://doi.org/10.32614/CRAN.package.shiny.

Chen, Zhenmin. 2010. "A Note on the Runs Test." *Model Assisted Statistics*

and Applications 5: 73–77. https://doi.org/10.3233/MAS-2010-0142.

Davis, Robert B, and William H Woodall. 1988. "Performance of the Control Chart Trend Rule Under Linear Shift." *Journal of Quality Technology* 20: 260–62.

Deming, W. Edwards. 1986. *Out of the Crisis*. MIT Press.

Goldratt, Eliyahu M., and Jeff Cox. 2022. *The Goal: A Process of Ongoing Improvement, 3rd Edition*. Routledge.

Jeppegaard, Maria, Steen C. Rasmussen, Jacob Anhøj, and Lone Krebs. 2023. "Winter, Spring, Summer or Fall: Temporal Patterns in Placenta-Mediated Pregnancy Complications—an Exploratory Analysis." *Gynecol Obstet* 309: 1991–98. https://doi.org/10.1007/s00404-023-07094-6.

Koetsier, A., S. N. van der Veer, K. J. Jager, N. Peek, and N. F. de Keizer. 2012. "Control Charts in Healthcare Quality Improvement." *Methods of Information in Medicine* 51: 189–98. https://doi.org/10.3414/ME11-01-0055.

Laney, David B. 2002. "Improved Control Charts for Attributes." *Quality Engineering* 14: 531–37.

Langley, Gerald J, Ronald D Moen, Kevin M Nolan, Thomas W Nolan, Clifford L Norman, and Lloyd P Provost. 2009. *The Improvement Guide*. Jossey-bass.

Lilford, Richard, Mohammed A Mohammed, David Spiegelhalter, and Richard Thomson. 2004. "Use and Misuse of Process and Outcome Data in Managing Performance of Acute Medical Care: Avoiding Institutional Stigma." *The Lancet* 363: 1147–54. https://doi.org/10.1016/S0140-6736(04)15901-1.

Mohammed, M A. 2024. *Statistical Process Control*. Elements of Improving Quality and Safety in Healthcare. Cambridge University Press. https://www.cambridge.org/core/elements/statistical-process-control/60B6025BF62017A9A203960A9E223C10.

Mohammed, M A, Anthony Rathbone, Paulette Myers, Divya Patel, Helen Onions, and Andrew Stevens. 2004. "An Investigation into General Practitioners Associated with High Patient Mortality Flagged up Through the Shipman Inquiry: Retrospective Analysis of Routine Data." *BMJ* 328 (7454): 1474–77.

https://doi.org/10.1136/bmj.328.7454.1474.

Mohammed, M A, P Worthington, and W H Woodall. 2008. "Plotting Basic Control Charts: Tutorial Notes for Healthcare Practitioners." *BMJ Qual Saf* 17 (2): 137–45. https://doi.org/10.1136/qshc.2004.012047.

Mohammed, Mohammed A, Jagdeep S Panesar, David B Laney, and Richard Wilson. 2013. "Statistical Process Control Charts for Attribute Data Involving Very Large Sample Sizes: A Review of Problems and Solutions." *BMJ Quality & Safety* 22 (4): 362–68. https://doi.org/10.1136/bmjqs-2012-001373.

Montgomery, Douglas C. 2020. *Introduction to Statistical Quality Control, Eighths Ed.* Wiley.

Nelson, Lloyd S. 1982. "Control Charts for Individual Measurements." *Journal of Quality Technology* 14 (3): 172–73. https://doi.org/10.1080/00224065.1982.11978811.

Neuburger, Jenny, Kate Walker, Chris Sherlaw-Johnson, Jan van der Meulen, and David A Cromwell. 2017. "Comparison of Control Charts for Monitoring Clinical Performance Using Binary Data." *BMJ Quality & Safety* 26 (11): 919–28. https://doi.org/10.1136/bmjqs-2016-005526.

Perla, Rocco J, Lloyd P Provost, and Sandy K Murray. 2011. "The Run Chart: A Simple Analytical Tool for Learning from Variation in Healthcare Processes." *BMJ Qual Saf* 20: 46–51. https://doi.org/10.1136/bmjqs.2009.037895.

Plessen, Christian von, Anne Marie Kodal, and Jacob Anhøj. 2012. "Experiences with Global Trigger Tool Reviews in Five Danish Hospitals: An Implementation Study." *BMJ Open* 2 (5). https://doi.org/10.1136/bmjopen-2012-001324.

Rogers, Chris A., Barnaby C. Reeves, Massimo Caputo, J. Saravana Ganesh, Robert S. Bonser, and Gianni D. Angelini. 2004. "Control Chart Methods for Monitoring Cardiac Surgical Performance and Their Interpretation." *The Journal of Thoracic and Cardiovascular Surgery* 128: 811–19. https://doi.org/10.1016/j.jtcvs.2004.03.011.

Schilling, Mark F. 2012. "The Surprising Predictability of Long Runs." *Mathematics Magazine* 85: 141–49. https://doi.org/10.4169/math.mag.85.2.141.

Shewhart, Walther A. 1931. *Economic Control of Quality of Manufactured*

Product. D. Van Nostrand Company.

Spiegelhalter, David J. 2005. "Funnel Plots for Comparing Institutional Performance." *Statistics in Medicine* 24 (8): 1185–202. https://doi.org/10.1002/sim.1970.

Taylor, Wayne. 2018. *Normalized Individuals (IN) Control Chart*. https://variation.com/normalized-individuals-control-chart/.

Thor, Johan, Jonas Lundberg, Jakob Ask, et al. 2007. "Application of Statistical Process Control in Healthcare Improvement: Systematic Review." *BMJ Qual Saf* 16: 387–99. https://doi.org/10.1136/qshc.2006.022194.

Western Electric Company. 1956. *Statistical Quality Control Handbook*. Western Electric Company inc.

Wheeler, Donald J. 2000. *Understanding Variation – the Key to Managing Chaos*. SPC Press.

Wheeler, Donald J. 2010. "The Right and Wrong Ways of Computing Limits." https://www.spcpress.com/pdf/DJW205.pdf.

Wheeler, Donald J. 2016. "The Levey-Jennings Chart – How to Get the Most Out of Your Measurement System." http://www.spcpress.com/pdf/DJW290.pdf.

Yihui Xie, Garrett Grolemund, J. J. Allaire. 2023. *R Markdown: The Definitive Guide*. Chapman & Hall/CRC.